AI Agents Mastery Program — Beginner End-to-End Notes

A plain, beginner-friendly transcription of the course notes. Every section below is drawn only from the 55 extracted transcripts in this folder.

Table of Contents

Demo 1: What Is an AI Agent? (Concepts, Examples, and Why Automation Matters) 9
What you'll learn 9
Step-by-step notes 9
Key takeaways 11
Demo 2: Introduction to Vibe Coding with Google AI Studio 12
What you'll learn 12
Step-by-step notes 12
Key takeaways 14
Demo 3: More Vibe Coding Practice — Image Transformation and Legal Apps with Google AI Studio 15
What you'll learn 15
Step-by-step notes 15
Key takeaways 17
Demo 4: AI vs. Generative AI vs. Agentic AI — Clearing Up the Confusion 18
What you'll learn 18
Step-by-step notes 18
Key takeaways 20
Demo 5: AI Agent Architecture, Multi-Agent Systems, Memory, and the Automation Mindset 21
What you'll learn 21
Step-by-step notes 21
Key takeaways 23
Demo 6: What Is Automation, and What Is N8N? 25
What you'll learn 25
Step-by-step notes 25

Key takeaways 27
Demo 7: Building Your First AI Agent in N8N 28
What you'll learn 28
Step-by-step notes 28
Key takeaways 30
Demo 8: Understanding the N8N Dashboard — Canvas, Nodes, Configuration, and Logs 32
What you'll learn 32
Step-by-step notes 32
Key takeaways 33
Demo 9: Controlling Agent Behavior with System Messages, and Building an Email Automation Agent 34
What you'll learn 34
Step-by-step notes 34
Key takeaways 36
Demo 10: Building a Form-Based Feedback Automation (No AI Needed) 38
What you'll learn 38
Step-by-step notes 38
Key takeaways 40
Demo 11: Building a Loan Eligibility Check Automation in n8n 41
What you'll learn 41
Step-by-step notes 41
Key takeaways 44
Demo 12: Saving Form Feedback to Google Sheets, Throttling with the Wait Node, and Sending a Thank-You Email 46
What you'll learn 46
Step-by-step notes 46
Key takeaways 50
Demo 13: Sending Feedback Form Attachments via Gmail and Google Drive in n8n 52
What you'll learn 52
Step-by-step notes 52
Key takeaways 55
Demo 14: Using OpenRouter for Free/Multiple LLM Models with a Gmail-Triggered AI Agent 57

What you'll learn 57
Step-by-step notes 57
Key takeaways 60
Demo 15: Building a Resume Screening AI Agent (Job Filtering Workflow) in n8n 62
What you'll learn 62
Step-by-step notes 62
Key takeaways 66
Demo 16: Fixing Resume Attachments with n8n's Merge Node, and Building a Scheduled "Message Generator" Workflow 68
What you'll learn 68
Step-by-step notes 68
Key takeaways 70
Demo 17: What You Need Before You Can Build a WhatsApp Bot with n8n (Meta/Facebook Setup Overview) 72
What you'll learn 72
Step-by-step notes 72
Key takeaways 74
Demo 18: Live Troubleshooting of the WhatsApp Trigger Connection in n8n 75
What you'll learn 75
Step-by-step notes 75
Key takeaways 77
Demo 19: Successfully Completing the WhatsApp Bot — Full Recap, Dynamic Setup, and Making the Access Token Permanent 79
What you'll learn 79
Step-by-step notes 79
Key takeaways 82
Demo 20: Getting Real-Time Data into AI Agents with the Tavily Node (Live Score, Gold Price, and News Alerts) 83
What you'll learn 83
Step-by-step notes 83
Key takeaways 86
Demo 21: Discovering and Reusing n8n's Ready-Made Workflow Library 87

What you'll learn 87
Step-by-step notes 87
Key takeaways 89
Demo 22: Reviewing Core n8n Concepts Through a 15-Question MCQ Quiz 90
What you'll learn 90
Step-by-step notes 90
Key takeaways 92
Demo 23: Life Purpose (Ikigai) and Building a Multi-Category Feedback Workflow with AI Classification (Part 1) 93
What you'll learn 93
Step-by-step notes 93
Key takeaways 95
Demo 24: Completing the Feedback Classification Workflow — Switch Routing, Multi-Sheet Logging, Email Replies, Slack Notifications, and Parallel Execution 96
What you'll learn 96
Step-by-step notes 96
Key takeaways 98
Demo 25: Understanding Webhooks — Connecting External Applications to n8n 99
What you'll learn 99
Step-by-step notes 99
Key takeaways 101
Demo 26: Understanding Webhooks and Connecting a Local HTML App to n8n 102
What you'll learn 102
Step-by-step notes 102
Key takeaways 104
Demo 27: Building a Student Feedback Form Connected to n8n (Webhook + AI Agent + Gmail + Google Sheets) 106
What you'll learn 106
Step-by-step notes 106
Key takeaways 109
Demo 28: Building a Hospital Appointment Booking Automation with n8n Forms 110

What you'll learn 110
Step-by-step notes 110
Key takeaways 113
Demo 29: Understanding RAG and MCP (Model Context Protocol) — Concept Session 114
What you'll learn 114
Step-by-step notes 114
Key takeaways 116
Demo 30: Understanding LangChain — Concept Session 117
What you'll learn 117
Step-by-step notes 117
Key takeaways 119
Demo 31: LangChain vs LangGraph vs n8n — Understanding the Framework Landscape 121
What you'll learn 121
Step-by-step notes 121
Key takeaways 123
Demo 32: Reviewing a Student-Built Hospital Appointment Workflow in n8n (with Code Node Logic) 124
What you'll learn 124
Step-by-step notes 124
Key takeaways 125
Demo 33: Understanding RAG (Retrieval-Augmented Generation) — Concepts and Self-Test 127
What you'll learn 127
Step-by-step notes 127
Key takeaways 128
Demo 34: Building a Smarter Hospital Appointment Workflow — Live Walkthrough of a Student's Solution 130
What you'll learn 130
Step-by-step notes 130
Key takeaways 132
Demo 35: Telegram Bot Automation in n8n — Building a Course Info & Interview-Question Bot 133
What you'll learn 133
Step-by-step notes 133

Key takeaways 136
Demo 36: Building a Telegram Bot in n8n with an AI Agent (Course Recommender) + Two Student Project Reviews 137
What you'll learn 137
Step-by-step notes 137
Key takeaways 139
Demo 37: Telegram Callback Queries — Building Button-Driven Menus in n8n 141
What you'll learn 141
Step-by-step notes 141
Key takeaways 142
Demo 38: Prompt Chaining in n8n — Multi-Agent Workflows for Better Quality Output 144
What you'll learn 144
Step-by-step notes 144
Key takeaways 146
Demo 39: Routing in n8n — Auto-Labeling Gmail with a Text Classifier 147
What you'll learn 147
Step-by-step notes 147
Key takeaways 149
Demo 40: Extending Routing — Auto-Reply and Telegram Forwarding by Category, Plus a Student's Scalable Metadata-Driven Design 150
What you'll learn 150
Step-by-step notes 150
Key takeaways 152
Demo 41: Parallelism in n8n — Running Multiple AI Agents at the Same Time 153
What you'll learn 153
Step-by-step notes 153
Key takeaways 154
Demo 42: Merge vs Aggregate Deep Dive, Plus Two Real Student Projects 156
What you'll learn 156
Step-by-step notes 156
Key takeaways 158

Demo 43: Introducing the Evaluate–Optimize (Evolve-Optimizer) Framework and Looping in n8n 159
What you'll learn 159
Step-by-step notes 159
Key takeaways 161
Demo 44: Live Debugging Session — Fixing the Retry-Counter Bug in the Evaluate-Optimize Loop 162
What you'll learn 162
Step-by-step notes 162
Key takeaways 163
Demo 45: Fixing the Retry-Counter Bug, Recapping the 4 n8n Patterns, and Introducing MCP 165
What you'll learn 165
Step-by-step notes 165
Key takeaways 168
Demo 46: Understanding MCP (Model Context Protocol) and Connecting ChatGPT to n8n 169
What you'll learn 169
Step-by-step notes 169
Key takeaways 171
Demo 47: Connecting Lovable (No-Code App Builder) to n8n — Via MCP and Via Webhook 172
What you'll learn 172
Step-by-step notes 172
Key takeaways 174
Demo 48: MCP with Xfield (AI Video Generation) + Introduction to RAG (Retrieval Augmented Generation) 176
What you'll learn 176
Step-by-step notes 176
Key takeaways 178
Demo 49: RAG Deep Dive — Embeddings, Vector Databases, Semantic Search, and Building a RAG Pipeline in n8n 180
What you'll learn 180
Step-by-step notes 180
Key takeaways 183

Demo 50: RAG in Practice — Chunk Size, Overlap, Embedding Batch Size Explained with a Cricket Rules Example 184
What you'll learn 184
Step-by-step notes 184
Key takeaways 187
Demo 51: Life Lessons on Systems, Consistency, and Delegation (Mindset Session) 189
What you'll learn 189
Step-by-step notes 189
Key takeaways 190
Demo 52: Building a RAG Workflow in n8n Using Supabase as the Vector Database 192
What you'll learn 192
Step-by-step notes 192
Key takeaways 194
Demo 53: Building a Voice-to-Voice AI Agent in n8n Using Telegram and ElevenLabs 196
What you'll learn 196
Step-by-step notes 196
Key takeaways 198
Demo 54: Auto-Generating n8n Workflows with n8n's Built-In AI Assistant 199
What you'll learn 199
Step-by-step notes 199
Key takeaways 201
Demo 55: Self-Hosting n8n on Hostinger, Plus Community Forum Resources 202
What you'll learn 202
Step-by-step notes 202
Key takeaways 204

Demo 1: What Is an AI Agent? (Concepts, Examples, and Why Automation Matters)

This first session introduces the AI Agents Mastery Program and explains, in plain language, what an "AI agent" actually is, why businesses want them, and how an agent differs from a simple chatbot.

What you'll learn

- What an AI agent is, in simple terms
- The difference between a static auto-reply and a true AI agent
- Why companies and individuals want to automate repetitive work
- The basic "flow" an agent follows to complete a task
- Real-world examples of AI agents already in use (customer support, email, resume screening, market research)
- How to write a simple prompt that makes an AI act like an agent

Step-by-step notes

1. **The big idea: AI agents are digital assistants that work automatically.**

The instructor defines an AI agent as a computer system that acts like a smart assistant — it can do tasks on your behalf (personal or business) without you manually doing them every time. If something is done manually today (reading emails, replying to customers, screening resumes, gathering market data), and you build a system to do it automatically, that system is an AI agent.

2. **Example: automating business email.**

A small business gets many emails every day — customer questions, job applications, meeting requests, marketing messages. Instead of a person reading and replying to each one, you can build a "smart assistant" (an AI agent) that:

- Reads all incoming emails
- Replies to customer questions
- Sends interview invitations
- Collects market information

The benefit: time is saved, and responses happen instantly instead of after hours or days.

3. **AI agent vs. a human assistant.**

The instructor contrasts human assistants (who make mistakes, need breaks, holidays, salary, and are affected by emotions/mood) with an AI agent (works 24/7, doesn't get tired, doesn't need a salary, doesn't have "bad days"). This is why companies prefer automation for repetitive work.

4. **AI agent vs. a simple "auto-reply."**

An auto-reply (like a WhatsApp "I'm currently away" message) always sends the *same* static message — it is not an AI agent. A true AI agent *reads and analyzes* the incoming message and generates a *dynamic*, context-appropriate response, similar to how a human would respond. The key difference: static vs. dynamic response.

5. **Why AI agents are needed (the core problem they solve).**

People waste a lot of time on tasks that are:

- Repetitive
- Time-consuming
- Boring

These are exactly the tasks that AI agents are good at automating. Examples given: a student searching for information, summarizing articles, and preparing notes; a business handling customer support, lead generation, and resume screening; a content team finding trending topics, writing scripts, and producing videos.

6. **The advantage of automating with AI agents:** saves time, money, and energy/effort. The instructor stresses that even if your company isn't using AI yet, you personally are still responsible for your own repetitive tasks (reports, Excel updates, replying to emails) — you can build your own small AI agent to reduce that workload, even without company buy-in.
7. **Real-world examples already in production, as described in the session:**

- **Customer support** — companies use AI agents to read customer messages, understand the problem, and send replies automatically (e.g., chat support on platforms like Flipkart/Amazon/Swiggy/Zomato, and even phone-based customer care agents that identify themselves as AI when asked).
- **Email automation** — AI reads emails, categorizes them, and sends automatic replies.
- **Resume screening** — AI reads resumes, analyzes skills, shortlists candidates, and can even conduct the first round of interviews before a human takes over for later rounds.
- **Market research** — AI agents collect information about competitors, market trends, and customer opinions, then summarize it into a report — work that used to take many hours of manual research.

8. **The basic "flow" an AI agent follows** (this is the mental model to remember):
9. Receive user request/instruction
10. Understand/think about the task
11. Select the required tool(s), if needed (e.g., internet search, database, email system)
12. Execute the task
13. Return the result (send a report, reply by email, message on WhatsApp, etc.)
14. **The most important distinction: AI agent vs. chatbot.**

A chatbot only answers questions/generates responses for conversation. An AI agent *performs actions* — it does something in the real world (sends an email, books something, creates a video), not

just answers text. When you add "action" capability to generative AI, that combination is called agentic AI.

15. **Example prompt to make an AI act like a simple agent** (shown live in ChatGPT):

"Act as a market research agent. Find the top 5 electric vehicle companies in the world. Provide a short summary of each company."

This single prompt makes the AI search, gather, and summarize information that would otherwise take a person a full day or more to research manually. You can follow up by asking it to analyze the data further (e.g., "analyze strengths/weaknesses and suggest where a new EV company should focus"), and the AI will keep helping step by step.

Key takeaways

- An AI agent = a digital assistant that can receive instructions, think about the task, use tools if needed, and perform an action automatically — not just answer a question.
- A chatbot only answers; an AI agent *does* things.
- A static auto-reply message is not an AI agent — a true agent analyzes and responds dynamically.
- AI agents are useful for repetitive, time-consuming, boring tasks — for individuals and businesses alike, not just large corporations.
- The basic agent flow: receive request → understand/think → use tools if needed → act → return result.
- Well-written prompts (e.g., "Act as a ___ agent, do X, provide Y") can already make general AI tools behave like simple agents.

Demo 2: Introduction to Vibe Coding with Google AI Studio

This session is a live demo of "vibe coding" — building working web apps just by describing what you want in plain English, without writing any code, using Google AI Studio (Gemini).

What you'll learn

- What "vibe coding" means and why it matters
- How to access and use Google AI Studio to build an app from a text prompt
- How to iterate on an app by simply changing your prompt (not the code)
- How to fix bugs/errors in a generated app using plain-English instructions
- The limits and considerations (API keys, cost, publishing, databases) when building apps this way
- Why the instructor believes traditional hand-written coding is becoming far less necessary

Step-by-step notes

16. **What is vibe coding?**

Instead of writing code (choosing a language, learning syntax, writing logic), you simply describe your app idea in natural language, and an AI tool (in this demo, Google AI Studio / Gemini) writes the full application for you. You act like a "director," guiding the AI with your ideas rather than writing code yourself.

17. **Accessing the tool.**

Go to **ai.dev** (a short URL) which redirects to **aistudio.google.com**. In the interface, choose the **Build** option (a chat-style interface: "Describe an app and let Gemini build it"). You type a description of the app you want; Gemini generates the app.

18. **Demo app 1: a "funny astrology" web app.**

Prompt used (paraphrased): *"Build an astrology web application where the user has to enter name, date of birth, and gender. The app should provide a funny, user-specific astrology conclusion, and also tell when the user is going to get married (date and time), plus provide a sample image of their life partner."*

- Gemini (model shown as "Gemini 3.1 Pro preview") builds the app in under 2 minutes, generating files like `metadata.json`, `App.layout.tsx`, etc.
- The app can immediately be tested by entering sample details — it produced humorous astrology text and an AI-generated "life partner" image.
- Iteration: when the response used overly complex English, the instructor simply added to the prompt: *"The response should be in very simple English and should be short. The life partner image should be more realistic and should be Indian."* Sending this new instruction in the chat window automatically rebuilt/updated the app — no code was touched.

- Lesson stressed repeatedly: **if the app doesn't do what you want, you change the prompt, not the code.**

19. **UI changes are also just prompts.**

To change the app's look and feel, the instructor typed: *"Please change UI with rich bright colors, like black and gold."* The app's colors updated automatically ("revising color scheme").

20. **Demo app 2: a "Telugu educational video generator."**

Prompt (paraphrased): *"Create an app where the user has to provide a topic name and video duration. Your job is to create a video explaining that concept in Telugu, suitable for beginners."*

- This is more complex because it needs video/audio generation, so it required a **Gemini API key** (already set up in the instructor's Google Cloud Platform/GCP account).
- Selecting "Google Gemini API key" inside AI Studio and choosing the linked project allowed the app to generate a real video with Telugu narration and script.
- When an error appeared ("element has not supported sources"), the instructor clicked the **"Fix"** option in the interface, and AI Studio automatically corrected the code — again, no manual code editing.

21. **API keys and cost.**

- If your app makes no API calls (e.g., simple text/image generation handled inside AI Studio), it's generally free to use and share.
- If the app needs external API calls (e.g., video/audio generation via models like **Veo 3**), an API key is required, and usage may be billed (someone in the chat mentioned loading around $5 to their account for API usage).
- You create the Gemini/Google API key through your Google Cloud Platform (GCP) account.

22. **Publishing and running apps.**

- Once you're happy with an app, you can **Publish** it to get a shareable link (hosted on Google's cloud, e.g., via Cloud Run) — anyone with the link can use it.
- You can also **download the full generated code** and run it locally, but then you become responsible for setting up your own database and any backend infrastructure yourself (AI Studio doesn't manage that for you if you go local).
- AI Studio also supports selecting different frameworks (React, Angular, Next.js) and choosing database support directly from its settings — this capability keeps expanding.

23. **Development approach: build in phases.**

For bigger apps, don't ask for everything at once. Build phase by phase (e.g., "first the home page," then "add a products page," then "add billing") similar to how development sprints work — check each phase works before adding the next feature.

24. **When something doesn't work / fixing bugs.**

It's normal for the first generation not to fully meet your requirements. Simply describe the problem back to the AI in the chat (e.g., "the audio isn't in Telugu, please fix that") and let it regenerate — repeat as needed, sometimes 2–4 iterations, until it's right.

25. **Broader discussion: is coding as a skill becoming obsolete?**

The instructor argues that with vibe coding, deep knowledge of specific programming languages (Java, Python, etc.) is becoming much less critical for building applications — what matters most is being able to describe requirements clearly (i.e., prompting skill). He notes that in modern software teams, when something breaks, engineers increasingly feed the error back to the AI and ask it to fix the issue, rather than manually debugging line by line. Other vibe-coding tools mentioned by name in the discussion: **Claude, GitHub Copilot, Cursor, Factory AI, Codex.**

Key takeaways

- Vibe coding = describing your app idea in plain language and letting an AI (here, Google AI Studio / Gemini) generate the working app for you.
- To fix or change an app, change your **prompt**, not the code.
- Simple apps (no external API calls) can usually be built and shared for free; apps needing services like video generation require an API key and may incur cost.
- Build complex apps in phases/iterations rather than asking for everything at once.
- You can publish an app to get a shareable link, or download the code to run locally (but then you must handle your own database/backend setup).
- Prompting/idea clarity is becoming the core skill — the instructor sees this as a fundamental shift in how software gets built.

Demo 3: More Vibe Coding Practice — Image Transformation and Legal Apps with Google AI Studio

A continuation of the vibe coding demo, focused on more advanced app ideas (photo/age/style transformation apps and a legal-assistant app), plus practical tips on troubleshooting, API keys, and thinking of app ideas for real use cases.

What you'll learn

- How to browse pre-built example apps inside Google AI Studio's "Gallery"
- How to build a photo-transformation app (age progression / country-style transformation) from a prompt
- Common real-world hiccups (API quota limits, key selection, project setup) and how to handle them
- How database and hosting work when you publish an AI Studio app
- A mindset shift: "ask ChatGPT/AI first" instead of asking a person, for almost any how-to problem
- A worked example of a legal-help app suggesting applicable law sections and jurisdiction

Step-by-step notes

26. **Browsing the built-in app Gallery.**

Inside AI Studio's Build section, there's a **Gallery** tab showing pre-made example apps (categories like Gemini 3, Nano Banana, games & visualization, gen media, multimodal code gen). The instructor opens an app called **"Past Forward"** (a "personal time machine" app) which lets you upload a photo and see yourself reimagined in past decades (1950s–2000s) with era-appropriate styling, using Gemini 2.5 Flash Image and "character consistency."

27. **Trying the Past Forward app.**

Upload a photo → click Generate → the app produces images of "you" styled for different decades (e.g., 1970, 1990, 2000, 1960). Occasionally it failed with errors like "failed to call Gemini API, user exceeded quota" — solved by waiting a minute and selecting the correct API key/project in Settings.

28. **Building a custom version: location/costume transformation app.**

Prompt built live (paraphrased): *"Build an app where the user has to upload a photo. The app should transfer the image into a location-specific / territory-specific style — user selects a location such as India, Iran, China, Russia, Canada, Afghanistan, Africa — and the image should be downloadable."*

- The app ("Global Persona") let users upload a photo and pick a country/territory; it reimagined the person in traditional/cultural attire associated with that country (e.g., "Kente cloth" style for Africa, "Hanfu/Cheongsam" for China), while noting it changed costume, not the person's underlying photo color/skin tone unless explicitly asked.
- Multiple class participants tried the same prompt and got working apps with slightly different names/results, showing that the exact same prompt won't always generate an identical app.

29. **Common real-world issues encountered live:**

- **API key / quota problems** — some participants were asked to select or create an API key, others weren't (depends on account state, free credits, or default project). Selecting "Default Gemini project" or creating a new API key/project in the settings usually resolved it.
- **"You are out of generations for this project"** — meant the free quota was used up; solution was to choose/create a different project or API key.
- When publishing an app, AI Studio may ask you to set up **billing information** before the link becomes shareable.

30. **Important safety note:** Don't share your AI Studio app URLs publicly/broadly. If you publish an app, anyone using it consumes **your** API key/quota, not their own — so uncontrolled public sharing can run up your usage/cost.
31. **Database and hosting explained:**

- If you keep the app running through AI Studio's own cloud (e.g., Cloud Run), the platform manages the backend/database complexity for you — you don't need to configure JDBC drivers or similar low-level plumbing; the app and its data storage are handled behind the scenes.
- If you instead download the generated app and want to publish it yourself (e.g., to the Google Play Store), you become responsible for setting up your own database and hosting.

32. **Case study demo: a legal-help app.**

A participant (a lawyer) built an app where you type a description of a case (e.g., "somebody stole a computer from my store") and a location (e.g., "Ameerpet, Hyderabad"). Using a "Smart Suggest" feature, the app suggested the applicable law section (e.g., referenced Section 305 under Bharatiya Nyaya Sanhita) and the correct court/jurisdiction to approach (e.g., Court of Metropolitan Magistrate for a theft matter). This showed how AI Studio apps can pull from publicly available reference information (like legal codes) without the developer needing to store that data themselves — you just reference it in the prompt.

33. **Framework and model options available in AI Studio (shown in Settings):**

- Model choice: Gemini 3 Flash preview, Gemini 3.1 Pro preview, Gemini 3.1 Flash Lite preview, etc.
- Frontend framework choice: React, Angular, Next.js (Next.js is noted as more powerful since it can also handle backend logic, not just frontend).
- Database support can also be selected directly.

34. **General mindset advice repeated throughout:**

Whenever you don't know how to do something technical (e.g., connecting an external webcam, configuring an app setting), **ask ChatGPT (or any AI) directly and follow its step-by-step instructions**, rather than calling a person for help or assuming something is "not possible." The instructor shared personal examples (connecting an external camera for Zoom, solving a smart-community app permission issue) that AI resolved in minutes.

35. **Building bigger/personal apps.**

You're not limited to apps for work — you can build entirely personal-use apps (e.g., "give me an optimized daily schedule based on my skills and goals for the next 10 days"). If a pre-built app on the Play Store/App Store doesn't fit your exact need, you can now simply build your own custom one via a prompt.

Key takeaways

- AI Studio's **Gallery** has ready-made example apps you can study or reuse as starting points.
- The same prompt can produce different results/app names for different users — don't expect exact reproducibility.
- API key, project selection, and billing setup are common practical stumbling blocks — usually solved by picking the right project/key in Settings.
- Never widely share a published AI Studio app URL — visitors use your API quota/cost, not their own.
- When hosted through AI Studio's own cloud, database/backend setup is handled for you; if you self-host (e.g., publish to Play Store), you must set up your own database.
- Default habit to build: when stuck on "how do I do X," ask an AI model directly and follow its steps, instead of asking a person or assuming it's impossible.

Demo 4: AI vs. Generative AI vs. Agentic AI — Clearing Up the Confusion

This session steps back from hands-on demos to clearly define and contrast three terms that are often confused: (normal/traditional) AI, Generative AI, and Agentic AI, and introduces the "Observe, Think, Act" loop that describes how agents work.

What you'll learn

- A clear definition of (traditional) Artificial Intelligence, with everyday examples
- A clear definition of Generative AI and how it differs from traditional AI
- A clear definition of Agentic AI and how it differs from Generative AI
- Common beginner misconceptions about AI, generative AI, and jobs
- The "Observe → Think → Act" loop that agentic AI follows
- Why this knowledge is valuable for spotting automation/business opportunities

Step-by-step notes

36. **What is (traditional) Artificial Intelligence?**

Definition given: *"Teaching machines to perform tasks that normally require human intelligence"* — such as understanding language, recognizing images, making decisions, and solving problems. AI systems learn these abilities from data (this learning process is called **machine learning** — training AI with huge amounts of data so it learns patterns).

Everyday examples given:

- **Google Maps** finding the best/shortest route based on traffic and road data
- **Gmail spam detection** automatically moving suspicious emails to the spam folder
- **Phone face unlock**
- **Amazon's "recommended products for you"**
- **Netflix's "recommended shows for you"**

These are all AI, but none of them *create new content* — they analyze data and predict/decide/recommend something that already exists.

37. **What is Generative AI?**

Generative AI is a **special subset of AI** that *creates new content* — text, images, video, audio, or code — rather than just analyzing existing data. Examples: ChatGPT, Gemini, Claude. Business uses mentioned: generating customer support responses, generating marketing content, generating code.

Key distinction drilled in: *AI and Generative AI are related, but not identical* — Generative AI is a specific category inside the broader field of AI. A common beginner mistake is assuming "AI" and "Generative AI" mean the same thing.

38. **Common misconception #1: "AI systems think like humans."**

False. AI systems only learn statistical **patterns from data** — they do not have real understanding like humans. The instructor references earlier sessions on transformer architecture, vector embeddings, and semantic distance to explain that what looks like "understanding" is really pattern-matching, not consciousness. AI can't love, taste, smell, or feel — it's a machine, not a person (though it can tirelessly answer the same question 100 times without getting irritated, unlike a human).

39. **Common misconception #2 (implied/continued): "AI will replace all jobs."**

The instructor's standard answer: *"AI won't replace you, but a person WITH AI knowledge will replace you."* The message is to build AI skills rather than fear AI itself.

40. **What is Agentic AI?**

Agentic AI is a **special case of Generative AI** that goes a step further: instead of just generating content or an answer, it **performs an action** to complete a task. Example contrast used repeatedly:

- *"What is the capital of India?"* → simple factual answer (plain AI).
- *"Find today's weather report in 5 bullet points"* → Generative AI (creates formatted new content).
- *"Find today's weather report in 5 bullet points AND send it to my email every morning at 6 AM"* → **Agentic AI** (it must perform an action — checking, formatting, and sending — not just generate text).
- Booking a restaurant table, or booking a movie ticket, requires **action** (checking availability, reserving), so these require agentic AI — a simple "list of good restaurants" only requires generative AI.
- Note: agentic AI does not always require *generating* new content — e.g., sending the same daily reminder image is still "agentic" because it performs a repeated action, even without generating anything new.

41. **Nesting of these concepts (important mental model):**

AI (broadest) → Generative AI (a special type of AI) → Agentic AI (a special type of Generative AI that can take actions). The instructor compares this to how Machine Learning and Deep Learning are nested subfields of AI in typical textbook diagrams.

42. **AI Agent vs. Agentic AI — a subtle distinction.**

- **"AI agent"** = an actual system/assistant you build to perform a specific activity (the "thing").
- **"Agentic AI"** = the underlying skill/capability/behavior of being able to take autonomous action (the "ability" or "knowledge area"). You use agentic AI *knowledge* to *build* AI agents. This entire course teaches "agentic AI" as a skillset, which you then use to build individual "AI agents."

43. **The "Observe, Think, Act" (OTA) loop — the heart of agentic AI.**

Example: *"Book a meeting tomorrow with my team between 10 AM and 5 PM."*

44. **Observe** — read/understand the request

45. **Think** — check calendar availability (this is also called "reasoning")
46. **Act** — schedule the meeting and send invitations

This loop can repeat multiple times until the task is fully completed (e.g., if a generated result — like an app from the earlier vibe-coding demos — doesn't meet requirements, you go around the loop again with corrections). This is sometimes also referred to as **ReAct prompting** (Reason + Act), a term from earlier in the course.

47. **Business framing: where to find automation opportunities.**

The instructor encourages thinking about any business (e.g., a hospital with 10 employees) in terms of: how much of daily operations (e.g., "87% of activities") could be automated, and the resulting cost/revenue impact. This is framed as a genuine business/freelancing opportunity — identifying repetitive processes in any business and proposing automation for a share of the resulting savings/profit.

48. **Homework/mindset exercise given:** Think about your own repetitive personal tasks (e.g., checking morning news, checking weather) and consider designing an AI agent to handle them automatically (e.g., getting 5 daily news headlines delivered automatically every morning at 6 AM) instead of doing it manually.

Key takeaways

- **Traditional AI**: analyzes data and makes decisions/predictions/recommendations (e.g., Google Maps, spam filters, recommendations) — does not create new content.
- **Generative AI**: a subset of AI that creates new content (text, images, video, audio, code) — e.g., ChatGPT, Gemini, Claude.
- **Agentic AI**: a subset of Generative AI that performs real actions/tasks (not just generates content) — e.g., booking, sending, scheduling.
- AI does not "think" like a human — it only recognizes patterns from training data.
- "AI agent" is the built system; "Agentic AI" is the underlying skill/capability of taking autonomous action.
- The core mechanism behind agent behavior is the **Observe → Think → Act loop**, repeated until the task is complete.
- A person with AI knowledge is positioned to replace/outcompete a person without it — not the AI itself replacing jobs directly.

Demo 5: AI Agent Architecture, Multi-Agent Systems, Memory, and the Automation Mindset

This session builds directly on Demo 4, going deeper into agent terminology (AI agent vs. AI agency/multi-agent systems), the four core building blocks of any AI agent's architecture, why agents need memory, and a practical step-by-step mindset for identifying what to automate — setting up for the upcoming hands-on n8n (N8N) automation platform sessions.

What you'll learn

- Recap: traditional AI vs. generative AI vs. agentic AI (nested relationship)
- The difference between a single "AI agent" and an "AI agency" (multi-agent system)
- The four main components of AI agent architecture: Input, Model, Tools, Action
- Why AI agents need memory, and the benefits memory provides
- A practical, repeatable method for deciding what tasks are worth automating
- Common beginner mistakes when planning automation projects
- A first mention of automation platforms (n8n, Zapier, Make) and concepts like nodes/workflows

Step-by-step notes

49. **Quick recap of the AI nesting model.**

Traditional/normal AI (trained on data, makes decisions/predictions) → Generative AI (special case that creates new content) → Agentic AI (special case of generative AI that also performs actions). Example reused: asking for tomorrow's weather report is traditional AI; asking for it summarized in 5 bullet points is generative AI; asking for it to be automatically emailed to you every day is agentic AI.

A student asks about **reinforcement learning**; the instructor explains it's a separate branch of machine learning that may be used *inside* agentic systems to help agents learn behavior over time (e.g., learning from past interactions/memory), but says this will be explored later — it is not yet fully covered here.

50. **AI Agent vs. AI Agency (Multi-Agent System).**

- **AI agent** = one system that receives instructions, analyzes the task, uses tools if needed, and performs an action — like a single digital assistant/worker.
- **AI agency** (also called a **multi-agent system**) = a *group of agents working together*. These terms are used interchangeably.
- Example: a customer support system might have separate agents for different jobs — an **email agent**, a **chatbot agent**, and a **ticket management agent** — working together as one overall system (an "agency").
- Example using the instructor's own training institute (Durgasoft): daily repetitive work includes student inquiries, fee-related questions, batch timing questions, follow-up calls, and WhatsApp messages (currently handled by ~30 non-technical staff). This could be split across multiple

specialized agents: an **inquiry response agent**, a **batch schedule agent**, a **follow-up reminder agent**, a **payment reminder agent**, and a **feedback collection agent** — together forming a multi-agent system / agency.

- Whether agents work independently or dependently (one agent's output feeding into another) depends on the specific use case — this becomes clearer once you actually build one.
- A brief discussion also touched on system reliability concerns (backup/redundant agents, load balancing, scalability) — acknowledged as important production concerns to be addressed later, similar to how backup web/app server instances work.

51. **AI Agent Architecture — the four core parts.**

Every AI agent is described as having four main components:

52. **Input** — the system receives the user's request (e.g., *"Find the cheapest flight from Hyderabad to Bangalore tomorrow"*).
53. **Model** — the "brain" of the agent; the AI model analyzes the request and understands what's being asked. This is where AI reasoning/thinking happens (also called "reasoning").
54. **Tools** — sometimes the AI needs to reach outside itself to complete the task, e.g., communicating with a Google Sheet, a database, an external API, or a search engine, in order to gather information. Examples of tools: APIs, databases, search engines, external applications.
55. **Action** — after analysis, the agent performs the required action (also sometimes called an "action node") — e.g., generating a report, sending an email, booking a meeting.

Simplified flow: **Input → Model → Tools → Action**.

56. **Why AI agents need Memory.**

Without memory, an agent is "stateless" — e.g., if you ask an agent to create a Telugu video today, and tomorrow ask again, a stateless agent would forget your language preference and ask again, which feels unintelligent/repetitive.

Memory allows an agent to:

- Remember past conversations
- Remember user preferences
- Use past data/context to make better decisions going forward

Benefits of memory (why it matters):

- Feels more like a natural human conversation
- Faster responses (no need to repeat information every time — similar to how a hospital pulls up your record from a patient ID instead of asking you to re-explain your history each visit)
- Better business value: more engagement, more trust

The instructor illustrates this with a personal-life analogy about remembering people's names (calling someone by name builds rapport/trust) — the same principle applies to why a memory-equipped agent

feels more valuable to a user than a "forgetful" one. (Note: exact technical implementation of memory in n8n/agent tools is deferred to a later, dedicated session.)

57. **The practical mindset for building an AI agent — a repeatable method.**

Steps recommended for identifying what to automate in any business or personal workflow:

58. **Observe daily tasks** — look at what you/your business does every day.
59. **Find repetitive tasks** — identify what repeats often (same-format emails, same types of summaries/reports/reminders are good candidates).
60. **Check if a tool can help** — e.g., an email system, spreadsheets, calendar, internet search, APIs, or databases.
61. **Design the agent workflow** — map out Input → Thinking (Model) → Tools usage → Action for that specific task.

Concrete examples given as automatable: an **invoice generator** (fixed format, triggered when an order comes in) and an **email/deadline reminder system**.

62. **Important caveat / myth to avoid:** *not every task or business process can be 100% automated.* Some activities are one-time, unstructured, or require a human touch. The realistic goal is identifying the (often large) percentage of repetitive, rule-based, structured work that *can* be automated — not assuming everything can be.
63. **Two common beginner mistakes when planning automation, called out explicitly:**
64. Trying to automate *everything* rather than being selective about what's actually repetitive/rule-based/time-consuming.
65. Starting by asking "which tool/framework should I use?" *before* clearly identifying the actual problem. The correct order is: **first identify the problem, then choose the tool** — not the other way around.
66. **First mention of automation platforms and vocabulary for upcoming sessions.**

To actually build agents and workflows, you need a platform. Names mentioned: **n8n (N8N)**, **Zapier**, and **Make** — these are described as automation platforms where you build workflows and AI agents using visual building blocks. Related vocabulary introduced for later sessions: **nodes**, **trigger nodes**, and **workflows**. The instructor confirms the course will primarily use **n8n** going forward.

Key takeaways

- **AI agent** = a single system performing a task; **AI agency / multi-agent system** = multiple agents working together (interchangeable terms).
- Every AI agent's architecture has four parts: **Input, Model (the "brain"), Tools, and Action**.
- **Memory** lets an agent remember past conversations/preferences, leading to faster responses and a more natural, trustworthy user experience — but not every agent necessarily needs memory.
- Practical automation method: **observe daily tasks → find what's repetitive → check what tools can help → design the agent's input/think/tools/action workflow.**

- Not everything can or should be automated — focus on repetitive, rule-based, time-consuming work.
- Common beginner mistakes: trying to automate everything at once, and picking a tool before clearly defining the problem.
- The course will now move toward hands-on practice using automation platforms, primarily **n8n**, building on the theory covered in Demos 1–5.

Demo 6: What Is Automation, and What Is N8N?

This session is a theory-only introduction (no hands-on building yet). It explains what "automation" means in a business context, the two ways to build automation systems, and introduces N8N as a no-code/low-code automation platform.

What you'll learn

- What automation means and why businesses need it
- The difference between "coding automation" and "platform automation"
- What N8N is, where its name comes from, and how it works conceptually
- The core vocabulary: workflow, node, trigger, action
- How N8N can be used (cloud vs. self-hosted), and roughly what it costs

Step-by-step notes

67. **Why automation matters.** The instructor uses an example: a small company gets repeated student emails (asking for syllabus, batch details, fees, etc.). Two employees manually reply to these repetitive questions all day. Problems with this manual approach:

- The same answers get typed over and over — no creativity needed, just wasted effort.
- If a staff member is on leave, replies stop.
- Staff can't realistically respond 24/7 (e.g., late at night), so customers complain about delayed replies.
- The fix: build a system that reads incoming messages, understands what's being asked, and sends the right response automatically — running 24/7/365, never on leave, freeing up humans for more creative work. This is "automation."
- At large scale (e.g., Amazon), this becomes essential — millions of inquiries and warehouse operations are handled by automated systems and robots, with humans stepping in only where robots can't (e.g., final delivery, though drone delivery is also emerging in places like China).

68. **Two ways to build automation:**

- **Coding automation**: A developer writes a program (e.g., in Python), configures API calls to communicate with external systems (email servers, spreadsheets, etc.), and runs it continuously (often on a cloud server). This gives full control and can handle very complex requirements, but requires programming knowledge, takes more time, and requires managing servers and integrations yourself.
- **Platform automation (no-code/low-code)**: A ready-made platform lets you build workflows visually — drag, drop, and connect pre-built components ("nodes") instead of writing code. You don't write code to talk to an email server or an API; you just add the relevant node and configure it (e.g., right-click → add Gmail node → provide credentials). This is much easier for non-programmers, but you're

limited to whatever nodes/features the platform already supports. If your requirement is very unusual, the platform might not support it, and you'd need to fall back to coding automation.

- Analogy used: buying a ready-made shirt (platform automation — quick, easy, fits most cases) vs. getting a shirt custom-stitched (coding automation — more powerful and customizable, but requires more effort).
- Both approaches are equally "secure" — platform automation isn't less secure, it just hides the coding details from you. Behind the scenes, platform automation tools still generate code (often Python), but you typically don't see or edit that code directly.
- Examples of no-code automation platforms mentioned: **N8N, Zapier, Make**.

69. **Key vocabulary introduced:**

- **Workflow**: a sequence of actions/steps that perform tasks automatically.
- **Node**: a single step or building block in a workflow. Every action (send an email, call an API, process data) is represented as a node.
- **Trigger / Trigger node**: the starting point of a workflow — the event that kicks off the automation (e.g., receiving an email, a new WhatsApp message, or a form submission).
- **Action node**: the node(s) that do the actual work after the trigger fires.

70. **Common beginner mistakes to avoid** (explicitly called out):

- Thinking automation platforms remove the need to understand logic entirely — they reduce coding, but you still need to design the logic/sequence of your workflow.
- Trying to automate everything at once — start small, with repetitive tasks.
- Ignoring workflow design — automation only works if the workflow (the sequence of steps) is clearly thought out first.
- Not understanding the difference between triggers and actions.

71. **What is N8N specifically?**

- N8N is a no-code/low-code **automation platform** — a tool that connects different apps/systems and automates tasks by letting you build workflows visually.
- The name breaks down informally as "node-based automation" — "n8n" is a numeronym (like i18n = internationalization) where "8" stands in for the 8 letters between the first "n" and the last "n" in "nodemation"/"node-based automation." (This is the instructor's own explanation of the naming pattern, similar to how "i18n" abbreviates "internationalization.")
- Every element in an N8N workflow — the trigger, the actions, even an AI agent — is represented as a **node**.
- N8N supports 300+ (the instructor later corrects this to 500+ once actually using the platform) pre-built app integrations/nodes (Gmail, Google Sheets, WhatsApp, Telegram, OpenAI, etc.). If a needed integration doesn't exist as a node, you'd need coding automation instead.
- Analogy: N8N is like a "smart employee who never sleeps" — you give instructions once, and it works 24/7 without complaint.

72. **How to run N8N — two deployment options:**

- **Cloud version**: hosted by N8N; you sign up and start building immediately, no installation needed. Easiest for beginners. Pricing is roughly €20/month for a paid plan, with a **14-day free trial** offering up to 1,000 executions.
- **Self-hosted version**: install N8N on your own computer/server (e.g., using Docker, or via Node.js). Free to run locally, but you must configure things like API authentication manually (the cloud version can auto-authenticate certain services like Gmail with a click) and you must keep your own server running 24/7 for the automation to work continuously.
- Workflows can be exported/imported as JSON files, so you can move a workflow between environments (similar to exporting a project from GitHub), though credentials/API keys need to be reconfigured on the new platform each time.

73. **Other points raised in Q&A:**

- APIs allow one program/system to talk to another without a UI — e.g., "Sign in with Google" internally uses an API call to authenticate and pull your profile data.
- Whether workflows built on such platforms can be patented/copyrighted was discussed but left unresolved — the instructor suggested it's more like software that could be copyrighted rather than patented, but wasn't certain.

Key takeaways

- Automation means letting a system handle repetitive tasks so humans can focus on higher-value work.
- There are two paths to automation: **coding automation** (full control, requires programming) and **platform automation** (visual, no/low-code, faster to learn, limited to available nodes).
- N8N is a leading no-code automation and AI agent platform, built around **workflows** made of **nodes**, always starting with a **trigger**.
- N8N can run in the cloud (easy, paid after 14-day trial) or self-hosted (free, more setup work, more control).
- Even with no-code platforms, you still need to think clearly about your workflow's logic — the platform removes coding, not the need for planning.

Demo 7: Building Your First AI Agent in N8N

This is the first hands-on session. The instructor signs up for N8N, creates a workflow, and builds a simple chat-based AI agent step by step — adding a trigger, an AI agent node, a chat model (OpenAI), a calculator tool, and memory — then tests how the agent behaves with and without tools/memory.

What you'll learn

- How to sign up for N8N and create your first workflow
- The parts of the N8N workflow builder: canvas, node panel, node configuration
- How to add a chat trigger, an AI agent node, connect an LLM (chat model), add a tool, and add memory
- How AI agents decide when to use a tool vs. answer directly from the LLM
- Why memory is needed for an agent to hold conversation context

Step-by-step notes

74. **Recap of trigger concept**: A trigger is the starting node of any workflow — the event that starts the automation. Examples given: an alarm clock triggers your morning routine; a WhatsApp message triggers a chatbot; a form submission triggers a "collect and thank" workflow.
75. **AI agent building blocks (explained before building):**

- Every AI agent needs a **brain** — an LLM model (e.g., OpenAI, Anthropic/Claude, or Gemini).
- An AI agent can optionally have **memory**, so it can remember previous interactions/context. Without memory, every message is treated as a brand-new, unrelated request.
- An AI agent can optionally use **tools** (APIs, databases, a calculator, etc.) to extend what it can do.

76. **Signing up for N8N:**
77. Go to **n8n.io** (search "n8n" — it's the "AI workflow automation platform").
78. Click **Sign in** → **Get started**.
79. Enter an email (personal email is fine), submit.
80. You automatically get a **14-day free trial**.
81. Fill in name, user ID, and password to complete signup.
82. **Creating your first workflow:**
83. On the free trial, you're limited to **one project**. Use **+ → Create workflow** inside your project.
84. Rename the workflow (the default is "My workflow" — renaming is effectively required, since running with no name causes errors). Example used: "automation one."
85. N8N **auto-saves** every few seconds — there's no manual "Save" button in the current version.
86. The main design area is called the **workflow canvas** — think of it as your drawing board where you drag, drop, and connect nodes.
87. Note: N8N also offers a "Build with AI" option where you describe a workflow in a prompt and AI generates it for you, but the instructor built the first one manually for learning purposes (manual building gives more control).

88. **Adding the trigger node:**
89. Click the **+** (node panel) on the empty canvas.
90. Choose **On chat message** as the trigger — this runs the flow whenever a user sends a chat message, and is meant for use with AI nodes.
91. No extra configuration was needed for this trigger — just close the panel to confirm it's added.
92. **Adding the AI agent node (action node):**
93. Click **+** again after the trigger, search for and select **AI Agent**.
94. As soon as it's added, N8N shows an error: *"No node connected to required input: Chat Model"* — this means the agent needs a brain (LLM) connected before it can run.
95. Click the **Chat Model** slot (marked with a required "*" — mandatory, unlike Memory and Tools which are optional) and choose **OpenAI Chat Model** (other options include Anthropic, AWS, Cohere, etc.).
96. Choose a model (e.g., **GPT-5** or **GPT-5 mini**).
97. N8N will say *"Credentials for OpenAI Chat Model are not set."* Double-click the node to open its **configuration panel**, then click **Set up credentials**.
98. Get an API key from **platform.openai.com** → Account/API keys section → **Create secret key** → copy it.
99. Paste the key into N8N's credential setup and save. (N8N also offers some free OpenAI credits for testing, but once those run out you need your own paid API key.)
100. Credentials are saved per project — once set up, they're reused for other workflows in that same project without asking again.
101. **Adding a tool — Calculator:**
102. On the AI agent node, click the **+** next to **Tools** and choose **Calculator** (a simple, built-in tool needing no extra configuration).
103. Tools, like Memory, are optional on the agent (only Chat Model is mandatory).
104. **Running/testing the agent:**
105. Click **Execute** (the play/start icon) on the trigger node.
106. A chat window opens where you type a message and press enter/send.
107. Observed behavior:

- Simple greeting ("Hello, friend") → goes straight to the LLM (OpenAI), which replies — calculator not used.
- Small math ("4 + 5") → the LLM answers directly on its own knowledge — calculator still not triggered.
- A large/complex calculation (e.g., a big multiplication) → the flow visibly goes AI agent → OpenAI → Calculator tool → OpenAI → response. The LLM **delegates** big calculations to the calculator tool for accuracy, because LLMs are not always reliable at large arithmetic.

108. This demonstrates that the **AI agent (backed by the LLM) decides at runtime** whether a tool is needed, based on the request — you don't have to manually route requests to tools.

109. You can push the model to always use a specific tool by adding an instruction in the **system message/prompt** (e.g., "for any mathematical calculations, use the calculator tool only") — this is a way to force consistent tool use ("guardrail rule"). This was mentioned but not fully configured in this session (left for later).
110. **Demonstrating why memory is needed:**
111. Ask the agent: "Provide information about Donald Trump in a single line" → it answers correctly using the LLM.
112. Then ask a follow-up: "What is his age?" → the agent fails, responding with something like *"Who do you mean by 'his'? I don't have that context."* This proves the agent, by default, has **no memory** of the previous message — every request is independent.
113. Fix: click **+** next to **Memory** on the AI agent node and add **Simple Memory** (an option that requires no external credentials — it stores memory locally inside the N8N instance). Default **context window length** is **5** (meaning it remembers the last 5 past interactions).
114. After adding memory and resetting the chat session, repeating "Provide information about Donald Trump" then "What is his age?" works correctly — the agent now remembers "his" refers to Donald Trump from the earlier message.
115. Note: N8N's "items" counter in the execution log reflects how many times a given node was used in that run, not how many memory records are stored — this was a point of confusion clarified during the session.
116. **Other points covered in Q&A:**

- You can run each node individually (for debugging/testing), or run the whole workflow via the chat's send arrow — normally you don't need to run every node one by one; that's just for debugging.
- Multiple LLMs can be used across different nodes (e.g., different agents in the same or different workflows using OpenAI, Gemini, Anthropic, Grok respectively) — this was confirmed as possible but wasn't demonstrated live.
- Real ChatGPT also uses an internal calculator tool for big calculations, similar logic to what was shown here — inside N8N, the agent talks to the OpenAI LLM via API, and that LLM (when it decides it needs precision) uses the calculator tool the workflow provides.
- **SERP API** (Google Search-based) was mentioned as a tool you can add to N8N so agents can fetch up-to-date information the LLM's training data might be missing (e.g., current events); a Google account can generate a free API key for it. This wasn't demonstrated live but was flagged for a future session.
- This chat-based agent could later be embedded into a real website/app as a chatbot interface, or connected to WhatsApp, Telegram, etc. — not shown yet in this session, planned for later.
- N8N cost: ~€20/month after the 14-day free trial (with ~1,000 free executions during trial).

Key takeaways

- Building a basic AI agent in N8N takes just a few nodes: **Trigger (chat message) → AI Agent → Chat Model (LLM) → optional Tools → optional Memory**.

- The AI agent node itself is just a coordinator; the actual "thinking" is done by the connected **chat model (LLM)**, and it automatically decides when to call a **tool** (like a calculator).
- Without **memory**, an agent cannot hold context across messages — each message is treated in isolation.
- N8N requires you to connect a valid LLM (with an API key) before an AI agent node will work — Chat Model is mandatory, Tools and Memory are optional add-ons.
- You can run/test workflows directly inside N8N's built-in chat window before deploying them anywhere else.

Demo 8: Understanding the N8N Dashboard — Canvas, Nodes, Configuration, and Logs

This session is a theory/review class recapping and deepening understanding of the N8N interface components seen in Demo 7, plus a Q&A covering practical limits and possibilities (multiple agents, multiple LLMs, connecting workflows, node limits).

What you'll learn

- The four core dashboard components of N8N: workflow canvas, nodes, node configuration panel, execution logs
- How to open and use the node configuration panel and execution logs
- Practical facts: node limits, whether AI agents are mandatory, using multiple LLMs/agents, connecting workflows

Step-by-step notes

117. **Workflow canvas** — the main workspace where you build automation workflows. Think of it as a drawing board: you drag nodes onto it and connect them to define how automation flows from step to step.
118. **Nodes** — the building blocks of a workflow. Each node performs one task (send email, call an API, process/transform data, trigger a workflow, etc.). In simple terms: **a node = one action in automation**. Types of nodes mentioned: trigger nodes, action nodes, and logic nodes (e.g., if/else, switch — for branching, to be covered later).

- Everything in a workflow — including the AI agent itself, and even a tool like a calculator — is represented as a node. This is why N8N is called "node-based automation."

119. **Node configuration panel** — opened by **double-clicking** any node. This is where you set that node's specific settings: for an email node, this includes recipient, subject, message body, etc. Any node-specific setup (API keys, parameters, credentials) happens here.
120. **Execution logs** — shown at the bottom of the canvas after running a workflow. They record what happened during execution: which steps ran successfully, which failed, and what data was processed at each step. This is essential for debugging when something goes wrong. There's also an **Executions** tab (top of the dashboard) that keeps a history of all past runs of a workflow, so you can go back and inspect any previous execution's logs — not just the current/latest one.
121. **Recap flow of building a workflow** (sequence taught):
122. Create workflow
123. Add trigger node
124. Add action node(s)
125. Connect nodes (either via the **+** icon on an existing node, or by dragging in a standalone node and connecting it afterward)
126. Run/execute the workflow

127. Check execution logs to verify/debug
128. **A useful learning prompt suggested by the instructor**: ask an LLM something like *"Act as a beginner-friendly instructor and explain the N8N dashboard interface and its main components"* — this can generate a clear, simplified explanation (the instructor demonstrated this, getting an analogy: N8N is like a factory — nodes are the machines, connections are the conveyor belt moving data, and the dashboard is your control room).
129. **Q&A / clarifications on practical N8N behavior:**

- **Is there a limit on the number of nodes in one workflow?** No fixed limit like "only 10 or 50 nodes" — you can add as many as you want. In practice, performance depends on your system: roughly 10–20 nodes runs smoothly, 100+ nodes may get slow, 300+ nodes can hang or crash. Cloud plan limits are based on **number of executions**, not directly on node count (the 14-day free trial allows about 1,000 executions).
- **Can you use multiple LLMs in a single workflow?** Yes — different agent nodes within the same workflow can each be connected to a different LLM (e.g., one agent using OpenAI, another using Gemini, another using Grok/Anthropic), each requiring its own separate API key/credentials.
- **Can you add more action nodes after the AI agent node?** Yes — the AI agent is not necessarily the last node. For example: Chat Trigger → AI Agent → Google Sheets (to save data) → WhatsApp (to send a message) is a valid extended workflow.
- **Can you have multiple AI agents in a single workflow?** Yes.
- **Can you have multiple workflows on the same canvas panel?** No — one canvas/panel represents exactly one workflow. However, a **project** can contain multiple separate workflows.
- **Can you connect two separate workflows together** (e.g., one workflow's output feeding into another workflow)? Yes, this is possible (e.g., via an "Execute Workflow" node that calls another workflow) — this avoids building one giant unmanageable workflow and keeps things modular/maintainable. Full details were deferred to a later session.
- **Is an AI agent mandatory in every N8N workflow?** No — N8N existed as a general automation platform before AI agents were added as a feature. You can build purely rule-based automation (e.g., "if a mail like this arrives, send this fixed reply") without any AI agent or LLM at all.

Key takeaways

- N8N's dashboard has four core things to know: the **canvas** (where you build), **nodes** (the actions), the **configuration panel** (double-click a node to set it up), and **execution logs** (for checking results and debugging).
- There's no hard cap on how many nodes a workflow can have, but very large workflows can hurt performance — keep workflows focused and modular, and split large automations into multiple connected workflows instead of one giant one.
- N8N supports multiple AI agents and multiple different LLMs within the same workflow.
- AI/LLM agents are optional in N8N — plenty of useful automation doesn't need any AI at all.

Demo 9: Controlling Agent Behavior with System Messages, and Building an Email Automation Agent

This session covers two main things: (1) how to use a **system message** to control an AI agent's behavior (forcing it to always use a specific tool), and (2) building a complete email-sending AI agent using Gmail, including formatting the email content properly via a system prompt.

What you'll learn

- How to configure a system message on an AI agent node to control its behavior
- How to add and authenticate a Gmail tool so an agent can send emails
- How to let the AI agent decide email recipient/subject/message content automatically from the chat instruction
- How to write a system prompt that enforces a consistent email format
- Practical Gmail tool options (attachments, CC, BCC, append attribution)

Step-by-step notes

130. **Recap and motivation**: In the previous session's test agent, small math was answered by the LLM directly, while large math triggered the calculator tool. The instructor wanted to force **all** math (big or small) to go through the calculator tool, for the sake of demonstrating configuration control.
131. **Configuring a system message:**
132. Open the earlier test agent workflow, double-click the **AI Agent** node.
133. Find/add the **System Message** field in the configuration panel.
134. Enter an instruction, e.g.: *"For any mathematical calculations, use calculator tool"*.
135. Close the panel — N8N auto-saves the change (a note appears saying output may change when the node runs again — this is expected and not an error).
136. Result: after this change, even a simple calculation like "3 + 4" now routes through the calculator tool instead of being answered directly by the LLM, because the system message instructs the agent to always delegate math to that tool.
137. **Key concept**: the **system message** is a standing instruction that shapes how the AI agent behaves for every subsequent request — similar to a configuration/rule that persists, rather than something you'd have to repeat in every individual prompt (which would be wasteful and increase token cost).
138. **Building a new Gmail-sending AI agent workflow:**
139. Create a new workflow inside the same project, rename it (e.g., "email workflow two").
140. Add trigger: **On chat message** (same as before, no config needed).
141. Add action: **AI Agent** node.
142. Configure the **Chat Model**: select a language model (e.g., OpenAI) and set up credentials/API key (same process as Demo 7 — get key from platform.openai.com, or an existing credential can be reused across workflows in the same project).

- Important: **any workflow using an AI agent must have an LLM/chat model connected — this is mandatory.** Without a brain, the agent node will show an error and won't function.
- API keys are billed based on usage; you can create as many separate API keys as you like without extra charges — charges are based on token usage, not number of keys. (OpenAI required loading a minimum balance, e.g., around $5, in this case; Gemini was mentioned as offering a free tier — roughly 1 million free tokens — as an alternative.)

143. Add **Memory**: Simple Memory (same as before), left at default settings.
144. Add a **Tool**: search for and add the **Gmail** tool (specifically the one for calling the Gmail API, not "Human approval" or other Gmail-related options that may appear in the list).
145. Configure the Gmail tool's credentials: choose **Sign in with Google**, complete mobile number verification if prompted, and grant the requested account access permissions. This uses Gmail's OAuth authentication internally — no manual username/password or API key needed (unlike the OpenAI chat model, which does need an API key).
146. For the Gmail tool's fields (**To**, **Subject**, **Message/email body**), you can select **"Let the model define it"/"Defined automatically by the model"** for each — meaning the AI agent itself decides the recipient, subject, and message content based on what the user typed in chat, rather than you hardcoding these values.
147. **Testing the email agent:**
148. In the chat trigger, type something like: *"Send a mail to [email] about the importance of N8N. Give conclusions in two points."*
149. The AI agent (LLM) composes the email content, and the Gmail tool sends it using the connected Gmail account.
150. Confirmed working: the recipient received a properly composed email with the requested content.
151. You can also address multiple recipients in one instruction (e.g., "send a mail to X and Y about...") and the agent will send to all of them (though the instructor noted N8N doesn't validate whether an email address is properly formatted/real before attempting to send).
152. Flow explanation given: Chat message → AI Agent (treats the chat text as a prompt) → LLM decides content and recipient/subject → hands off structured info to the Gmail tool → Gmail tool sends the email.
153. **Improving email formatting with a system message:**

- Problem observed: the auto-generated email lacked a proper closing/signature, and included an unwanted default line like "This email was sent automatically with N8N."
- Rather than repeating formatting instructions in every chat prompt (which wastes tokens and isn't reliable), the fix is to add a **system message** on the AI agent node describing the required email format, e.g.:
- Subject: short and clear, reflecting the email's purpose
- Greeting: polite, context-appropriate (e.g., "Dear Sir/Madam," "Hello," "Hi Team")
- Body: clearly state purpose, main content, and a call to action

- Closing: polite sign-off with sender details (name, company, contact info if available)
- Style: simple, plain English; avoid complex words/long sentences
- Output rules: don't add extra explanations or headings like "Here is your email" — output only the final email content
- This system message was generated with AI assistance (asked an LLM to draft a suitable system message for this exact purpose) and pasted into the AI Agent node's **System Message** field.
- After this change, the emails came out properly formatted with correct subject, greeting, body, and signature — consistently, without needing to repeat instructions each time.
- Separately, in the **Gmail tool's own configuration** (not the AI agent), there's an **"Append attribution"** option (enabled by default) that adds the "sent automatically with N8N" line — this can be **disabled** directly in the Gmail node's options to remove that line entirely.

154. **Other Gmail tool options mentioned**: attachments, CC, BCC, "send replies to" — all available under an "Options"/"Add option" section within the Gmail tool's configuration panel, allowing further customization of how emails are sent.

155. **Q&A / additional points:**

- You can schedule recurring sends (e.g., a daily automated email) — this will be covered in a future session (mentioned but not demonstrated).
- Sending personalized emails to many people (e.g., 100 people about different topics) — the plan for a future session is to pull recipient lists and per-person topics from a Google Sheet, with the agent composing and sending each one, rather than hardcoding everything in the chat prompt.
- A related idea (buying the "best phone under a budget," searching the web, and even completing a purchase automatically) was raised by a student; the instructor said this kind of web-search + purchase automation would be explored in later sessions.
- This session's flow is still chat-triggered (testing inside N8N's chat panel) — connecting this agent to a real website/front-end chatbot interface was deferred to a future session, though the instructor described conceptually how it would work (embed a chat widget or form on a website; user interactions call into this same N8N workflow).

Key takeaways

- A **system message** on the AI Agent node is a powerful way to permanently control agent behavior (e.g., forcing tool use, enforcing output formatting) without repeating instructions in every request — this also saves on token costs.
- Any workflow using an AI Agent node must have an LLM (chat model) connected — this is mandatory; Memory and Tools remain optional.
- The **Gmail tool** in N8N uses Google Sign-in (OAuth) rather than a manual API key, and can be configured so the AI agent automatically decides the recipient, subject, and body based on the user's natural-language request.
- N8N does not validate whether email addresses are real/correctly formatted before attempting to send.

- Node-level options (like the Gmail tool's "Append attribution" setting) let you fine-tune behavior beyond what the AI agent decides on its own.

Demo 10: Building a Form-Based Feedback Automation (No AI Needed)

This session builds a complete, practical automation without any AI agent or LLM: a public feedback form that emails a thank-you note to the submitter and forwards the feedback to the company's inbox — then publishes the form live for real users to try.

What you'll learn

- How to create an N8N-hosted web form as a trigger (the "N8N Form" trigger node)
- How to add and configure form fields (text, email, text area, required fields)
- How to send two different emails from one workflow (to the submitter, and to an internal team) using form data
- How to reference submitted form field values inside email content
- How to publish a form to a live production URL and later unpublish it
- Recognizing when AI/LLM is unnecessary — plain automation is enough for fixed-content tasks

Step-by-step notes

156. **Goal for this session**: Build a feedback collection workflow: a student fills out a feedback form → the company receives the feedback by email → the student receives an automatic thank-you email. No AI agent, no database, no Google Sheets — just email, kept intentionally simple.
157. **Creating the workflow:**
158. Create a new workflow in the project, rename it (e.g., "feedback workflow").
159. Add the trigger: instead of "chat message," search for and select **N8N Form** → choose **"On new form submission"** trigger (previously shown as "On new N8N form event").
160. Rename the form node meaningfully, e.g., "Student feedback form."
161. Leave **Authentication** off (not required for a public feedback form).
162. Set the **Form title** (e.g., "Student feedback form") and optionally a description.
163. **Adding form fields** (using "Add form element" repeatedly):
164. **Name** — Element type: **Text input**, marked as a **Required field**.
165. **Email** — Element type: **Email** (a dedicated email field type), marked as required.
166. **Feedback** — Element type: **Text area** (since feedback may span multiple lines), marked as required.

- Note: a **Submit button** is automatically added by N8N by default — you don't need to add it yourself; all other fields (name, email, feedback, etc.) must be added manually.
- "Required field" means that field is **mandatory** — the end user cannot submit the form without filling it in.
- You can test the form directly from the trigger node before publishing (it provides a **Test URL**), by running the node and filling in sample data.

167. **Adding the first Gmail node — thank-you email to the submitter:**
168. Add a **Gmail** node (not an AI agent this time — direct node-to-node automation, since the email content is fixed/templated, not something an LLM needs to generate).
169. Set the operation to **Send a message**.
170. Reuse the existing Gmail credential set up in a previous session.
171. **To**: dynamically reference the email address submitted in the form (so the thank-you email goes to whoever submitted feedback).
172. **Subject**: e.g., "Thank you, [Name], for your feedback" — referencing the submitted name field.
173. **Message** (text): e.g., "Hello [Name], Thank you for your valuable feedback. It is really helpful for us to improve our services. Thanks and regards, Durga Soft Online."
174. The **From** address is whatever Gmail account was set up in the credential (e.g., "justlearn786@gmail.com" from an earlier session) — not reconfigured here.
175. Run/execute the node to test — confirmed the thank-you email was received correctly by the test submitter.
176. In the Gmail node's **Options**, disable **"Append attribution"** to remove the unwanted "This email was sent automatically with N8N" line from the message.
177. **Adding a second Gmail node — internal notification to the company:**
178. Add another Gmail node, again set to **Send a message**.
179. **To**: a fixed internal address (e.g., "durgasoftonline@gmail.com") — this one always goes to the same place, regardless of who submitted feedback.
180. **Subject**: e.g., "Feedback made from [Name]" — pulling the submitted name into the subject line.
181. **Message**: e.g., "Hello Durga Soft team, the student [Name] provided the following feedback: [Feedback text]. Regards, Automation Team." — pulling both the name and feedback fields from the form submission into the email body.
182. Run/execute to confirm — the internal team's inbox receives a notification with the submitter's name and feedback content.
183. This demonstrates: **one workflow can branch out to multiple independent actions** (two separate emails, to two different recipients, with two different messages) after a single trigger.
184. **Publishing the form live:**
185. At the top of the form trigger node, there's a **Publish** option. Click it (accepting/skipping any extra prompts about notifications, time tracking, MCP access, etc., which weren't needed here).
186. Once published, N8N provides a **Production URL** (different from the earlier Test URL) — this is the real, shareable link.
187. Share this Production URL with real users; anyone who opens it can fill out and submit the form.
188. Live test: several participants opened the link, submitted feedback, and confirmed they received the thank-you email; the instructor separately confirmed the internal "durgasoftonline@gmail.com" inbox received all the feedback notifications correctly.

189. Important: **while published (live), the workflow keeps running 24/7 in the cloud** — it continues to work even if you close your laptop or sign out of N8N, as long as the workflow is published and your account/trial is still active.
190. You can **Unpublish** the form at any time — after unpublishing, the production URL stops working, showing a "problem loading form" error.
191. Caveat: this live/production behavior only lasts as long as your N8N account access does — during the 14-day free trial, the published form will stop working once the trial period ends (you'd need to upgrade to a paid plan to keep it running indefinitely).
192. **Why no AI agent was used here**: The instructor explicitly points out that this is "simple plain automation without AI" — since the email content is fixed/templated (always the same structure, just inserting form field values), there's no need for an LLM to generate new content each time. AI agents/LLMs are for when you need the system to compose or reason about varying content; for fixed-format notifications, direct node-based automation is simpler and sufficient.

Key takeaways

- N8N's **Form trigger** node lets you build and host a simple web form (with text, email, and text-area fields, required-field validation) without writing any HTML yourself.
- A single trigger (form submission) can fan out to multiple independent actions — here, two separate emails to two different recipients with two different messages, both pulling data from the same form submission.
- You can reference submitted form field values (name, email, feedback text) directly inside downstream node configurations like email subject/body.
- Forms/workflows have a **Test URL** (for private testing) and a **Production URL** (only active after you click **Publish**) — publishing makes the automation live and continuously running in the cloud, even when you're offline.
- Not every automation needs AI — when the output content is fixed or template-based, plain node-based automation (no LLM, no AI agent) is simpler, cheaper, and sufficient.
- Free-trial-hosted workflows only stay live in production for the duration of the trial; continuing to run them after that requires a paid plan.

Demo 11: Building a Loan Eligibility Check Automation in n8n

This demo walks through building a complete n8n workflow that collects a loan application through a form, checks whether the applicant's salary qualifies them for a loan, and automatically sends a congratulations or a sorry email depending on the result.

What you'll learn

- How to build a form with multiple field types (text, email, number, text area) using the n8n Form Trigger node
- How to use the Edit Fields (Set) node to combine fields (first name + last name into full name) and clean up data
- Why a Number-type field can silently break values that need a "+" prefix (like +91 mobile numbers), and how to fix it by switching to String type
- How to use the If node to branch a workflow into a "true" path and a "false" path based on a condition
- How to configure two separate Gmail nodes to send different emails depending on which branch was taken
- How to use ChatGPT to quickly format a plain-text/HTML email body
- How to test a workflow node-by-node, publish it, and view past runs in the Executions tab

Step-by-step notes

1. Plan the workflow

The goal: build a "Loan Eligibility Check" form. An applicant fills in first name, last name, email, mobile number, salary, and address. If salary is greater than or equal to 35,000, the system sends a "Congratulations, you are eligible" email; otherwise it sends a "Sorry, you are not eligible" email with some encouragement to try again. This example checks only salary, but the instructor notes you could extend it later (e.g., add a civil score condition, save data to Google Sheets/a database, add WhatsApp/SMS notifications) — none of that extra part is built in this session.

The workflow will use four types of nodes, in this order:

193. **Form Trigger node** – collects applicant details
194. **Edit Fields (Set) node** – cleans/combines the data (e.g., builds full name)
195. **If node** – checks the salary condition (true branch / false branch)
196. **Two Gmail nodes** – one connected to the true branch (congratulations mail), one connected to the false branch (sorry mail)

2. Create the Form Trigger node

- Click the "+" to add a node, search for "form", and select **"On form submission"** (the n8n Form Trigger).
- Double-click the node to open its configuration panel.

- Set the **Form Title**, e.g. "Loan request form".
- Add a description if desired.
- Add form elements one at a time using "Add form element":
- **First name** – Label: First name, Type: Text, marked Required.
- **Last name** – Label: Last name, Type: Text, marked Required.
- **Email** – Label: Email, Type: Email (chosen instead of plain Text so the form automatically validates that the value looks like a real email address), marked Required.
- **Mobile number** – Label: Mobile number, Type: Number (initially), marked Required.
- **Salary** – Label: Salary, Type: Number, marked Required.
- **Address** – Label: Address, Type: Text area (allows multiple lines, so city/street/pin code can all go in one field), left as **not required/optional**.
- Field order in the live form matches the order you add them in — you can reorder as you build.
- Click "Execute step" (test the node) to open the actual test form and try submitting it, e.g.: First name "Durga", Last name "soft" (example names used), Email "durgasoftonline@gmail.com", Mobile "8096969696", Salary "45000", Address "Hyderabad" (optional, can be left blank).
- If a required field (like email format or salary) isn't filled correctly, the form blocks submission and shows a validation message — this is standard HTML-style form validation, not custom coding.
- After submitting, the node shows "Workflow executed successfully" and the submitted data becomes available to the next node.

3. Add the Edit Fields (Set) node

- Search for "edit" and choose **"Edit Fields"** (also called the Set node) — it modifies, adds, or removes fields.
- Purpose: clean/transform data coming from the form before it's used further (e.g., combine first + last name, add a country code) — this is data meant for the workflow, not shown back to the end user.
- Add a new field named **"Full name"**, and drag in `first name` then `last name` (separated by a space) so the value becomes e.g. "Mohan Kumar".
- Also demonstrated: adding a mobile field with "+91" prefixed to the number.
- **Beginner mistake shown:** the mobile number field from the form was Number type. When trying to prepend "+91" in the Edit Fields node and executing, the workflow failed — because "+" is not a valid number, so it gets stripped/rejected.
- **Fix:** go back to the Form Trigger node and change the mobile number field's input type from **Number** to **Text (String)**. After that, adding "+91" works correctly.
- Fields created in this node (and from the original form) are available to all later nodes in the workflow, not just the immediate next node.

4. Add the If node (logic/condition check)

- Search for and add an **If** node (other logic nodes like Switch exist in n8n but aren't used here).
- It automatically shows two outputs: **true branch** and **false branch**.

- Condition setup: drag the **Salary** field from the form data into the condition, set the type to **Number**, choose the operator **"is greater than or equal to"**, and enter the value **35000**.
- Test by executing the node: with salary = 45,000, the true branch is selected (since 45,000 ≥ 35,000).

5. Add the Gmail node for the true branch (Congratulations email)

- Add a **Gmail** node, set Resource = Message, Operation = Send.
- Gmail credentials were already configured in an earlier session, so no new credential setup was shown here.
- **To:** drag in the customer's email field from the form submission data.
- **Subject:** e.g. "Congratulations, [First Name], you are eligible for loan" — first name only is used here since the subject line is short.
- **Message body:** uses the **Full name** field (from the Edit Fields node), e.g. "Hello [Full Name], congratulations, you are eligible for the loan," mentioning the salary and that the support team will contact them soon, with a closing/regards line.
- The instructor used **ChatGPT** to reformat/clean up this message body text for use in the Gmail node (prompting it to format the message for a Gmail node in an n8n automation), then copy-pasted the result in.
- Executed the workflow and confirmed the congratulations email was actually received in the test Gmail inbox with the correct name and salary filled in.

6. Add the Gmail node for the false branch (Sorry email)

- Instead of building from scratch, the true-branch Gmail node was **copied** (right-click > Copy, then Ctrl+V to paste — there's no explicit "paste" menu option, but the Ctrl+V keyboard shortcut works) and then edited:
- **Subject** changed to something like "Sorry, [First Name], you are not eligible for loan."
- **Message body** changed to a "not eligible" version — again drafted with ChatGPT's help — explaining the salary doesn't currently meet the eligibility criteria, encouraging the applicant to apply again in the future, and inviting them to contact the support team with questions.
- The recipient email field (drag from form data) stays the same.
- The node was optionally renamed (e.g. to something clearer than the default "Send a message").
- This new Gmail node is then **connected to the false branch output** of the If node.

7. Test both branches end to end

- Re-running the whole flow with a high salary (e.g. 45,000) triggers the true branch and the congratulations email.
- Re-running with a low salary (e.g. 20,000, address left blank since it's optional) triggers the false branch and the sorry email is received correctly (e.g., "Sorry, Ravi, you are not eligible for loan").
- A node that isn't executed because the flow took the other branch shows no result/tick mark — this is expected and simply means that path wasn't taken this run.

8. Publish and let others test

- The workflow can be **published**, which makes the form's production URL available (versus the test URL used earlier).
- Several students tested the live form (both the eligible/congratulations path and the not-eligible/sorry path) and confirmed they received the correct emails.
- Note: since a personal Gmail account was used (not a business email), there's a daily sending limit (roughly around 500/day) and personal Gmail may flag automated sending as suspicious; a business/official email address is recommended for production use to avoid this.
- Custom domains for the form URL aren't available on the free/trial n8n plan — that requires a paid plan and your own domain.

9. Reviewing submissions afterward

- The top navigation has an **Executions** tab (next to Editor) that lists every time the workflow ran, with timestamps.
- Clicking an execution reopens that specific run so you can see exactly what data was submitted (e.g., which applicant, what values) — useful for troubleshooting a specific user's submission.
- The workflow, as built in this demo, does not save submitted data anywhere permanent (like Google Sheets or a database) — the instructor noted that doing so (and reading from it later) will be covered in a future session, along with ideas like limiting the number of form submissions or looking up whether a specific request came in.
- To embed the form on an external website, you can link a button's hyperlink to the form's target URL so that clicking it opens the form.

Key takeaways

- A full "useful" workflow typically needs more than just a trigger: this one used Form Trigger → Edit Fields (Set) → If → Gmail (x2), each adding a distinct capability.
- The Edit Fields (Set) node is for transforming/enriching data for internal workflow use (combining names, adding prefixes) — not for changing what the end user sees on the form itself.
- Watch field types carefully: a Number-type field will silently drop non-numeric characters like "+", which broke the "+91" mobile number prefix; switching to String/Text type fixed it.
- The If node produces a true branch and a false branch; each branch can be wired to different downstream nodes (here, two different Gmail nodes) to send different messages.
- Copying an existing node (via right-click Copy, then Ctrl+V to paste) and just editing its subject/body is a fast way to build the "opposite" branch's email instead of configuring it from scratch.
- Data from every earlier node (form fields, edit-field outputs) remains available to all later nodes in the workflow, not just the immediately following one.
- ChatGPT was used purely as a writing/formatting aid to draft clean email text — it's not an n8n node in this workflow.
- The Executions tab lets you inspect the exact data of any past run, which is essential for troubleshooting individual user submissions.

- This demo intentionally does not persist data to a database or spreadsheet, or add extra channels like WhatsApp/SMS — those are flagged as topics for a future session.

Demo 12: Saving Form Feedback to Google Sheets, Throttling with the Wait Node, and Sending a Thank-You Email

This video builds a complete "Feedback" workflow in n8n: a form collects name, email, mobile number, and feedback, the data is written to Google Sheets (with a close look at Append vs. Append or Update), a Wait node is added to avoid hitting Google/Gmail rate limits, and finally a Gmail node sends a "thank you for your feedback" confirmation email.

What you'll learn

- How to build a Feedback form using the Form Trigger node with multiple fields
- How to connect a Google Sheets node to write form data automatically
- The difference between the "Append row" and "Append or update row" operations
- How to map form fields to Google Sheet columns (automatic mapping by matching names)
- How to create a new Google Sheets document and sheet directly from n8n
- Why Google Sheets has write-rate limits and why Gmail has a daily sending limit (~500/day)
- How to use the Wait node to pause a workflow (e.g., 65 seconds) to avoid rate-limit issues
- A real concurrency bug the class discovered live when many people submitted the form at once
- Why a spreadsheet is not a substitute for a real database when many users write at the same time
- How to add a Gmail node to send a confirmation "thank you" email after data is saved

Step-by-step notes

1. Building the Form Trigger

- Created a new workflow named "Feedback."
- Added a **Form** node (new form event). No authentication needed.
- Form Title: "Feedback request form" (any wording is fine), plus a description.
- Added form fields:
- **name** — Input type: Text, marked as a required field
- **email** — Input type: Email, marked as required
- **mobile number** — Input type: Number, marked as required
- **feedback** — Input type: Text area, marked as required
- Tested by clicking the form's test URL, filling in sample data (name, email, mobile, feedback), and clicking Submit — confirmed the node executed successfully and the data appeared in the next node.
- A student asked if the email field could be auto-collected (e.g., from a logged-in Google account); the instructor said a simple form alone can't do that, but it's possible with other configuration — not covered here.

2. Why saving to Google Sheets is needed

- Every time the form URL is submitted, the workflow runs as a separate instance — if 100 people submit the form, the workflow executes 100 times.
- The plan for this workflow: **Form → Google Sheets → Gmail** (later Google Drive will be added).

3. Adding the Google Sheets node

- Added a **Google Sheets** node after the Form node.
- Resource: "Sheet within document" (a spreadsheet file is the "document"; each tab/page inside it is a "sheet").
- Operation options seen: Append row, Append or update row, Clear sheet, Create sheet, Delete sheet, Delete rows, Get row(s), Update row, Delete sheet/spreadsheet, Create/Delete spreadsheet, etc.
- Credentials: signed in with a Google account ("Sign in with Google") to connect Google Sheets — this can be the same Gmail account used earlier or a different one.

4. Creating the spreadsheet and sheet

- From the node, opened Google Sheets and created a new spreadsheet, initially named "Feedback Sheet," then renamed the document to **"Feedback Document"** (also referred to at one point as "Agentic AI").
- Inside that document, selected/created the sheet to use (multiple sheets/tabs can exist inside one document, similar to Excel workbook tabs).
- Back in the node, selected the document "From list" (it auto-populated since it was tied to the signed-in Google account) and selected the sheet from list.
- Noted: if a Google account has multiple sheets/documents, the "From list" option shows a dropdown to choose; there's also a "From URL" option where you paste the document link directly.

5. First attempt — no columns / mapping error

- Initially saved the sheet with no column headers, to intentionally demonstrate a common mistake.
- Executing the Google Sheets node threw an error: "no columns found in Google sheet" and later "Parameter document is required / Parameter sheet is required" (because the document/sheet hadn't been selected yet in the node).
- Fix: went into the actual Google Sheet and manually created column headers matching the form field names exactly: **name, email, mobile number, feedback** (copy-pasted the exact names to guarantee an exact match — recommended approach to avoid mapping issues).
- Explained: if a form field name doesn't exist as a column, n8n just skips it; you don't need every form field to have a matching column — only the ones you want stored (e.g., you could choose to store only email and mobile number).

6. Mapping column mode

- After adding columns, running the node still threw: "at least one value has to be added under values to send," because the mapping mode was set to manual (each column mapped by hand).

- Fixed by switching mapping mode to **"Map automatically"** — this works as long as the incoming field names match the Google Sheet column names exactly.
- n8n's own guidance: if form field names differ from sheet column names, use an **Edit Fields** node (used in a previous session) to rename fields before they reach Google Sheets.
- After this fix, executing the node worked and the submitted data appeared correctly in the Google Sheet (name, email, mobile, feedback, submission timestamp, and form mode i.e. "test").

7. Append vs. Append or Update — live demonstration

- With **Append row** selected: executing the node multiple times simply added a new row each time (tested up to 4 executions — got 4 rows, no problem).
- Switched operation to **Append or update row**: at first, without specifying a matching column, it kept adding new rows instead of updating — this is because n8n needs a **"Column to Match"** to know whether a record already exists.
- Set "Column to Match" to **email** (compared to a primary key in databases).
- Re-tested: submitting the same email again did NOT add a new row — instead it updated the existing row's fields (e.g., changed feedback text from "good" to a new value) while keeping the same row/name/timestamp for matching fields.
- Confirmed: if the email differs, even if all other fields (name, mobile, feedback) are identical, a new row is added, since matching is based on email only.
- A student asked about matching on more than one column (composite key); the instructor said n8n doesn't offer this directly in the node, though there may be an indirect way — left as something to check in a future session.

8. Google Sheets write-rate limits and Gmail's daily send limit

- Explained that Google Sheets enforces a limit on how many write ("right") operations can happen within a short time period (e.g., per minute) — if too many happen at once (such as 100 people submitting a shared form URL simultaneously), Google may reject/limit the requests.
- Update operations count as write operations too, same as inserts.
- Separately explained Gmail's sending limit: a personal Gmail account can send **around 500 emails per day** maximum. If a workflow tries to send more than that, Google may flag the account as sending spam and **block the account**.
- The workaround real-world senders use: don't send all emails in one burst — space them out (e.g., one email every 1-3 minutes) so Gmail sees it as normal, non-automated-looking activity rather than a spam campaign.

9. Adding the Wait node

- To handle these rate limits, introduced the **Wait** node: "Wait before continue with execution" — pauses the workflow for a specified time before moving to the next node.
- Wait options include: after time interval, at a specified time, on webhook call, or on form submission.

- Configured: **Wait for 65 seconds** (using "after time interval").
- Positioned the Wait node **between the Form node and the Google Sheets node** — disconnected the existing Form → Google Sheets link and rewired it as Form → Wait (65 sec) → Google Sheets.
- Tested: after submitting the form, the workflow visibly paused ("waiting") for 65 seconds before the Google Sheets append happened.
- Clarified (important): the 65-second wait applies **per individual workflow instance/request**, not globally across all users. Each form submission triggers its own separate workflow instance, so multiple people submitting at the same time each get their own independent 65-second wait running in parallel — it does NOT queue people one after another.

10. Live class experiment — the concurrency bug

- Instructor published the workflow to production and asked several students (5-10, deliberately not all ~100) to submit the form at roughly the same time, using the "Append or update row" operation (matched on email).
- Result: some data went missing / got overwritten. At least one student's submitted entry never appeared in the sheet, and another student's name appeared to be duplicated/entered twice.
- Discussion of cause: when multiple requests hit Google Sheets nearly simultaneously, the "Append or update" operation's existence-check logic could not always properly detect/handle near-simultaneous writes, causing an overwrite ("abnormal behavior" — described as a bug/edge case, not expected behavior).
- A student researched this and reported that per Google's Sheets API behavior, requests are ordered by millisecond-level timestamps, and it's very rare for two requests to land on the exact same millisecond — but in that rare case, the last write can win and overwrite another update, causing lost data.
- Instructor's conclusion/fix: switching back to plain **Append** (instead of Append or Update) avoids this problem entirely, since append operations don't need to check/match existing rows — every submission always becomes a new row, so nothing gets silently overwritten.
- Re-tested with Append only under simultaneous submissions from students — all submitted records appeared correctly with no data loss.
- Bigger-picture lesson: even with Append fixed, Google Sheets' write-rate limits remain a concern at higher scale (e.g., true "lakhs" of users). The Wait node only staggers each user's own request — it does not solve simultaneous mass write-rate limiting or true concurrent-write handling. For that, the instructor said you need a **real database**, not a spreadsheet, since spreadsheets/file-based systems aren't built for high-concurrency writes the way databases are.

11. Adding the Gmail "thank you" confirmation email

- Added a **Gmail** node after the Google Sheets node (Gmail credentials were already configured from an earlier session).
- Operation: Send a message.
- Configured:

- To: the submitter's email (dynamic from form data)
- Subject: "Thank you for your feedback" (with the person's name inserted)
- Message body: a "Hello {name}, thank you for your feedback, it's very valuable for us" style message, plus a line about downloading course material (attachment to be added in a future session via Google Drive)
- Tested by having a student submit the form using their real email and confirming they received the thank-you email after the Wait period completed.
- Briefly mentioned (not built): you could add an If/Else node after the Google Sheets node to send a different message depending on whether the row was appended or not — left as a concept, not implemented in this session.
- Also briefly discussed: n8n does not automatically validate input formats (e.g., restricting mobile number to exactly 10 digits) — a student found that entering an unusual number caused it to display in exponential notation. The instructor noted this would require some manual logic/coding (e.g., an Edit Fields node or a dedicated mobile-number-parser node) since n8n is "low-code," not fully no-code.

Key takeaways

- **Google Sheets node — Resource "Sheet within document":** a spreadsheet file is the document, and each tab inside it is a sheet.
- **Append row** always adds a new row per execution — simple and safe, but can create duplicate records for the same person.
- **Append or update row** needs a **Column to Match** (e.g., email) to decide whether to update an existing row or add a new one — without specifying this, it behaves just like Append.
- **Map automatically** works only when incoming field names exactly match the Google Sheet column names; otherwise use an **Edit Fields** node first to rename fields, or map manually.
- Google Sheets has write-rate limits, and Gmail personal accounts have a daily send limit of roughly **500 emails/day** — exceeding these can get automated activity flagged as spam or blocked.
- The **Wait node** ("wait for 65 seconds" in this demo) helps pace individual requests, but it waits **per workflow instance**, not globally — it does not serialize or queue all users' requests together.
- Live testing revealed a real bug: **"Append or update"** can lose/overwrite data when many people submit at nearly the same time, because Google Sheets' API resolves near-simultaneous writes by timestamp and the last write can win.
- Fix demonstrated: use plain **Append** to avoid this overwrite bug when concurrent submissions are expected.
- For true high-concurrency, production-scale systems (many simultaneous users), a real **database** should be used instead of Google Sheets — spreadsheets are not designed to handle that kind of concurrent write load.
- Next session (per the instructor): adding Google Drive to attach/send files (e.g., course material) as part of the thank-you email flow.

Demo 13: Sending Feedback Form Attachments via Gmail and Google Drive in n8n

This video shows how to build a feedback form workflow in n8n that saves responses to Google Sheets and automatically emails the submitter a file (downloaded from Google Drive) as an attachment — plus why local files won't work when n8n is self-hosted but the workflow needs cloud storage.

What you'll learn

- How Gmail Trigger + Google Sheets can be combined to auto-log incoming emails (a workflow the instructor built earlier and left running)
- The difference between "Append row" and "Append or Update row" nodes in Google Sheets, and when to use "Column to Match On"
- How to build a Form (n8n Form node) with Name, Email, and a text-area Feedback field
- How to map form fields to Google Sheet columns manually when field names/counts don't match
- Why local file paths (e.g., an E: drive path) do NOT work as email attachments when n8n is cloud-hosted (or generally not reliable) — files must come from a cloud location like Google Drive
- How to use the Google Drive node to download a file, then attach that binary data to a Gmail "Send a message" node
- How to publish/activate a workflow so it runs continuously instead of only in test mode
- How to export a workflow as JSON and import it into another workflow/project

Step-by-step notes

1. Context: an already-running Gmail-to-Sheet automation

The instructor showed a simple 2-node workflow already published and running since the morning:

- Node 1: **Gmail Trigger**, configured to check for new mail every 1 minute.
- Node 2: **Google Sheets** (append), which stores each new incoming email into a sheet.
- By the time of the demo, ~41 emails had been captured automatically. Read/already-processed mails are not touched again.
- Key point discussed: a workflow must be **published** ("made live") to run automatically/continuously. Without publishing, you can only use the **test URL**, and you'd have to trigger it manually every time.
- A student raised a concern about Gmail rate limits/blocking if replying to too many emails too quickly — instructor noted normal reply volume (10-15) is not an issue, but very high-speed bulk replies could look like abnormal activity to Gmail.

2. Building the new "GD workflow" (Google Drive demo)

Started a brand-new workflow to demonstrate Google Drive integration.

3. Create the Form

- Added a **Form** node ("On form submission" trigger), then clicked "New".

- Set **Form Title**: "Feedback form" (also mentioned as material/feedback request form).
- Added form elements:
- **Name** — type: text (marked required via "Add attributes" → required field).
- **Email** — type: Email (marked required).
- **Feedback** — type: Text area (for multi-line input), not marked as required in the same way as the others.
- Tested the form in test mode by submitting a sample entry (name, email like "online@gmail.com", and feedback text).

4. Save form responses to Google Sheets

- Added a **Google Sheets** node, operation: **"Append or Update Row in Sheet"** (not plain Append), because if the same email already exists, it should update instead of duplicate.
- Selected credential (already configured from a previous session — just Google login needed).
- Selected **Document**: a new spreadsheet named "material request" (could also be created directly from the node using the "Create Sheet" option instead of pre-creating it).
- Selected the **Sheet** inside that document.
- Important lesson: the Google Sheet columns were named differently from the form fields (sheet had "Name of the student" and "Email ID" instead of "Name"/"Email"), and the form actually produced 5 fields internally (though only 3 were filled) vs. only 2 columns in the sheet.
- Because names/counts didn't match, the instructor used **"Map each column manually"** to explicitly map:
- Form's "Name" → Sheet's "Name of the student"
- Form's "Email" → Sheet's "Email ID"
- Feedback field was left out — only these two fields were actually saved to the sheet.
- Set **"Column to Match On"** to the Email ID column — this tells "Append or Update" to check if that email already exists and update that row instead of creating a duplicate.
- Executed the node and confirmed the row was saved correctly with only the required fields.
- Takeaway confirmed with the class: form field names do NOT need to match sheet column names exactly — mapping is done manually in the node's configuration panel.

5. Add a Gmail node to reply with the material

- Added a **Gmail** node, operation: **"Send a message"** (not "Reply", since this is a new outgoing email, not a reply to an existing thread).
- Configured:
- **To**: the email captured from the form.
- **Subject**: something like "Thanks for providing feedback, please find/download the material."
- **Email type**: Text (HTML was mentioned as an alternative option).
- **Message body**: e.g. "Hello [Name of the student], thank you for providing valuable feedback. Please download the last session workflow attached."

- Optionally used a "format in a promotional way" copy-paste helper to make the message look neater.

6. The mistake: trying to attach a local file path

- Instructor tried to attach a file by pasting a local file path (from the E: drive, e.g., a "loan eligibility workflow.json" file saved earlier).
- Result: **this does not work.** Reason (confirmed by a student): the file is on the **local machine**, but n8n (self-hosted in this case, but same logic applies for cloud n8n) needs the file to come from somewhere accessible — a cloud location, not a local disk path.
- Lesson: Local files are not reachable by the node when the workflow runs; you must host the file somewhere accessible (like Google Drive) and download it as part of the workflow instead of referencing a local path directly.

7. Add a Google Drive node to download the file

- Added a **Google Drive** node **before** the Gmail node (disconnected it first, inserted the Drive node, then reconnected).
- Options seen in the Google Drive node: Copy file, Create file from text, Delete a file, **Download a file**, etc. — selected **Download file**.
- Configured:
- Credential: same Google login used for Gmail/Sheets.
- Resource: File; Operation: Download.
- File selection: chose "From list" and selected the previously-uploaded "loan eligibility workflow.json" file from Google Drive.
- Executed this node alone to confirm the file downloaded successfully (as binary data).
- Used the toolbar's **"Tidy up"** option to auto-arrange/format the node layout on canvas (other options noted: zoom to fit, zoom in, zoom out).

8. Attach the downloaded file to the Gmail node

- Important concept: files coming from nodes like Google Drive in n8n are handled as **binary data**, not as file paths/names.
- In the Gmail node, added an **Attachment** field (via "Add option" → Attachments) and set it to reference the binary property name (e.g., "data") coming from the previous Google Drive node's output.
- If multiple attachments are needed, you add multiple binary property names separated by commas (e.g., "data, data2") — or use "Add another attachment."
- Executed the full workflow — mail was sent and received successfully **with the attachment included**, confirmed by checking the inbox.

9. Publish and test live with the class

- Instructor clicked **Publish** to make the workflow live/production so students could submit the form and actually receive the attachment by email.
- Noted the workflow had no **Wait node**, so there could be a short delay/some requests might not process instantly if many people submit near-simultaneously.
- Students submitted the form, and several confirmed receiving the reply email with the attached file.
- The Google Sheet accumulated entries showing how many people had requested/downloaded the material — described as a simple built-in tracking mechanism (count of submissions in the sheet).

10. Exporting and importing workflows as JSON

- To share a workflow, use the three-dot menu on a workflow: options include Edit, Description, Duplicate, **Download**, Share, etc.
- Downloading produces a **.json** file representing the entire workflow (JSON = JavaScript Object Notation).
- To reuse it: create a new workflow, then use **"Import from file"** (other options seen: Import from URL) and select the downloaded JSON — this recreates all the nodes without manually rebuilding them.
- Note: after importing, you must reconfigure your own credentials (Gmail, Sheets, Drive) in your own n8n instance/account for it to run, since credentials aren't shared in the JSON.

11. Related Q&A points covered briefly

- Downloading multiple files from Google Drive in one node isn't natively obvious — instructor suggested using multiple Google Drive nodes (one per file) if a single node can't handle multiple files, or exploring options like zipping files. Suggested checking ChatGPT for specifics.
- n8n supports database nodes for various databases (Oracle, MySQL, MongoDB were named) — you configure connection details in the node's panel; a database node will be covered in an upcoming session.
- Confirmed it's possible to send the same file via Gmail, WhatsApp, and Telegram nodes in one workflow (three separate nodes for three channels).
- For bulk sends (e.g., to thousands of users), a **separate workflow instance runs per user/trigger event** — this is how n8n scales per request.
- Mentioned upcoming topics: WhatsApp automation, Telegram automation, MCP, agentic frameworks, Gemini configuration, image generation, and later — coding-based automation using Python/LangChain as an alternative to no-code n8n platform automation.
- Briefly clarified that "Notebook LM" (a Google tool for turning documents into video/audio-like content) is a different concept from "LLM" (the underlying language model) — a student had confused the two; instructor said Notebook LM specifics would be covered separately.

Key takeaways

- Publishing (activating) a workflow is required for it to run continuously/automatically on its trigger schedule — unpublished workflows only work via manual test runs.

- Use **Google Sheets "Append or Update Row"** with a **"Column to Match On"** (like Email ID) when you want to avoid duplicate rows and instead update existing ones.
- Form field names and spreadsheet column names do not need to match — use **"Map each column manually"** to connect them explicitly.
- Files attached to emails in n8n must be **binary data from a cloud source** (like Google Drive) — local file paths on your machine will not work as attachments, regardless of whether n8n is self-hosted or cloud-hosted.
- The typical pattern for "send a file in response to a request" is: Form (trigger) → Google Sheets (log the request) → Google Drive (download the file) → Gmail (send the file as an attachment).
- Workflows can be exported/downloaded as JSON files and imported into other n8n instances/projects, but credentials must be reconfigured manually after import.
- Each trigger event runs its own separate workflow instance, which is how n8n handles sending to many users at once (e.g., WhatsApp broadcast to thousands).

Demo 14: Using OpenRouter for Free/Multiple LLM Models with a Gmail-Triggered AI Agent

This video shows how to sign up for OpenRouter (a third-party platform that gives access to hundreds of LLM models, including free ones) and use it inside an n8n workflow so that a Gmail trigger feeds an AI Agent, which then auto-replies to incoming emails.

What you'll learn

- What OpenRouter is and why it's useful (avoiding "quota exceeded" errors from a single provider like OpenAI or Gemini)
- How to sign up on OpenRouter and generate a free API key
- How to connect an OpenRouter Chat Model as the "brain" of an n8n AI Agent
- How to build a workflow: Gmail Trigger → AI Agent → Gmail node (send reply)
- How to configure the Gmail Trigger (poll interval, "On message received")
- How to reference the incoming email's data (subject, snippet) as input to the AI Agent
- How to send the AI Agent's output back as an email reply
- A plain-language introduction to what RAG (Retrieval Augmented Generation) means, for context (to be covered in detail later)

Step-by-step notes

1. Why use OpenRouter

- Problem: when practicing with a single LLM provider (OpenAI, Gemini, etc.) you often hit "limit exceeded" errors.
- OpenRouter is a third-party vendor/aggregator that provides access to many different LLM models (the trainer mentioned somewhere around 650+ models total) through one account and one API key.
- If one model's free quota runs out, you can simply switch to another free model in OpenRouter instead of being stuck — similar to swapping a SIM card when one runs out of balance.
- OpenRouter offers both free models and paid models. You can pick based on your needs.
- Note discussed: OpenRouter itself doesn't maintain conversation "context" across different models — switching models means starting fresh context, so this approach is mainly recommended for practice/learning, not for maintaining long conversations.

2. Building the demo workflow

The trainer built a new workflow with the following nodes, in order:

197. **Gmail Trigger** node

- Trigger event: "On message received" (this is the only relevant trigger option — it fires when a new mail arrives).

- Credential: uses Gmail OAuth login (not an API key) — you log in with your Gmail account, not a key.
- **Poll Times**: set to "Every Minute" for testing purposes. Other options available: every hour, day, week, month, or custom. Poll time controls how frequently n8n checks the Gmail inbox for new messages (n8n cannot get an instant push notification the moment a mail arrives — it has to check on an interval).
- The trigger checks which mails have already been processed vs. new ones, so it won't "miss" a mail that arrived between polls, and won't double-reply to already-read/processed mail (as long as it's tracking processed status correctly).
- Discussion point: to avoid auto-replying to spam or to already-answered threads, you'd need extra filtering logic (e.g., checking if the subject already starts with "Re:" to skip replies that are already part of an ongoing conversation) — mentioned as something you *can* configure, not built in this demo.
- Trainer tested the trigger with "Fetch Test Event" to confirm it pulled a sample Gmail message.

198. **AI Agent** node

- By default, the AI Agent node expects a "Chat Input" (JSON chat input), which is meant for chat-trigger-style workflows.
- Since this workflow is driven by the Gmail Trigger (not a chat trigger), you must change the source: set the prompt to **"Define below"** instead of expecting chat input.
- In the "User Prompt" field, the trainer wrote a prompt like: *"Act as an experienced technical trainer. Explain and analyze..."* followed by inserting the incoming email content.
- The actual email text is found in the **`snippet`** field of the Gmail Trigger output — this is what was inserted into the prompt so the AI Agent knows what question/content to respond to.
- (Note: in the demo, one test email had no message body, only a subject line, so the snippet came through empty for that message.)

199. **Chat Model (for the AI Agent)**

- Click into the AI Agent's Chat Model selector — you'll see many built-in options like Anthropic Chat Model, Azure OpenAI, AWS, etc.
- Select **"OpenRouter Chat Model"** from the list.
- This requires an OpenRouter credential (API key) to be set up; without it, executing the node throws an error: *"Node OpenRouter Chat Model does not have access to the credential."*

3. Getting an OpenRouter API key

200. Go to **openrouter.ai** (searching "open router" in Google gets you there).

- The site describes itself as "the unified interface for LLMs."

201. Sign up / log in using your Gmail account (no long questionnaire — just Gmail sign-in).
202. Once logged in, find the **"Get API Key"** option (visible from the homepage).
203. Click to create a new key (the trainer named his key something like "n8n workflow 2").

204. **Important:** Copy and save the key immediately — OpenRouter (like most providers, e.g. Gemini, OpenAI) only shows you the key once at creation time. If you lose it, you must generate a new key; you cannot retrieve the old one.

4. Adding the credential in n8n

205. Double-click the OpenRouter Chat Model node in n8n.
206. Under Credentials, click **"Setup Credential"** (shows "No credentials yet").
207. Paste in the API key you copied from OpenRouter.
208. Click **Save** — you'll see "Credential successfully created."
209. This resolves the earlier "does not have access to credential" error.

5. Exploring available models on OpenRouter

- On the OpenRouter homepage, under "Models," the trainer showed the breakdown (approximate figures shown live): ~632 text models, ~17 image models, ~25 embedding models, 5 audio models, 4 video models, 3 rerank models — roughly 650+ models total.
- You can search/filter models, e.g. searching "free" shows about 26 free text models (examples mentioned: OpenAI GPT-OSS 120B free, GPT-OSS 20B free, various Google/Gemini free models).
- **Important:** model selection inside the OpenRouter Chat Model node in n8n is manual — you have to pick which model to use yourself; it does not auto-switch models for you if one hits its limit.
- If a selected model returns an error like "too many requests" or hits its quota, you can manually change to a different free model in the node's model dropdown and re-run.
- Clarification on API keys: the OpenRouter API key authenticates your connection to OpenRouter itself (so it can route your request to whichever model you pick) — it is not a separate key per model. One OpenRouter key gives access to all its available models.

6. Sending the reply email

210. Add a **Gmail** node after the AI Agent, set to send an email (trainer used "Send a message"; noted there's also a "Reply to message" option he ran into some difficulty configuring live, so he used a plain Send for the demo).
211. **To**: mapped from the original sender's email address (from the Gmail Trigger's data).
212. **Subject**: set to something like "Reply to [original subject]" (the trainer typed "Reply to..." and referenced the trigger's subject).
213. **Email Type**: Normal text message.
214. **Message body**: constructed by combining:

- A greeting line ("Hello friend, please find the answer for your question:")
- The original question — pulled from the Gmail Trigger's `snippet` field
- The AI Agent's answer — pulled from the AI Agent's `output` field

215. Executed the workflow — confirmed the reply email was successfully received by test participants who had emailed in.

216. Published/activated the workflow so it would run live (polling every minute) as students sent test emails to the configured inbox.

7. Live test results

- Students sent test emails to the trainer's configured Gmail address with random technical questions (e.g., about n8n, Python, OOP).
- Within roughly a minute or two (due to the poll interval plus model response time), several students received auto-generated reply emails from the AI Agent via OpenRouter.
- Some delay/failures were attributed to free model rate limits ("too many requests") — solved by switching to a different free model in the OpenRouter Chat Model node.

8. Brief conceptual aside: what is RAG?

- Explained informally (not implemented in this demo, just introduced conceptually for a future session):
- Normally, an LLM answers using its own trained general knowledge.
- With RAG (Retrieval Augmented Generation), you instead provide specific documents (e.g., a company's policies) and instruct the model: "Don't use your general knowledge — answer only based on these documents."
- Example given: a health insurance company wouldn't want a generic LLM answer; it would feed the model its specific policy documents so answers are always accurate to the company's own information.
- Technical description given: relevant data is retrieved from a vector database (as embeddings), that retrieved content is "augmented" onto the user's question, and both are handed to the LLM, which generates the final response.
- The trainer also briefly distinguished this from MCP (mentioned only as "we'll discuss compulsorily later, it's an important hard concept") — not explained in this video.

Key takeaways

- OpenRouter lets you access hundreds of LLM models (many free) through a single account and single API key, which is very useful for avoiding "quota exceeded" issues while practicing.
- The Gmail Trigger's poll interval determines how often n8n checks for new mail — it cannot receive instant push notifications, so there will always be some delay (the trainer used 1-minute polling for testing).
- When connecting an AI Agent to a non-chat trigger (like Gmail), switch the prompt source to "Define below" and manually map the input from fields like `snippet`.
- To connect OpenRouter in n8n: select "OpenRouter Chat Model" as the AI Agent's chat model, then create/paste an OpenRouter API key as a new credential.
- Save your OpenRouter API key immediately upon creation — it cannot be viewed again later.
- Model selection in OpenRouter is manual; if one model errors out or hits quota limits, you must manually pick another (often another free one).

- Automating repetitive communication tasks (auto-replies, WhatsApp check-ins, status alerts, etc.) was discussed as a broader theme — using workflows to remove manual repeated work from daily life/business.
- RAG (Retrieval Augmented Generation) means restricting the LLM's answers to specific provided documents/data instead of its general knowledge — introduced conceptually here, to be covered in more depth in a future session, along with MCP and WhatsApp/Telegram automation.

Demo 15: Building a Resume Screening AI Agent (Job Filtering Workflow) in n8n

This session builds a realistic end-to-end use case: an n8n workflow where a candidate submits a job application form (with a resume PDF), an AI agent analyzes the resume against a job requirement, and the workflow automatically emails the candidate and HR based on whether the candidate is eligible or not.

What you'll learn

- How to design a real-world "resume screening" automation using a Form Trigger, an AI Agent, an If node, Gmail nodes, and Google Sheets
- How to convert an uploaded PDF (binary data) into text/JSON so an AI model can read it, using the **Extract from File** node (Extract from PDF operation)
- How to write a **system message** for an AI Agent that defines its role, the job requirements, and rules for analysis
- How to force an AI Agent to return a **fixed, structured output** (JSON) instead of free-text description, using **"Require Specific Output Format"** and the **Structured Output Parser**
- Why boolean (true/false) output is better than plain "yes/no" text when feeding results into an If node
- How to route eligible vs. non-eligible candidates using the **If node**, and send different Gmail messages down the True and False branches
- How to save shortlisted candidate details into **Google Sheets** using Append Row with manual column mapping
- How to try attaching the resume PDF to an HR notification email (and a real error encountered while doing this)
- General best practice: execute nodes one at a time while building, then run the full workflow at the end

Step-by-step notes

1. Define the use case

The scenario: a company posts a job opening, candidates submit resumes, and instead of HR manually opening thousands of resumes, an AI agent screens each resume against the job description (JD) and decides eligibility. The workflow should then:

- Save eligible candidates to a Google Sheet
- Email HR when a good candidate is found
- Email the candidate to say they are shortlisted (if eligible) or not eligible (if not)

This is explicitly presented as a **draft/idea version**, not a fully optimized production workflow — extensions (storage, more automation, better prompts) are left for later.

2. Create the Form Trigger

- Add a **Form Trigger** node (n8n's "On form submission" event).
- Authentication: set to **None** (not required for a public form).
- Form Title: `Job Application Form` (this is the form shown to the candidate/job seeker).
- Add form elements (fields), in this order:

217. **Full name** — Input type: Text, Required: Yes
218. **Email** — Input type: Email, Required: Yes (needed to send further communication)
219. **Mobile number** — Input type: Number, Required: Yes
220. **LinkedIn profile** — Input type: Text, Required: **No** (optional, since not everyone has one)
221. **Resume** — Input type: **File**, Required: Yes. The label was written to mention "Resume in PDF" so the agent later knows to expect a PDF file specifically.

- Executed the form: it correctly showed Full Name, Email, Mobile Number, LinkedIn Profile (no asterisk since optional), and Resume (expects PDF). Filled in test data and uploaded a PDF resume file — the node executed successfully.
- Note: a participant asked about file size restriction (e.g., PDF < 2MB) — Durga said that option needs to be explored further; not covered in this session.

3. Problem: AI Agent cannot read a PDF binary file directly

- Adding an AI Agent node right after the form and pointing the user message at the resume file did not work — the file wasn't accessible as plain text because it's stored as **binary data**, and AI models can only understand text/JSON, not raw binary/PDF.
- Solution: add an **Extract from File** node between the Form Trigger and the AI Agent.
- In the node's operation list, choose **Extract from PDF** (other options included Extract from CSV, HTML, ICS, JSON, ODS).
- Set **Input Binary Field** to the binary field name coming from the form (the resume's binary data field).
- Executed the node — the resume text became readable in a **text** output variable, confirming the PDF-to-text conversion worked.

4. Build the AI Agent to analyze the resume

- Add an **AI Agent** node after Extract from File.
- **User Message (Source for Prompt)**: set to "Define below" (not connected to a chat trigger), and set the message to the extracted resume text.
- **Chat Model**: added an **OpenAI Chat Model** (GPT model) connected as the agent's brain, using the credential/API key already configured in an earlier session. (Mentioned that Gemini or other chat models could also be used.)
- **System Message** (added via the agent's "Add Option" → System Message): written as the recruiter's instructions, e.g.:

- "Act as a resume filter assistant. Analyze the provided resume for a Java trainer requirement with a minimum of 8 years of relevant experience."
- Additional rules added: "Must have strong knowledge in Java-related technologies," "should have relevant experience," "teaching experience must be essential."
- Key lesson: the system message is where the job description (JD) and screening rules live — the quality/specificity of this prompt directly determines how accurately the agent filters.
- First test: uploaded a resume for an ASP/SQL/HTML trainer with 10+ years experience but **no Java experience**. The agent correctly responded that the candidate was "Not suitable for Java trainer role" with a text summary explaining the mismatch (Java requirement not met despite strong teaching background).

5. Problem: agent's answer was in free-text/description form, not usable by the If node

- The agent's response was a paragraph description, not a clean variable — this makes it hard for downstream nodes (like If) to check "is this candidate eligible?" programmatically.
- Solution: configure the agent's output format.
- In the AI Agent node, find the option **"Require Specific Output Format"** (disabled by default) and **enable** it.
- This exposes an **Output Parser** option. Connect an output parser node on the canvas.
- Parser choices seen: Auto-fixing Output Parser (deprecated, skip), Item List Output Parser, and **Structured Output Parser** ("Return data in a defined JSON format") — this is the one selected, since JSON output is what's needed.
- In the Structured Output Parser, provide a **JSON example** defining the expected fields:

```
{
  "eligibility for job": "yes/no (later changed to boolean true/false)",
  "reason": "string",
  "summary": "string"
}
```

- Only the **left-side variable names** matter here (the actual values are filled in by the agent after analysis).
- Re-executed the AI Agent node alone (recommended: build/test node by node, then run the whole workflow at the end) — output now returned as structured JSON with `eligibility for job`, `reason`, and `summary` as separate fields instead of a paragraph.

6. Fix eligibility field to use boolean instead of text

- Initially the eligibility field was text ("yes"/"no"/"eligible"/"not eligible"), which caused confusion in the If node (comparing strings like "not eligible" against "yes" behaves unexpectedly).
- Fix: change the JSON example field type to **boolean** (`true`/`false`) instead of a string. This removes the ambiguity — the If node condition can now use "Boolean → is true" directly instead of string comparison.

7. Add the If node

- Add an **If** node after the AI Agent.
- Condition: `{{$json.eligibility for job}}` — data type **Boolean**, operator **is true**.
- If eligibility is `true` → **True branch**; if `false` → **False branch**.
- Tested with the non-Java resume: correctly routed to the **False branch**.

8. False branch (not eligible) — send rejection email

- Add a **Gmail** node → operation **Send a Message**.
- **To**: the candidate's email from the form submission (`{{$json.email}}` from the form).
- **Subject**: e.g. "Sorry [Name], you are not eligible" / "Better luck next time."
- **Message body**: a polite rejection message, e.g. "So sad to say you are not eligible for this job. Better luck next time and looking forward to work with you in future. Regards, HR team."
- Executed — confirmed the rejection email was received. Also found and **disabled the "Append Attribution"** option in the Gmail node to remove the auto-added "sent via n8n" footer text.
- A student asked about delaying the email 15 days before sending — Durga confirmed you could add a **Wait node** (covered in an earlier session) for that purpose, though it wasn't added into this workflow.

9. True branch (eligible) — save to Google Sheets

- Add a **Google Sheets** node → operation **Append Row** (not update).
- Selected the target spreadsheet (e.g., "shortlisted candidates") and sheet tab (e.g., "Java").
- Chose **Map each column manually** (recommended when multiple fields are involved).
- Mapped columns: Full Name, Email, Mobile Number, LinkedIn Profile, Reason, Summary — all pulled from the AI agent's structured output and the original form fields.
- Tested with a genuine Java trainer resume (10+ years Java experience) — the agent correctly returned `eligibility: true`, with a summary confirming the candidate met/exceeded requirements, and the row was appended to the Google Sheet successfully.

10. True branch — notify HR and notify candidate

- Added a second **Gmail node** to notify HR: "To" = HR email, Subject: "One resume waiting for your review," Message: candidate's name, email, mobile number, reason, and summary (built by copy-pasting field values from the previous node's output rather than typing raw expressions, to avoid mistakes).
- Added a third **Gmail node** to notify the candidate: Subject "Congratulations, you are shortlisted," Message informing them their resume was shortlisted for the job and that HR will contact them soon regarding further steps.
- Executed the full workflow end-to-end with a real Java resume — confirmed both emails were sent successfully, and multiple students tested with their own resumes (Java resumes got shortlisted, non-Java resumes like C++ or J2EE got correctly rejected).

11. Attempted enhancement: attach the resume PDF to the HR email

- Tried adding the resume as an attachment on the HR Gmail node using the binary data from the "Job application form" / "Resume in PDF" node output.
- This produced an error: *"The operation expected node input data to contain a binary file but none was found in item."*
- Diagnosed as a binary-data-reference issue (the binary property name/path wasn't correctly pointed to, possibly needing something like `.0.fileName` or similar array indexing). Durga acknowledged this was unresolved in the session and said he'd fix and share the corrected workflow afterward (attaching resumes was left as an open issue, not solved live).

12. Other Q&A points covered

- A student's separate version of this workflow (with a percentage-match rule like "eligible if match > 85%") gave inconsistent results (85% vs 64%) on the same resume across two runs — Durga explained this happens because AI models aren't fully deterministic for numeric/calculation-heavy judgments, and advised making the system prompt much more specific and explicit (e.g., clearly defining required vs. optional skills and exact matching logic) to get consistent results; also mentioned tools/ATS-score-style scoring as a possible future improvement.
- Confirmed resumes/data are **not stored anywhere in n8n** — the workflow just processes data in runtime memory during execution and discards it (no persistent storage unless you explicitly add one, e.g., a database).
- Confirmed each node can be renamed with a custom name for clarity.
- Confirmed CC/BCC options exist in the Gmail node (e.g., to add a manager's email) via "Add Option."
- Discussed a further use-case idea from a student (bulk offer-letter emails to multiple shortlisted candidates from a Google Sheet) — Durga confirmed this kind of automation is possible but was not built in this session.
- General closing remarks: n8n has 500-600+ integrated apps, so most "search for a tool" needs are unnecessary — you just describe what you want in the agent's system message/brain, and can add extra tools to the agent as needed.

Key takeaways

- AI models can only process text/JSON — any binary file (like an uploaded PDF resume) must first be converted using a node like **Extract from File → Extract from PDF** before an AI Agent can read it.
- A good **system message** (job description + explicit rules) is critical — vague prompts lead to inconsistent AI judgments, especially for percentage/scoring-based logic.
- To make an AI Agent's output usable by downstream logic nodes (like If), enable **"Require Specific Output Format"** and attach a **Structured Output Parser** with a JSON example defining your exact field names.
- Prefer **boolean (true/false)** fields over free-text yes/no strings for conditions you'll check in an If node — it avoids string-matching bugs.

- Always test/execute nodes one at a time while building a workflow, then execute the full workflow at the end to verify everything together.
- This workflow is a **draft/proof-of-concept**, not production-ready — real-world versions would need refinements like resume attachment handling, storage, stronger prompts/scoring logic, and wait/delay steps.
- Nothing uploaded through the form is stored by n8n itself — data only exists in memory during a single workflow execution unless you explicitly add storage (e.g., Google Sheets, a database).

Demo 16: Fixing Resume Attachments with n8n's Merge Node, and Building a Scheduled "Message Generator" Workflow

This class first fixes a bug from the previous session's job-application workflow (the resume PDF wasn't reaching the HR email), which becomes a deep-dive into how n8n's **Merge node** works. The second half introduces the **Schedule Trigger** node, showing how to make a workflow run automatically and repeatedly (used here to build a joke workflow that sends funny recurring text messages, but with clear real-world uses like reminders and daily reports).

What you'll learn

- Why data can "disappear" partway through an n8n workflow, and how to preserve it
- How the Merge node's **Append** mode differs from **Combine (by position)** mode
- How to attach a file (like a PDF resume) to an email sent later in a workflow
- How to use the **Schedule Trigger** node to run a workflow automatically on a timer
- The difference between "trigger" (schedule-based) and "polling" (checking an inbox for new items)

Step-by-step notes

1. The original problem (from the previous day's workflow):

The workflow was: Job Application Form → Extract from File (converts the uploaded PDF resume into JSON) → AI Agent (analyzes the resume) → IF node (true = shortlisted, false = rejected) → Gmail (send a message to HR with the resume attached).

The instructor wanted to attach the candidate's resume PDF to the HR email, but the variable for the PDF was not available at that point in the flow. This happened because once the "Extract from File" step converts the PDF into JSON text, the original **binary file data** is no longer carried forward automatically — only the JSON is.

2. The fix: use a Merge node.

222. Add a **Merge** node that takes two inputs: one input is the JSON data path (coming from the AI agent/form), and the other input is the original PDF binary data (from "Extract from File").
223. Set the Merge node's **Mode** to **Combine** (not the default "Append").
224. Set **Combine by** to **Position**.
225. This merges the two separate items (one JSON item + one binary/PDF item) into a **single combined item** that contains both pieces of data.
226. Now the Gmail node, which needs the resume attachment, can find it because everything is inside one item instead of being spread across two separate items.

3. Why "Append" mode didn't work:

- **Append mode** (the default) keeps each input as a **separate item** — item 0 (JSON) and item 1 (binary/PDF). When the Gmail node looks for the resume PDF field, it searches item by item, and if

the field isn't in the item it's checking, it throws an error like: *"This operation expects the node's input data to contain a binary file 'resume in PDF' but none was found in item 0."*

- **Combine mode** merges both inputs into **one item**, so all fields (JSON fields + the PDF binary) are available together, and the Gmail attachment field can find the resume.
- Rule of thumb: Append = "give me both items one after another." Combine = "merge everything into a single item."

4. Testing the fix:

227. Submit the job application form with a small test PDF (the instructor mentions using very small files to avoid memory issues with node processing).
228. Watch the workflow execute node by node.
229. Confirm the candidate gets a "you are shortlisted" email, and the HR person receives an email with the resume PDF attached.
230. Test the "false"/rejected branch too — the candidate should get a "sorry, you're not eligible" message, and no resume attachment is needed there.
231. Common last step to remember: actually check the **"Attachment"** checkbox/field in the Gmail node and map it to the correct resume variable — forgetting this was the actual final bug in the session.

5. Alternative approach discussed: Instead of a Merge node, you could add a **Set/Edit node** right after the file is uploaded to make a backup copy of the original binary data in a separate variable, before the AI agent/JSON conversion happens. That backup variable would then be available everywhere downstream (both positive and negative branches), unlike the Merge node approach, which only exists on the one branch where it's placed.

6. New topic: Schedule Trigger node (building a "keep messaging automatically" workflow).

The instructor's example use case: automatically send a caring/funny message to his wife every 2 minutes, generated freshly each time by an AI agent (a light-hearted personal-life automation demo).

232. Create a new workflow and add a **Schedule Trigger** node as the first node.
233. Configure the trigger interval — you can choose Seconds, Minutes, Hours, Days, Weeks, Months, or a custom cron expression.
234. For the demo, set **"Minutes between triggers"** to **2** (fires every 2 minutes).
235. Add an **AI Agent** node after the trigger. Since there's no chat trigger providing the prompt, you set the prompt manually using **"Define below"**.
236. Give the agent a **system message** (persona) and a **user message** — in the demo, the user prompt asked the model to "provide three captions" like playful complaint messages a spouse might send.
237. Add a **Structured Output Parser** to the agent so the output comes back in specific fields — in this case three variables named **C1, C2, C3** (caption 1, 2, 3), each with an example value.
238. Add an **OpenAI Chat Model** node connected to the AI agent as its "brain" (credentials were already configured from earlier sessions).

239. Add a **Gmail → Send a Message** node after the agent. Configure the recipient email, subject ("Important message for you"), and in the message body, insert the C1/C2/C3 variables generated by the AI agent.
240. Test by executing the previous nodes individually if the message body doesn't populate correctly, then run the whole flow.
241. **Publish** the workflow — only once published does the Schedule Trigger actually run automatically and repeatedly ("for testing you can execute manually, but once you publish, it runs on the schedule you configured").
242. The instructor then duplicated the pattern into a second AI Agent + Gmail pair in the same workflow (romantic messages for a "girlfriend" scenario, purely for a fun illustration) to show that a **single Schedule Trigger** can fan out to multiple parallel branches, each running every 2 minutes.
243. Checked the **Executions** panel in n8n to see the history of every 2-minute run, confirming the workflow is actually firing repeatedly and generating different message text each time (because the AI model was asked to vary the wording, not repeat the exact same message).

7. Trigger vs. Polling — an important distinction raised by a student:

- **Schedule Trigger**: Starts the whole workflow (or continues a workflow if placed mid-flow) at a fixed time interval, regardless of whether anything changed.
- **Polling** (e.g., checking a Gmail inbox periodically, discussed in earlier sessions): The node checks periodically (e.g., every 2 minutes) whether something new happened (like a new email); the workflow only proceeds/triggers if there IS an update. If nothing changed, nothing executes.
- A Schedule node is commonly used as the very first (trigger) node, but it can technically be placed elsewhere in a workflow to control timing of a later step too.

8. Real-world use cases mentioned for Schedule Trigger:

- Sending reminders (e.g., a manager wanting a status update from staff at a fixed time daily)
- Bulk birthday/anniversary/festival wishes to a contact list stored in Google Sheets, using AI to generate a unique, personalized message each time
- Payment/bill reminders
- Any "repeat this action forever, on this schedule" personal or business automation

Key takeaways

- If a field seems to "disappear" partway through your n8n workflow, it's often because a conversion step (like Extract from File) dropped the original binary data — plan ahead to preserve or re-merge it.
- **Merge node → Combine mode → Combine by Position** is the standard way to merge two related pieces of data (e.g., JSON + a file) into one item so downstream nodes can access both.
- **Append mode** keeps items separate; **Combine mode** merges them into one — this single setting decides whether attachments/fields are "found" or "missing" downstream.

- The **Schedule Trigger** node lets you fully automate recurring tasks (every N minutes/hours/days/weeks/months, or custom cron) — but the workflow must be **published** for it to run on autopilot.
- **Trigger** = starts execution on a timer regardless of new data; **Polling** = checks periodically and only proceeds if something new is found.

Demo 17: What You Need Before You Can Build a WhatsApp Bot with n8n (Meta/Facebook Setup Overview)

This session is a conceptual walkthrough (not a completed working demo) of everything required on the Meta/Facebook side before you can connect n8n to WhatsApp. It ends with a live group attempt to create a Facebook Business Manager account and a Meta developer app, which the class works through together with mixed success — full technical configuration is promised for the next class.

What you'll learn

- Why WhatsApp automation requires a Facebook (Meta) account and a WhatsApp **Business** account (not a personal one)
- The concept of Client ID / Client Secret credentials for the WhatsApp trigger node
- The step-by-step account/app creation path on Meta's side: Facebook account → Business Manager → Developer App → WhatsApp product
- The overall shape of a WhatsApp bot workflow: WhatsApp Trigger → AI Agent → WhatsApp send-message action
- Real-world example of how a business (like a bank) might use a WhatsApp bot, and the difference between a simple menu-bot and an AI-agent-powered bot

Step-by-step notes

1. Why WhatsApp automation is harder than Gmail automation:

WhatsApp is owned by **Meta** (which also owns Facebook and Instagram; Meta acquired WhatsApp). To programmatically send/receive WhatsApp messages, you must go through Meta's developer platform, which requires several account layers, unlike the simpler Gmail/Telegram setups covered earlier.

2. Prerequisites, in order:

244. **A Facebook account** — mandatory. (The instructor notes he didn't personally use Facebook before but created an account just for this purpose. A brand-new Facebook account may be blocked from immediately creating a Business Manager — Meta may ask you to "wait at least one month" on very new accounts. If you hit this, the instructor's workaround was asking ChatGPT for guidance and switching from email-based to mobile-number-based signup, which worked.)
245. **A WhatsApp Business account** — mandatory for automation; a normal/personal WhatsApp account cannot be used. You do NOT need to buy anything — you can freely convert a personal WhatsApp account to a Business account (all your existing chats and contacts carry over, nothing is lost). Many people run two WhatsApp accounts on one phone using dual SIM: one personal, one business.

- This is also required for things like Facebook/Instagram ads — when a customer clicks an ad and messages you, the receiving number must be a business number.

3. In the n8n workflow, adding a WhatsApp trigger:

246. Add the **"WhatsApp Business Cloud"** trigger node.
247. Choose the **"On messages"** event/trigger option (there are other events like account updates, template status, phone number quality updates, etc. — "On messages" is what's needed for a chatbot).
248. n8n will ask you to set up new credentials, requiring a **Client ID** and **Client Secret**. These come from Meta, not from WhatsApp directly.

4. Getting the Client ID and Client Secret — the full path:

249. **Step 1 — Facebook account**: Already required above; used only for logging in.
250. **Step 2 — Create a Business Manager / Business Portfolio account**: Go to **business.facebook.com**, log in with your Facebook account, and create a business portfolio (give it any business name and provide your business email — it can be the same as your personal email). This is required because apps can only be created under a business portfolio, not a personal account.
251. **Step 3 — Create a developer App**: Go to **developers.facebook.com**, then to the **App Creation** page (URL used in class: `developers.facebook.com/apps/creation`). Click "Create App," give it a name (e.g., "DurgaSoft Bot"), choose a use case (the class selected "Others"), select app type **Business**, provide a contact email, and select the Business Portfolio account created in Step 2. Click "Create App."
252. **Step 4 — Add the WhatsApp product to the app**: Inside the newly created app, find the products list and click **Setup** under WhatsApp (the same is possible for Instagram/Messenger).
253. **Step 5 — WhatsApp Cloud API setup**: Under the WhatsApp product, go to **API Setup**. Here you can "Generate access token" and select a **test WhatsApp business account** (Meta provides a temporary test phone number for development, since a real/permanent number requires a paid plan).
254. **Step 6 — Get the actual Client ID / Client Secret**: Go to **App Settings → Basic**. Here you'll find the **App ID** (= Client ID) and **App Secret** (= Client Secret). Copy both.
255. Paste the App ID into the n8n credential's **Client ID** field and the App Secret into the **Client Secret** field, then Save. This completes the WhatsApp Trigger node's credential setup.

5. The overall workflow shape (explained but not fully built this session):

- **WhatsApp Business Cloud (trigger, "On messages")**: Fires whenever a message arrives at your configured WhatsApp number.
- **AI Agent**: Reads what the customer/user asked and prepares an appropriate reply (this is the "brain").
- **WhatsApp Business Cloud (action node, "Send a message")**: Takes the AI agent's reply and sends it back to the same user on WhatsApp.

6. Real-world example discussed (ICICI Bank scenario):

- A customer messages the bank's official WhatsApp number with "hello."
- The bank's WhatsApp trigger fires, hands the message to an AI agent (or a simpler rule-based bot), which replies with a menu: "Welcome to ICICI Bank. What service do you need? 1. Account balance, 2. Checkbook, 3. ..." etc.
- The class discussed that a **basic menu bot** (fixed set of options) doesn't necessarily need a full AI agent/LLM "brain" — it could be simple branching logic. But a true **AI agent** adds extra capability: it can handle open-ended questions (like "where's the nearest ATM?" or "I lost my debit card, what do I do?") using knowledge fed to it, rather than only fixed menu options.
- Important nuance: for a real business bot, the AI agent should answer using the **business's own data/knowledge base** (not generic LLM knowledge), since customers need accurate, business-specific answers (e.g., real ATM locations, real account processes), not generic AI guesses.

Key takeaways

- WhatsApp automation requires: Facebook account → Business Manager (business portfolio) account → Developer App (with WhatsApp product added) → Client ID (App ID) and Client Secret (App Secret) from App Settings → Basic.
- A normal personal WhatsApp account cannot be automated — it must be a **WhatsApp Business** account (free to convert, no data loss).
- Client ID and Client Secret are essentially API credentials, similar in spirit to how you use an API key to talk to ChatGPT — they let n8n authenticate with Meta's WhatsApp API.
- New Facebook accounts may face temporary restrictions on creating a Business Manager — patience (or waiting some time) may be required.
- The typical WhatsApp bot workflow pattern is: WhatsApp Trigger (on message) → AI Agent (decide reply) → WhatsApp Send Message (deliver reply) — the same trigger→brain→action shape used in earlier Gmail/Telegram automations.
- WhatsApp automation setup is significantly more involved than Telegram, mainly due to all the Meta account/app layers required.

Demo 18: Live Troubleshooting of the WhatsApp Trigger Connection in n8n

Following up on Demo 17's account setup, this session is a live, in-the-weeds attempt to actually connect the configured Meta app to n8n's WhatsApp Trigger node and get an end-to-end WhatsApp bot working. The instructor hits a persistent connection error and the session ends without a full success, but it documents every configuration step in detail, plus troubleshooting steps students used to get partial success.

What you'll learn

- How to fully wire up an n8n WhatsApp workflow: trigger credentials, AI agent, and the send-message action
- Where to find and copy the **Access Token** and **Business Account ID** needed for the WhatsApp action node
- A common real error: *"credential with ID does not exist for type WhatsApp Trigger API"* and how the class worked around it
- How to test using Meta's provided **test phone number**
- Why access tokens generated this way are **temporary** (expire quickly)

Step-by-step notes

1. Recap setup used in this session:

256. In n8n, create a **WhatsApp Business Cloud trigger**, event = **"on messages"**.
257. Remove any old/broken credentials first if re-testing from scratch (Credentials → delete existing WhatsApp accounts).
258. Go to the Meta **Business Manager** (business.facebook.com) — confirm you have a **Business Portfolio** (e.g., "Durga Agents"), since apps can only be created under one.
259. Go to **developers.facebook.com → My Apps → Create App**. Give it a name and contact email, select "Others" as use case, select app type, link it to your business portfolio, click **Create App**.
260. Under the app, add the **WhatsApp** product and click **Setup**.
261. Copy the **App ID** and **App Secret** from **App Settings → Basic** — these are the **Client ID** and **Client Secret** for the trigger node's credentials in n8n. Save the credential — it should show **"Connection succeeded."**

2. Building the AI Agent + reply flow:

262. Add an **AI Agent** node after the trigger.
263. Attach an **OpenAI Chat Model** to the agent (any chat model works).
264. Optionally attach a **Simple Memory** node so the agent can remember context across messages (the instructor noted this is optional and later removed it when it caused a "no session ID found" error).

265. Add a second **WhatsApp Business Cloud** node, this time as an **action** node ("Send a message" operation) to actually deliver the AI's reply back to the user.

3. Getting the credentials for the "send message" (action) node:

266. Go back to the Meta app's **WhatsApp → API Setup** page.
267. Click **"Generate access token"** — this token is **temporary** (roughly half an hour to an hour of validity).
268. Choose to opt in either "current WhatsApp account only" or "all current and future WhatsApp accounts."
269. Copy the generated **Access Token**.
270. Also copy the **WhatsApp Business Account ID**, shown on the same API Setup page.
271. In n8n's WhatsApp action node credential form, paste the Access Token and the Business Account ID, then Save.
272. Configure the action node: **Resource** = message, **Operation** = send, **From** number = the test number Meta provided, **To** number = your own WhatsApp number (for testing), and a **Text Body** field.
273. Verified the raw API setup works independently by clicking "Send message" directly inside Meta's API Setup screen — instructor received the test message on his personal WhatsApp, confirming Meta's side was configured correctly.

4. Connecting the AI Agent's output to the WhatsApp send node, and the trigger's input to the AI Agent:

274. In the AI Agent's prompt source, choose **"Define below"** since there's no chat-trigger node feeding it automatically.
275. Populate the **user message** field with whatever the end user typed on WhatsApp (coming from the WhatsApp Trigger node's output).
276. Configure a **system message** describing the bot's persona/purpose (e.g., "You are a helpful assistant for a software training center; answer common student questions politely").
277. In the WhatsApp send-message node, set the **Text Body** field to the AI Agent's output text so the generated reply gets sent back to the user.

5. The blocking error encountered:

When trying to execute the workflow (either manually or by sending a real WhatsApp message to trigger it), the class repeatedly got: **"credential with ID does not exist for type WhatsApp Trigger API."** Despite double-checking every setting (account ID, client ID, client secret, "on messages" event), the trigger itself would not connect/fire. The instructor spent roughly two hours trying to debug this without full success in his own environment, and suspected it might be related to having created multiple Meta apps under the same account, or an "app not verified in your Meta app dashboard" issue.

6. What DID work for some students:

- One student ("Jai Lakshmi") successfully got the flow working after: (a) removing the Simple Memory node (which was causing a separate "no session ID found" error), (b) making sure the workflow was **published** (not just saved), and (c) sending a real WhatsApp message from her phone to the Meta-provided test number. She received AI-generated answers about Java topics (e.g., "important concepts in Java" → basics, OOP, core APIs, etc.), confirming the trigger → agent → reply loop worked end-to-end for her setup.
- A couple of small "no prompt specified" / "expression could not be resolved" errors appeared intermittently but resolved after re-checking that the **Text/Body** field (not a stale reference) was correctly selected as the prompt source.

7. Notes on the test number setup:

- The "from" number in testing is a Meta-provided **test number** (a US number, e.g., +1 5144279xx), not your own number — you must reply to that specific number from your own WhatsApp to trigger the flow.
- To send your OWN phone number a message from the bot, Meta's API Setup panel provides a **"Send message"** button that sends you an initial message from the test number, so you can then reply to it.
- A student flagged that creating **more than one or two apps** under the same Facebook/Business account seemed to cause failures; documentation apparently warns about this. Deleting an account also takes **30 days** to fully process on Meta's side, which blocked one student from quickly retrying with a fresh setup.
- The instructor noted the same automation is possible via an **Instagram**-linked path as an alternative to the Facebook-app path, with slightly different account limitations.
- This particular class recording was **not uploaded to YouTube** because personal phone numbers were visible on screen during setup.

Key takeaways

- The n8n WhatsApp integration needs two separate credential sets: (1) Client ID/Client Secret for the **trigger**, and (2) Access Token + Business Account ID for the **send-message action** — both come from the same Meta app but different screens.
- Access tokens generated from "Generate access token" in API Setup are **temporary** (expire in about an hour) — a longer-lived token requires an extra step (covered in the next session).
- A common real-world error is the trigger not connecting even when everything looks correctly configured — this can require trial, republishing, removing extra nodes (like memory), or simply retrying, and may relate to having multiple apps under one account.
- Always **publish** the workflow (not just save) before testing via a real WhatsApp message — unpublished workflows won't respond to live messages.
- When debugging, it helps to isolate: test the raw Meta API Setup send button first (confirms Meta side is fine), then test the n8n trigger in isolation, then the full chain.

Demo 19: Successfully Completing the WhatsApp Bot — Full Recap, Dynamic Setup, and Making the Access Token Permanent

This session picks up where Demo 18 left off: the WhatsApp bot is now working. The instructor demonstrates the completed bot live (a training-institute FAQ assistant), then rebuilds the entire setup from scratch on-screen as a clean, complete recap for anyone who missed earlier sessions, and adds two new topics: writing a detailed system prompt for a business bot, and converting the temporary access token into a long-lived (permanent-ish) one.

What you'll learn

- A working end-to-end demo of a WhatsApp FAQ bot for a training institute
- How to write an effective **system prompt** for a customer-facing WhatsApp bot
- The complete, clean setup steps for WhatsApp automation (a consolidated recap)
- How to convert a temporary (1-hour) access token into a **long-lived token** using Meta's Graph API Explorer and Access Token Debugger
- How the WhatsApp trigger data path (`$json.messages[0].text.body`) is used to grab the incoming user message
- Early discussion of making the recipient phone number **dynamic** instead of hardcoded

Step-by-step notes

1. Live demo of the finished bot:

The instructor messages "Hello" to the test WhatsApp number and receives: *"Hello, I'm DurgaSoft's assistant. We provide Python, Data Science, Gen AI courses. Which course are you interested in?"* He continues asking about course fees, syllabus, "core Java available?", "sessions?", and gets contextually relevant answers each time (all generated purely from prompt instructions — no document/RAG was used, no uploaded files, just a well-written system prompt containing FAQ-style business information).

- A student asks whether the phone number is fixed — for testing, yes (Meta assigns a fixed test number tied to your app's access token); in production, real customer numbers would be handled dynamically.
- A student suggests the bot could also pull live info from the DurgaSoft website instead of only from the static prompt — instructor agrees this is possible (a normal ChatGPT-style browsing/research capability could be added).

2. Full clean recap of the setup (repeated step-by-step for the whole class):

278. **Facebook account** — required, log in.
279. **Business Manager / Business Portfolio account** — create at business.facebook.com if not already done.

280. **Create a developer App** under that business portfolio (Settings → Accounts → Apps → Add, or go directly to developers.facebook.com). Give it a name, select "Others" as use case, select **Business** as app type, select your business portfolio, click Create.
281. **Add the WhatsApp product** to the app (WhatsApp → Setup).
282. From **App Settings → Basic**, copy the **App ID** (Client ID) and **App Secret** (Client Secret).
283. In n8n: create a new workflow, add **WhatsApp Business Cloud trigger**, choose **"on messages,"** set up credentials using the Client ID/Client Secret just copied.
284. Add an **AI Agent** node, attach an **OpenAI Chat Model**.
285. Add a second **WhatsApp Business Cloud** node as an action ("Send message" operation).
286. In the Meta app's **WhatsApp → API Setup** page, click **Generate access token**, opt in to "all current and future WhatsApp accounts," copy the token, and also copy the **WhatsApp Business Account ID** shown there.
287. Paste the Access Token and Business Account ID into the WhatsApp action node's credential setup in n8n.
288. Set the "From" number (the Meta test number, auto-populated based on your credentials) and the "To" number (your own number, hardcoded for testing — dynamic numbers require a paid production plan).
289. Set the **Text Body** field for the outgoing message.

3. Wiring the data flow (the part that was missing before):

290. In the AI Agent node, set the prompt source to **"Define below."**
291. For the **User Message** field, drag in the incoming text using the expression path **`{{$json.messages[0].text.body}}`** — this pulls the actual text the end user typed on WhatsApp out of the trigger's JSON payload. (The instructor notes you don't need to memorize the exact path — you can drag-and-drop the "text" field visually in the n8n UI and it fills this in for you.)
292. Configure the **System Message** field with a detailed persona and business information (see next step).
293. After executing the AI Agent, its text **output** needs to be dragged into the WhatsApp action node's **Text Body** field so the generated reply is what actually gets sent.

4. Writing a strong system prompt (the instructor asked ChatGPT to draft one, then used/adapted it):

Key elements of a good system prompt for a business WhatsApp bot, as demonstrated:

- **Role**: "You are a professional WhatsApp assistant for DurgaSoft. Your role is to guide students, job seekers, and working professionals about courses, training, and career growth."
- **Communication style**: Use simple Indian English; be friendly like a mentor, not robotic; keep answers short, clear, and useful.
- **Behavior rules**: Always respond politely; understand user intent before answering; give step-by-step guidance; don't give confusing/complex answers; if info isn't available, politely say "please contact our team for more details"; keep a positive tone; if the user asks multiple questions, answer one at a time; never use slang or disrespectful words; always represent DurgaSoft as a trusted institute.

- **Sample FAQ content baked into the prompt**: course details (Python, Data Science, Gen AI, etc.), fee info ("depends on the program and duration, please contact our team"), timing (morning/evening batches), duration (typically 30–90 days), contact info, and location.
- **Output format instruction**: "Keep answers within 2 to 4 lines."

This entire prompt is pasted into the AI Agent's **System Message** field.

5. Testing and fixing a "text body must be required" error:

The instructor showed that after wiring everything, publishing the app, and sending a real WhatsApp "hello," the bot responded correctly with the configured FAQ-style answers (course info, fees, timings, etc.) — confirming the end-to-end flow was working. He noted **memory is not required** here since each message was answered independently without needing to recall prior conversation turns.

6. Making the access token long-lived (not just 1 hour):

294. Go to your app in developers.facebook.com, and find the **"Tools"** menu (top of page).
295. Open **Graph API Explorer**.
296. Click **"Generate Access Token"**, select relevant **permissions** (e.g., publish, read insights, WhatsApp business management, etc. — select the ones relevant to your use case), then generate and copy the token.
297. Open the **Access Token Debugger** tool (also under Tools).
298. Paste the token in and click **Debug** — this shows token details including its expiry.
299. At the bottom of the debug results, click **"Extend Access Token."**
300. Submit — Meta will issue a **new long-lived access token** (in the demo, the new token was valid until **June 11, 2026** — roughly 60 days from generation).
301. Copy this new long-lived token and paste it into your n8n WhatsApp credential (replacing the old temporary one), Save.
302. **Unpublish and re-publish** the n8n app after changing credentials so the new token takes effect.
303. Re-test — the bot should continue answering correctly, now with a token that won't expire in an hour.

7. Discussion: making the recipient phone number dynamic:

- Currently the "To" (recipient) number in the send-message action is **hardcoded** — fine for single-user testing but not scalable.
- The class began exploring whether the number could instead be pulled dynamically from the incoming WhatsApp trigger payload (so replies go back to whoever messaged, not a fixed number) — several students shared their phone numbers live to test this. However, using Meta's shared **test number**, only a small number of hardcoded/pre-registered numbers can receive test messages at all — this is a Meta testing-tier limitation, not an n8n limitation. Numbers must be explicitly registered/verified as recipients in Meta's test environment before they can receive messages. Full dynamic "any number can message us" behavior requires a **paid Meta WhatsApp Business plan** with your own registered business number.

- The instructor explained the analogy: just like a bank or telecom provider only sends OTPs/data to your registered mobile number, Meta's test setup only allows communication with pre-approved test recipient numbers until you go into production with a real business number.

Key takeaways

- The full WhatsApp automation stack is: Facebook account → Business Manager account → Developer App with WhatsApp product → App ID/Secret (trigger credentials) → Access Token/Business Account ID (send-message credentials).
- The incoming message text from a WhatsApp trigger is found at `$json.messages[0].text.body` (or can simply be dragged from the UI).
- A well-structured system prompt (role, tone rules, behavior rules, sample FAQ content, output-length instruction) is enough to build a useful FAQ bot — no RAG/document upload was needed in this demo.
- Access tokens from "Generate access token" are temporary (~1 hour); use **Graph API Explorer → Access Token Debugger → Extend Access Token** to get a long-lived token (weeks/months) instead of regenerating constantly.
- After changing any credentials, **unpublish and republish** the n8n workflow so it picks up the new configuration.
- With Meta's free test number, only specifically registered test recipient numbers can receive messages — full dynamic "anyone can message us" support requires upgrading to a paid, verified business phone number.

Demo 20: Getting Real-Time Data into AI Agents with the Tavily Node (Live Score, Gold Price, and News Alerts)

This session addresses a well-known LLM limitation — models only know information up to their training cutoff and can't answer "what's happening right now" questions. The instructor introduces the **Tavily** node, a real-time web-search tool that can be plugged into n8n workflows, and builds a scheduled workflow that fetches live IPL cricket scores, gold prices, and AI news, summarizes them with an AI agent, and emails the result on a recurring timer.

What you'll learn

- Why LLMs can't answer real-time questions on their own (training data cutoff)
- What the **Tavily** node does and when to use it (real-time web search for AI agents)
- How to get a Tavily API key and connect it as an n8n credential
- Key Tavily search options: **Search Depth** (basic vs. advanced), **Include Answer**, **Start Date**, **Number of Results**
- Why the *specificity of your search query* massively affects result quality
- How to chain: Schedule Trigger → Tavily Search → AI Agent (summarize into bullet points) → Gmail (send report)
- How to use a **Structured Output Parser** to get a clean title + 5 bullet points from the AI agent

Step-by-step notes

1. The problem: LLMs have a knowledge cutoff.

A model trained with data up to (for example) April 2023 or May 2025 cannot know a live cricket score, today's gold price, or breaking news — it will either say it doesn't know, or worse, confidently give **outdated/wrong** information. For time-sensitive queries (gold prices, share market values, scores, weather, breaking news), you need a way to fetch live data from the internet and feed it to the AI agent.

2. What is the Tavily node?

- Tavily is described as a **"web search tool for AI agents"** — it performs a real web/Google-style search, retrieves up-to-date results, and returns them in a structured, AI-friendly format.
- Without Tavily: the AI answers purely from its training data (old/approximate).
- With Tavily: the AI (or the workflow) performs a live search first, then the AI can use those fresh results to compose an accurate, current answer.
- Good use cases mentioned: real-time news, stock/gold prices, weather, cricket/sports scores, restaurant/hotel/product availability, competitor analysis, market research and trend data.
- Tavily is a **third-party service** (unrelated to MCP) — it needs its own account and API key, separate from OpenAI.

3. Setting up the Tavily node and credential:

304. Add the **Tavily** node to your n8n workflow (install it if not already installed — n8n will prompt "Install node" the first time you search for it in the node panel).
305. Under its available actions (Crawl, Extract, Map, Create Research Task, Get Research Status, Search), choose **Search** — this matches the "look something up live" use case.
306. To set up credentials, click "Setup credential." If you don't already have a Tavily account, you're prompted to sign up (via Google sign-in, similar to other tools).
307. Go to Tavily's site (URL referenced in class: `app.tavily.com/home` or via "Get an API key"), sign in with Google, and generate an **API key** (give it a label, e.g., "IPL key"), then copy it.
308. Paste the API key into the n8n credential setup and Save.
309. **Important gotcha the class hit**: "Node does not have access to the credential" — this happened because the credential was created inside a different n8n project/workspace than the workflow being edited. Fix: go to **Credentials → [the credential] → Sharing/Access tab**, and set it to be available to **all workflows/projects** (or explicitly share it with the correct project), then save.

4. Configuring the Tavily search query and options:

310. In the **Query** field, type your search question (e.g., "What is the score of today's IPL match between SRH and RR?").
311. Under **Options**, key settings include:

- **Search Depth**: "Basic" (quick answer) vs. **"Advanced"** (deeper/more accurate search) — the instructor found Advanced gave noticeably better, more current results than Basic.
- **Include Answer**: turning this on makes Tavily return a synthesized quick/detailed answer, not just raw links.
- **Start Date** (and optionally End Date): restricting results to a specific date range (e.g., "today" or "yesterday") significantly improved accuracy — without it, old articles (e.g., from a prior year's IPL season) were being returned mixed in with current ones.
- **Max results / number of results**: can be limited (e.g., top 5 or 10) to keep the response fast and the AI summarization focused — the free tier allows around 1,000 API calls.

312. Execute the node to test — verify the results returned are actually current and relevant, adjusting Search Depth, Include Answer, and Start Date as needed until the data looks accurate.

5. The critical lesson on query specificity:

Tavily typically returns only its **top ~5 results**. If your query is generic (e.g., mixing "IPL score," "gold price," and "share value" together in one broad prompt), the top 5 results tend to get dominated by whichever topic is most commonly indexed (in the demo, IPL dominated even when gold price was also asked about). **Lesson: keep the search query narrow and specific to one topic at a time** for reliable, on-topic results — e.g., "gold price today in India" alone, rather than a combined multi-topic query.

6. Full workflow built in this session:

313. **Schedule Trigger** — set to run every 2 minutes (for demo/testing purposes; in real use you'd pick something like once daily).

314. **Tavily (Search)** node — configured with a specific query (e.g., "today gold price in India," "IPL 2026 match information today," or "top 5 AI related news today"), Search Depth = Advanced, Include Answer = on, Start Date = today.
315. **AI Agent** node — prompt source "Define below"; the **user message** is set to the Tavily node's `results` field (the raw search results/JSON); the **system message** instructs the agent to act as "a smart search assistant" that reads all given results carefully, identifies the main topic, ignores duplicate/low-quality info, and outputs a short, powerful summary with **5 bullet points** and a meaningful title.
316. A **Structured Output Parser** was added to the agent to force clean output fields: `title`, and five bullet points (`p1` through `p5`), so the email could be built from consistent variables.
317. **Gmail (Send a Message)** node — subject and message body built from the agent's structured output (title + 5 points), sent to a configured email address.
318. **Publish** the workflow so the Schedule Trigger actually fires every 2 minutes automatically, and confirmed via the inbox that fresh emails kept arriving with updated summaries (e.g., updated IPL score details, then later a "Today's gold price in India" report, then later a "Top 5 AI news" report — each triggered by changing the query and republishing).

7. Troubleshooting notes from the live session:

- An early error, **"Node search does not have access to the credential,"** was fixed by properly sharing the Tavily credential across the n8n project (see step 3 above).
- The class discussed why a plain ChatGPT chat answer sometimes seems to already have "live" info: ChatGPT's consumer UI may have its own built-in web-search tool enabled behind the scenes, but if you call an LLM **directly via API** (as n8n does), that model has no live web access unless you explicitly give it one — which is exactly the gap Tavily fills.
- A student asked whether multiple different queries (e.g., IPL + gold + TCS share value) could be combined in one prompt — the instructor confirmed it's technically possible to type multi-topic queries (just like typing multiple things into Google), but because only the top 5 results are used, one topic tends to dominate the returned results, so it's not very reliable — better to run focused, single-topic queries.
- The instructor also noted that other AI tools/nodes (like certain Gemini-based search/grounding options) were briefly explored as alternatives but weren't the right fit for this specific "search" operation in this session.

8. Broader real-world use cases suggested:

- Daily "newsletter"-style emails: trending AI news, top interview questions in a subject, industry updates — delivered automatically every morning.
- Competitor monitoring: e.g., automatically checking competing training institutes' websites for new batch announcements and emailing a summary to admin staff.

- Financial alerts: monitoring a stock or gold price via API/search on a timer, and triggering a WhatsApp/email alert (or even automated buy/sell logic like a stop-loss) if a price crosses a threshold.
- Could combine with earlier lessons: instead of Gmail, the final "send" step could easily be swapped for the WhatsApp send-message node built in Demos 18–19.

Key takeaways

- LLMs (including via API) do **not** have live internet access by default — their knowledge is frozen at a training cutoff date; real-time facts require an external search tool.
- The **Tavily** node adds real-time web search capability to n8n/AI agent workflows — useful for scores, prices, news, weather, and similar time-sensitive lookups.
- Getting good results from Tavily depends heavily on: (1) **Search Depth = Advanced**, (2) turning on **Include Answer**, (3) setting a **Start Date** to filter out stale articles, and (4) keeping the **query narrow and specific to one topic**.
- Because only the top ~5 results are considered, broad or multi-topic queries get dominated by whichever topic is most represented online — split unrelated questions into separate searches.
- A common, reusable automation pattern: **Schedule Trigger → Tavily Search (real-time data) → AI Agent (summarize into structured bullet points) → Gmail/WhatsApp (deliver report)** — useful for recurring reports, alerts, and newsletters.
- Remember to **share credentials properly across n8n projects** and to **unpublish/republish** after any configuration change, or the workflow may keep running on stale settings.

Demo 21: Discovering and Reusing n8n's Ready-Made Workflow Library

This session is about a huge, mostly free library of pre-built n8n automation workflows (called "templates") that anyone can browse, download, and adapt instead of building automations from scratch. The instructor walks through how to search this library by creator and by topic/domain, and how to use ChatGPT to reverse-engineer and document any workflow you download.

What you'll learn

- That n8n has a public template library with thousands of ready-to-use automation workflows built by the community.
- Two ways to find templates: by browsing top creators, or (more usefully) by searching by domain/functionality (marketing, IT operations, YouTube, sales, etc.).
- How to import a template into your own n8n workspace and see what credentials/setup it needs.
- How to use ChatGPT to get a plain-English, step-by-step explanation of any workflow (even by uploading the exported workflow JSON file).
- The business angle: these workflows can be resold/customized as paid automation services for non-technical clients.

Step-by-step notes

1. Why the template library matters

The instructor warns that people sell "10,000 ready workflows for 199 rupees" ads on Instagram — the same workflows are freely available directly from n8n's own community library, so there's no need to pay for such bundles.

2. Finding workflows by creator

319. Go to the n8n homepage (n8n.io).
320. Open the **Community** section.
321. Inside Community, open **Creators**.
322. This shows top/verified workflow creators (e.g., a contributor named Rahul Joshi with 249 templates out of over 9,000 total templates available).
323. Each creator's profile lists their published workflows with short descriptions (e.g., "Generate real-time IPL post-match analysis with GPT-4, Cricket API and Gmail").
324. Click any workflow, then choose **Use for free**, and either **import to your n8n cloud workspace** or **copy the JSON**.
325. n8n shows exactly which credentials are required to run it (e.g., OpenAI credentials, Google Sheets credentials).

3. Understanding an imported workflow

- Every good template usually includes an **overview description** in the workflow itself explaining: what it does, which nodes are involved, and how data flows between them (e.g., "fetches live cricket scores → computes match indicators like run rate/wickets/pressure → sends to GPT-4 → generates a short narrative → logs to Google Sheets → responds via webhook").
- Common nodes seen in these templates include: schedule trigger, manual test trigger, HTTP request nodes, If/Switch logic nodes, a "narrative generator" (AI agent) node, a code node, and Google Sheets nodes.
- Workflows often have **two triggers**: one schedule trigger for automatic/production runs, and one manual trigger for testing.

4. Better way to search: by domain, not by creator

326. On the n8n dashboard, click **Templates**.
327. Use the **search by apps, roles, and use case** filter (e.g., select "Marketing" — this alone showed around 2,800+ marketing workflows).
328. Narrowing to a specific niche (e.g., "YouTube") showed about 100 relevant workflows — enough to learn from by studying 2–3 of them closely.
329. This approach is recommended over searching by creator because it directly matches what you actually need (e.g., IT operations, sales, content creation).

5. Using ChatGPT to document any downloaded workflow

330. In n8n, export/download the workflow as a JSON file.
331. Upload that JSON file to ChatGPT.
332. Ask: "Analyze this workflow and provide step-by-step documentation in simple English so that I can understand it clearly and build it on my own."
333. ChatGPT will explain the big picture (trigger → processing → output), describe the purpose of each node, and even explain unfamiliar nodes like the "code node."
334. This turns any downloaded template into a personal learning/study material.

6. Business/freelance angle

- You can monetize workflows by hosting and running automations for non-technical clients — you don't need to tell them "this is built in n8n"; you simply solve their business problem (e.g., "I'll automate your WhatsApp / your business processes") and charge a fee, subscription, or percentage of the value created.
- n8n Cloud is a paid hosting service (monthly cost) if you want to keep automations running live; self-hosting on your own machine is a free alternative, to be covered in a future session.
- Downloaded templates are yours for a lifetime once downloaded — no expiry — but running/hosting them (especially on n8n Cloud) may have costs.

Key takeaways

- Don't pay for "workflow bundle" ads — n8n's own Community → Creators library and the Templates section already offer thousands of workflows for free.
- Searching by domain/use case is more efficient than browsing by creator.
- Use ChatGPT with the exported workflow JSON to get clear, beginner-friendly documentation of any workflow's purpose and node-by-node logic.
- Practicing by analyzing/rebuilding 5–10 existing templates builds strong practical n8n skill faster than starting from zero every time.
- These templates can form the basis of a freelance/consulting automation business.

Demo 22: Reviewing Core n8n Concepts Through a 15-Question MCQ Quiz

This session is a structured review of everything taught in the n8n portion of the course so far, delivered as a 15-question multiple-choice quiz (generated via ChatGPT) covering trigger nodes, set/edit nodes, if/switch logic, webhooks, HTTP requests, error workflows, and more.

What you'll learn

- Clear definitions and the purpose of the main n8n node types covered in the course.
- The difference between the old-style Cron node and the modern Schedule Trigger.
- Key terminology: trigger, execution, active mode, expressions, and workflow automation benefits.

Step-by-step notes

How this review was set up: The instructor asked ChatGPT to generate 15 MCQs on n8n, one at a time, wait for his answer, validate it, and give a short explanation plus a "learning conclusion" after each question — a good self-testing technique students can copy for their own revision.

Q1 — What is n8n mainly used for?

Answer: automating workflows between different applications (not a coding tool). n8n connects apps like Gmail, WhatsApp, Google Sheets, APIs, etc. and automates tasks without manual work.

Q2 — What is a trigger node?

Answer: a node that starts the workflow. Examples: chat trigger, scheduled trigger, form trigger, Gmail trigger (new mail), webhook trigger (API request), Cron/time trigger (e.g., every day at 9 PM). Every workflow must begin with a trigger — without one, nothing runs automatically.

Q3 — What is the role of a webhook node?

Answer: it receives data from external applications via an HTTP request — this is n8n's "entry point from the outside world." It works instantly/in real time when data arrives (unlike scheduled automation, which waits).

Q4 — What is the purpose of the Set node (also called Edit node)?

Answer: to manually define or modify data inside the workflow — e.g., convert a PDF's binary data to JSON, combine "first name" + "last name" into one "full name" variable, add a prefix like "91" to a mobile number, or clean/reshape data before sending it to the next node (API, sheet, WhatsApp, etc.). Set node and Edit node are the same feature.

Q5 — What is the purpose of the If node?

Answer: to check conditions and make decisions — it splits the workflow into two paths (true/false), e.g., "if amount > 1000, send approval message; else auto-process."

Q6 — What is the purpose of the HTTP Request node?

Answer: to send or receive data from APIs. HTTP is the common protocol two different systems use to talk to each other, and JSON is today's standard message format (replacing older XML). This node lets you connect to any external API/service even if n8n has no dedicated pre-built node for it.

Q7 — What is the purpose of the Cron node?

Answer: to run workflows at specific times or intervals (e.g., every day at 9 AM, every 30 minutes, every Monday) — useful for reports, reminders, data syncing, periodic checks. It's the older/legacy method for scheduling (same concept as Linux cron jobs), superseded by the modern Schedule Trigger node.

Q8 — What is the purpose of the Execute Workflow node?

Answer: to call and run another workflow from the current workflow — enabling reusable logic ("modular design"): build one workflow once, and call it from multiple other workflows instead of rewriting the same logic repeatedly.

Q9 — What is the purpose of the Merge node?

Answer: to combine data coming from multiple branches/nodes into a single output — e.g., one path fetches user data, another fetches order data, and Merge brings both together into one result.

Q10 — What is the purpose of an Error Workflow?

Answer: to handle failures and take action when a workflow fails, instead of silently stopping — e.g., send an alert, log the error to Google Sheets, retry, or take an alternative action. This is n8n's equivalent of "exception handling" in programming — production automations should never fail silently.

Q11 — What does "Active" mode mean for a workflow?

Answer: an active workflow runs automatically whenever its trigger condition is met (e.g., new Gmail arrives, scheduled time hits). An inactive workflow must be run manually.

Q12 — What does "execution" mean in n8n?

Answer: one run instance of the workflow. Every time a trigger fires, a new execution is created (e.g., 10 emails received = 10 executions). All executions can be reviewed for debugging.

Q13 — What is the purpose of the Wait node?

Answer: to pause the workflow for a specific time or until an event happens — useful for delays before publishing (e.g., wait 5 seconds after media is prepared before posting), avoiding overwriting Google Sheets data written too quickly, follow-ups, reminders, or approval waits.

Q14 — What are "expressions" in n8n?

Answer: the mechanism to dynamically access and manipulate data inside nodes instead of hardcoding fixed values, e.g., `{{$json.name}}`. This lets you pull real, dynamic data provided by the end user/previous nodes rather than fixed text.

Q15 — What is the main benefit of workflow automation tools like n8n?

Answer: reducing manual work and saving time by automating repetitive tasks — leading to increased business productivity.

Bonus discussion — Schedule Trigger vs Cron node: Schedule Trigger is the modern, recommended, easy-to-use method (simple UI, no syntax to remember, options like "every day," "every 10 minutes"). The Cron node is the older method requiring you to manually configure cron-style expressions (minute, hour, day, month values) — more flexible (anything expressible in cron syntax) but harder to use. Use Schedule Trigger unless you need a schedule pattern it doesn't support.

Key takeaways

- n8n is a no-code/low-code platform for automating workflows between apps — not a programming language replacement.
- Every workflow needs a trigger node to start it; webhook triggers are for real-time, event-driven starts from external systems.
- Set/Edit node = reshape and clean data; If node = 2-way branching; Switch node (mentioned as still to be covered) = for 3+ way branching.
- HTTP Request node is the "universal connector" for any API that doesn't have a dedicated n8n node.
- Error workflows are essential for production reliability — never let automations fail silently.
- Schedule Trigger (modern) is generally preferred over the legacy Cron node unless advanced scheduling flexibility is needed.

Demo 23: Life Purpose (Ikigai) and Building a Multi-Category Feedback Workflow with AI Classification (Part 1)

The first part of this session is a personal-development discussion on finding your "Ikigai" (life purpose) using four guiding questions. The second half begins a hands-on n8n build: a feedback form that uses an AI agent to automatically classify each submission (complaint, compliment, or new feature request) and route it to different Google Sheets using a Switch node.

What you'll learn

- The Ikigai framework: four questions to find work/activities that bring both fulfillment and income.
- How to design a form-based n8n workflow where an AI agent classifies free-text input into categories.
- Why a Switch node (not an If node) is needed when there are 3+ possible outcomes.
- How to write a clear system prompt so an LLM reliably returns a single classification label.
- How to use an Output Parser to force the AI agent's response into a specific variable/field format.

Step-by-step notes

1. The Ikigai concept (life-skills segment)

Ikigai is a Japanese concept meaning "life purpose." To find it, answer four questions and look for the common answer across all of them:

335. What do you love (enjoy doing)?
336. What are you good at?
337. What does the world need?
338. What can you be paid for?

The instructor used himself as an example: his answer to all four questions was "teaching," which he identified as his Ikigai. The discussion also touched on happiness/self-control themes (using the Gautam Buddha "bowl" story to illustrate: you control your own reaction, not other people's behavior).

2. Building the "feedback classification" workflow — the requirement

Goal: collect student feedback through a form, use an AI agent to classify it into one of three categories — **complaint**, **compliment**, or **new feature request** — and save each category into a separate Google Sheet (three sheets inside one Google Sheets document), rather than mixing all responses into a single sheet.

3. Step 1 — Build the feedback form (Form trigger node)

339. Add an **n8n Form Trigger** node ("On new form event"). Authentication: not required.
340. Set form title, e.g., "Feedback form."
341. Add form fields:

- Name — text input, required.
- Email — email type, required.
- Mobile Number — number type, not required.
- Feedback — text area, required.

342. Save and test-execute; filling and submitting the form confirms the trigger works (note: testing does not require the workflow to be published/live — it just needs a test execution).

4. Step 2 — Add an AI Agent node to classify the feedback

343. Add an **AI Agent** node, and attach a **Chat Model** node (e.g., OpenAI Chat Model) to it.
344. Since the input isn't coming from a chat trigger, set the agent's prompt source to **"Define below."**
345. Set the **User Message** to the feedback field from the form (the text the agent should analyze).
346. Write a **System Message** instructing the agent on its role. Example system prompt used:

"You are a feedback classification assistant. Your task is to analyze the given customer feedback and classify it into only one of the following categories: complaint, compliment, and new feature request. Carefully understand the intent of the feedback. If the feedback expresses dissatisfaction, problems, or issues, classify as complaint. If the feedback expresses appreciation, satisfaction, or positive experience, classify as compliment. If the feedback suggests improvements, new ideas, or additional features, classify as new feature request. Always choose the most relevant category. If multiple intents are present, choose the primary intent. Output only one word. Do not provide explanation. Do not add any extra text. Do not use punctuation. Final output must be exactly one of: complaint / compliment / new feature request."

347. Test-execute the node; the agent should return a single classification word (e.g., "complaint").

5. Step 3 — Force a structured output using Output Parser

348. Enable **"Require Specific Output Format"** on the AI Agent node.
349. Choose the **Structured Output Parser**.
350. Define one variable, e.g., `category`, so the LLM's answer is accessible as `{{$json.output.category}}` in later nodes — this guarantees a clean, predictable variable instead of relying on the agent "behaving" correctly every time.

6. Step 4 — Route with a Switch node (not an If node)

- Because there are **three** possible outcomes (complaint / compliment / new feature request), an If node (which only supports two branches: true/false) is not sufficient — a **Switch node** is required for 3+ branches.
- In the Switch node, define one routing rule per category, comparing the AI agent's output value (e.g., `{{$json.output.category}}`) to each expected string: `complaint`, `compliment`, `new feature request`.

- Best practice: paste (don't retype) the comparison values to avoid typos, and don't wrap them in quotation marks in the field.
- (This session ends mid-configuration of the Switch node; the rest — connecting each Switch branch to its own Google Sheet — is completed and demonstrated fully in Demo 24.)

Key takeaways

- Ikigai = the overlap of what you love, what you're good at, what the world needs, and what you can be paid for; identifying it is said to improve happiness and even lifespan.
- Use a Switch node whenever a value can fall into three or more categories; use an If node only for simple two-way (yes/no) decisions.
- AI agents are what make "soft" classification tasks (is this a complaint or a compliment?) possible without writing complex keyword-matching code — this is the core value of adding an LLM into an automation.
- Always pair a classification AI agent with an Output Parser so the result comes back in a clean, reliable variable rather than free-form text.
- One Google Sheets document can contain multiple sheets/tabs — you don't need separate documents for each category.

Demo 24: Completing the Feedback Classification Workflow — Switch Routing, Multi-Sheet Logging, Email Replies, Slack Notifications, and Parallel Execution

This session finishes the feedback-classification automation from Demo 23: it wires up the Switch node's three branches to write to three different Google Sheets tabs, adds an automatic email reply, adds Slack channel notifications, and demonstrates running independent branches in parallel.

What you'll learn

- How to fully configure a Switch node's routing rules and rename its default output labels for clarity.
- How to log AI-classified form data into different tabs of the same Google Sheet using manual column mapping.
- How to duplicate/reuse nodes efficiently to build out multiple branches.
- How to send an automatic Gmail reply based on which category was detected.
- How to create a Slack workspace/channels and post messages to them from n8n.
- How multiple independent branches in n8n can execute in parallel rather than strictly one-after-another.

Step-by-step notes

1. Rebuilding context

The workflow so far: Form trigger → AI Agent (with system prompt classifying feedback into complaint/compliment/new feature request, using an Output Parser to get a clean `category` variable) → Switch node. This session focuses on what happens after the Switch node.

2. Why an LLM is needed here (recap)

The instructor explicitly contrasts this with traditional code: without an LLM, you'd need to hardcode long lists of keywords ("worst," "best," etc.) with many conditional statements to guess whether feedback is a complaint or compliment — impractical and unreliable. An LLM, however, understands natural language the way a human would and can classify it directly, which is the real value of adding AI to automations.

3. Finishing the Switch node

351. In the Switch node, define three routing rules, each comparing the AI agent's `category` output to one of: `complaint`, `compliment`, `new feature request`.
352. By default, Switch outputs are just labeled 0, 1, 2 — confusing to work with later. Use the **"Rename Output"** option on each branch to relabel them clearly as "Complaint," "Compliment," and "New Feature Request."
353. Test by submitting different feedback text and confirming the correct branch lights up (e.g., a complaint-sounding message activates only the "Complaint" path).

4. Logging to Google Sheets (per branch)

354. Add a **Google Sheets** node using the **Append Row in Sheet** operation on the Complaint branch.
355. Sign in with Google credentials (already set up from earlier sessions).
356. Select the spreadsheet document (e.g., "Total Feedback Document") and choose the **Complaint** sheet/tab (the same document has separate tabs: Complaint, Compliment, New Feature Request).
357. **Important lesson on mapping**: n8n offers "map automatically" vs "map each column manually."

- Auto-mapping only works cleanly when the incoming fields line up 1:1 with sheet columns — but since the AI agent adds an extra `category` field that isn't wanted in the sheet, auto-mapping pulls in the wrong/extra field.
- **Manual mapping is recommended**: explicitly map Name, Email, Phone Number, and Feedback columns to their respective form fields, and skip the `category` field. This gives full control over exactly what gets written.

358. **Duplicate** this Google Sheets node twice (right-click → duplicate, or copy-paste) and connect one to the Compliment branch and one to the New Feature Request branch — just changing which sheet/tab each one targets, keeping the same column mappings.
359. Test end-to-end: submitting different types of feedback correctly appends rows to the matching sheet tab only.

5. Sending an automatic email reply (Gmail node)

360. Add a **Gmail** node with the **Send Message** operation on a branch (e.g., Complaint).
361. Sign in with Google.
362. Set **To** = the email address captured from the form.
363. Set **Subject** (e.g., "Sorry for the inconvenience") and **Text/body**, e.g.: "Hello, we noticed your complaint and sincerely apologize for the inconvenience caused. Our team is working on it and will resolve the issue and update you as soon as possible. Thank you."
364. Test-execute — the instructor confirms actually receiving the reply email.

6. Adding Slack notifications

365. Explain Slack's purpose: a team communication/collaboration tool — channels for organizing conversations, file sharing, integrations, and real-time/async messaging (compared to Microsoft Teams).
366. Set up Slack: create a Slack account/workspace (e.g., named "Agentic AI Feedback Group"), then create three channels: **complaint**, **compliment**, **new-feature-request** (made public in this demo).
367. In n8n, add a **Slack** node with the **Send Message** operation.
368. Configure Slack credentials (OAuth sign-in). Note: the instructor hit an issue where an old/wrong Slack workspace credential was selected — fixed by creating a new credential connection pointing to the correct workspace.
369. Choose the target channel (e.g., "complaints") and set the message text to include the feedback content.

370. Duplicate this node for the Compliment and New Feature Request branches, updating the channel selection each time.
371. Test: confirms messages appear correctly in the matching Slack channel.

7. Running independent actions in parallel

- A student asked whether Google Sheets logging, the Gmail reply, and the Slack message (for a given category) could run at the same time instead of strictly one after another, since none of them depends on the others' output.
- Demonstrated fix: connect the Switch node's output directly to **all three** action nodes (Google Sheets, Gmail, Slack) for that branch, instead of chaining them serially one into the next.
- Result: all three actions fire simultaneously when a matching category is detected — confirmed by checking that the email, Google Sheet row, and Slack message all appeared right away.
- n8n normally executes node-by-node in sequence, but nodes that don't depend on each other's data can be connected in parallel from the same source node to run concurrently.

8. Handling multi-intent feedback

- When feedback contains mixed signals (e.g., both a compliment and a feature request, or worded ambiguously/sarcastically), the system prompt's instruction ("if multiple intents are present, choose the primary intent") determines the outcome — but there's no strict guarantee of which one the LLM will pick; it depends on the AI's own judgment/training, not a fixed rule.
- To support more categories (e.g., adding a 4th category), you must update both the system prompt (to describe the new category) and the Output Parser definition, and add a corresponding new Switch rule.

Key takeaways

- Always manually map fields into Google Sheets when the incoming data has extra AI-generated fields you don't want written to the sheet — auto-mapping can pull in unwanted columns.
- Renaming Switch node outputs (instead of leaving 0/1/2) makes complex workflows far easier to read and maintain.
- Duplicating an already-configured node (Google Sheets, Gmail, Slack) and just changing its target (sheet/channel) is much faster than rebuilding from scratch.
- Nodes with no dependency on each other can be connected in parallel from the same source to execute simultaneously, improving speed compared to strictly sequential chains.
- LLM-based classification has no hard guarantee on ambiguous/multi-intent input — behavior depends on the model's own reasoning, not deterministic rules, unless the prompt is very tightly constrained.

Demo 25: Understanding Webhooks — Connecting External Applications to n8n

This session explains the webhook concept in depth and builds a live example: an external chat application (created with an AI app-builder tool) is connected to an n8n automation purely through a webhook URL, so that messages typed in the external app get answered by an AI agent running inside n8n.

What you'll learn

- What a webhook is and why it's needed to connect apps that don't have a pre-built n8n node.
- The difference between a Webhook (trigger) node and a Respond to Webhook (action/response) node.
- The difference between a workflow's Test URL and Production URL.
- How to configure a Webhook node's HTTP method (GET vs POST) and response mode.
- How to build and connect an external chat app to an n8n-powered backend, end to end.

Step-by-step notes

1. What is a webhook, conceptually?

A webhook is **a URL that receives data and automatically triggers a workflow in real time.** It is the mechanism used to connect n8n to any external application that doesn't already have a dedicated pre-built n8n node (e.g., Gmail, Google Sheets, WhatsApp, and Telegram all already have ready-made n8n nodes, so no webhook is needed for those — but a custom website like "durgasoftonline.com" has no pre-built node, so a webhook must be used to connect it to n8n).

Examples given:

- A contact form on an external website submits data → webhook receives it → workflow saves it to Google Sheets and sends an email reply.
- A customer makes a payment on an external system → webhook receives payment details → workflow sends a confirmation message.
- A user sends a WhatsApp-style message on a custom chat app → webhook captures it → bot replies automatically.

2. The two key nodes

- **Webhook node** (trigger node): starts the workflow whenever its URL is called by an external application; this is the "entry point"/input side. It receives incoming data (the request) — usually the first node in this type of workflow.
- **Respond to Webhook node** (action/response node): sends data back to whoever called the webhook — the "exit point"/output side. Usually placed at (or near) the end of the workflow, after all processing is done, to return the final response to the caller.

3. Building a live example — a custom AI chat app talking to n8n

Step-by-step build:

372. Create a new n8n workflow and add a **Webhook** node as the trigger.
373. In the Webhook node settings, set a custom **Path** (instead of the default randomly generated one) so the URL is memorable (e.g., ending in something like `/webhook/dscat`).
374. Set the **HTTP Method** to **POST** (not GET) — because the workflow needs to receive/post data from the caller, not just fetch something.
375. Set **Respond** to **"Using Respond to Webhook Node"** (instead of "Respond Immediately"), so the response is controlled by a separate node later in the flow.
376. Add an **AI Agent** node connected after the Webhook, with a Chat Model attached (e.g., OpenAI Chat Model). Set its prompt source to "Define below" and map the **User Message** to the incoming webhook's message body field (e.g., `{{$json.body.message}}` or similar — referred to in the demo as "message body").
377. Add a **Respond to Webhook** node at the end — no special configuration needed; it automatically sends back whatever the AI Agent produced to the original caller.
378. Test using the workflow's **Test URL**: while the workflow is in "listen for test call" mode, sending a request to the test URL (e.g., by typing a message in an external app pointed at that URL) triggers the workflow and returns a live AI-generated response.
379. To make it permanently usable (not just for one test call at a time), click **Publish** to activate the workflow, then use the **Production URL** instead of the Test URL — the production URL works continuously without needing to manually re-arm "test" mode each time.

4. Building the external chat app itself

- The instructor used an AI app-building tool (referred to as "Lovable" and also demonstrated using **Gemini's Canvas** feature) to quickly generate a simple chat web app, instructing it to "build a simple chat application that calls this webhook URL."
- The generated chat app sends whatever the user types to the n8n Production Webhook URL and displays the AI Agent's response — demonstrating that the actual "brain" answering questions lives entirely in the n8n workflow (backed by OpenAI), while the chat app itself is just a lightweight front end.
- Live tests worked: asking "Who is the PM of India?" and "What is the capital of India?" through the external chat app correctly returned answers generated by the n8n-connected OpenAI model — proving the two separate platforms (the external chat app and n8n) were successfully communicating via the webhook.
- A **CORS (Cross-Origin Resource Sharing)** issue came up when deploying the external app to call the webhook from a browser context; students suggested enabling "Allow Origins" / CORS settings and republishing to resolve cross-domain call blocking.

5. Clarifying questions covered

- Webhooks require **no coding** — this is presented as a major advantage of using n8n's Webhook node versus manually writing an HTTP client (e.g., Java's RestTemplate) to call a REST endpoint yourself.
- A webhook call is fundamentally the same idea as calling any REST API endpoint — the class discussed it as functionally equivalent to hitting a REST endpoint URL, just packaged as a no-code n8n feature.
- By default, the demo's webhook had **no authentication** configured (open endpoint) — the instructor noted that for production/enterprise use, you would add authentication (API keys/OAuth tokens) for security; this demo intentionally kept it simple/unsecured for learning purposes.
- A webhook only fires when a specific **event** occurs (e.g., a chat message is sent) — simply visiting/opening a website does **not** trigger the webhook by itself; some user action (like submitting a message) must occur.
- n8n can be used both ways: **external apps calling into n8n** (via a Webhook node, as demonstrated) and, conversely, **n8n calling out to external systems** (also via webhook-style HTTP calls) — the underlying concept is the same in both directions.

Key takeaways

- A webhook is the bridge that lets any external application (with no dedicated n8n node) connect to and trigger an n8n workflow, and it requires no coding to set up.
- Webhook node = entry point/trigger (receives data, starts the workflow); Respond to Webhook node = exit point (sends the final response back to the caller).
- Use the workflow's Test URL only while actively testing; switch to the Production URL (after publishing the workflow) for any real, ongoing external integration.
- Set the Webhook node's HTTP method to POST when the caller needs to send data in and receive a processed response back, and set its "Respond" option to "Using Respond to Webhook Node" for full control over the returned response.
- This webhook pattern is exactly how you would connect any custom website, mobile app, or third-party system's chat/form/payment events to an n8n-powered backend automation.

Demo 26: Understanding Webhooks and Connecting a Local HTML App to n8n

This session explains the webhook concept in depth and walks through building a simple standalone HTML chat page (with no backend code) that connects to an n8n workflow via a webhook, then deploys that page live using Netlify.

What you'll learn

- What a webhook is and why it is needed to connect apps that don't have a built-in n8n node
- How to build a webhook-triggered workflow in n8n (Webhook node + Respond to Webhook node)
- The difference between the n8n test URL and production URL
- How to build a simple chat front end using only HTML/CSS/JavaScript (generated with Gemini Canvas)
- How to connect that HTML page to an n8n webhook URL
- How to publish a static HTML page live using Netlify

Step-by-step notes

1. What is a webhook?

A webhook in n8n is a URL that listens for incoming data and automatically triggers a workflow in real time. Whenever any external system (a form, an app, a payment provider, etc.) sends a request to that URL, the webhook "wakes up," receives the data, and starts running the workflow. This is the entry point of the workflow. Examples given:

- A contact form submits data → webhook receives it → saves to Google Sheets → sends an email reply.
- A payment happens → webhook receives payment details → sends a confirmation message.
- Any external event → webhook logs the event → triggers automation.

Key idea: if you already have your own app (built in any language/framework, or plain HTML) and it does NOT have a ready-made n8n node, you use a webhook to connect it to an n8n workflow. The webhook is like an API endpoint — your app acts as the "client," n8n acts as the "server."

2. Why use a webhook for a local HTML chat app

The instructor's scenario: he wants a simple chat page saved on his local computer (pure HTML/CSS/JavaScript, no Node.js/React needed, works from just a browser). This local page cannot answer questions like "What's the time in India?" because it has no AI/LLM ability on its own. To give it that ability, he connects the HTML page to an n8n workflow using a webhook URL. When the user submits a message, the webhook URL is triggered, n8n runs an AI Agent + chat model, and the response is sent back through a "Respond to Webhook" node.

3. Building the n8n webhook workflow

380. Create a new workflow in n8n.
381. Add a **Webhook** node as the trigger. Configure:

- HTTP Method: **POST** (because data is sent in the request body, not as part of the URL — POST is used whenever the client is sending data).
- Path: instead of the random auto-generated path, give it a short memorable name (e.g., `chat`) by editing the path variable — this also updates the Test/Production URL automatically.
- "Allowed Origins" (CORS) setting: optional, but useful if your app and your n8n instance are on different platforms/domains (cross-platform communication) and you hit security/CORS issues — enable it if needed.

382. Add a **Respond to Webhook** node as the endpoint that sends the reply back to the caller (this is the "exit point" of the flow, just like the Webhook node is the "entry point").
383. Add an **AI Agent** node (with an OpenAI-based chat model) between the Webhook and Respond to Webhook nodes so incoming messages get an intelligent reply.
384. Test using **Execute Workflow** and the **Test URL** first; once everything works, click **Publish** to activate the workflow and get a live **Production URL**.

Important distinction: the Test URL only listens when you click "Execute Workflow" and wait for one event; the Production URL works continuously once the workflow is published/active — you do not need to manually execute the workflow each time.

4. Building the local HTML chat page

385. Open Gemini (in Canvas mode, same feature used in earlier sessions).
386. Prompt Gemini with something like: "I want to create one HTML based chart app with the n8n webhook URL [paste your webhook URL]. Use only HTML, CSS, and JavaScript. The app should contain only one HTML page." (No Node.js/React — the goal is a page that runs from just a browser with nothing else installed.)
387. Gemini generates a single HTML file (in this demo, 251 lines) containing HTML structure, inline CSS, and inline JavaScript all combined — the webhook URL is embedded in the JavaScript code that sends requests.
388. Copy the code and save it locally as an `.html` file (e.g., on the Desktop). When saving, choose "All files" as the save type and manually type the `.html` extension, and pick a suitable text encoding (Unicode/UTF-8) to avoid character/encoding glitches that broke the page in the demo.
389. Open the saved HTML file directly in a browser (Chrome/Edge) — no server needed.

5. Connecting and testing

- If the webhook URL in the HTML file doesn't match the currently active Test/Production URL in n8n, you'll get an error like "Could not connect to the webhook. Please check if the n8n workflow is active and configured correctly."

- Fix: make sure the URL in the HTML file exactly matches either the Test URL (while manually executing the workflow each time before testing) or the Production URL (once the workflow is published/active — no manual execution needed).
- Once matched correctly, typing a message like "hello" in the HTML page triggers the webhook, the AI Agent generates a reply, and the Respond to Webhook node sends it back — displayed in the browser.
- Note: without adding a Memory component to the chat model, there is no conversation memory/session handling — each request is a standalone question and answer (e.g., asking "What is his age?" right after asking about a person fails because the model doesn't remember who "he" is).

6. Making the app live with Netlify

390. Go to netlify.com and log in with Gmail.
391. Click "Add a new project."
392. Drag and drop (or browse and upload) the saved HTML file from the desktop.
393. Netlify requires the homepage file to be named **index.html** (or index.jsp, etc.) — this is the file every website looks for by default so you don't have to include the filename in the URL. Rename the uploaded file to `index.html` when prompted ("Rename and deploy").
394. Netlify deploys the app and provides a live public URL. Anyone with that URL can now open the HTML chat page from anywhere (not just the local machine) and it still talks to the same n8n Production webhook URL.
395. Custom domains can also be mapped to the Netlify-hosted app via DNS configuration if desired.

7. Q&A points covered

- The webhook doesn't maintain sessions — each request/response is independent unless you add memory.
- Any front-end technology (HTML, React, Angular, etc.) can call the same webhook URL — n8n doesn't care what technology sent the request, only that it hits the right URL with the right method.
- The response returned by "Respond to Webhook" is a plain HTTP response (can be JSON), and any client-side code (JavaScript fetch, Angular HTTP client, etc.) can capture and use it however needed.

Key takeaways

- A webhook is simply a URL that listens for and receives data, then automatically triggers an n8n workflow.
- **Webhook node** = entry point of a workflow; **Respond to Webhook node** = exit point that sends the reply.
- POST is used because data is sent in the request body; the path can be customized for a cleaner, memorable URL.
- You don't need any backend coding skill to connect your own front-end app to powerful backend automation — n8n handles the "backend" with zero code, as long as you wire it up with a webhook.

- Test URL vs Production URL matter: test requires manually executing the workflow each time; production works automatically once published/active.
- A static HTML page with no server can be made publicly live for free using Netlify, as long as the main file is named `index.html`.
- Without adding memory to the AI Agent/chat model, there is no conversational context between messages.

Demo 27: Building a Student Feedback Form Connected to n8n (Webhook + AI Agent + Gmail + Google Sheets)

This session builds a complete, practical example: an HTML feedback form (name, email, mobile, feedback) that, through a webhook-triggered n8n workflow, analyzes the feedback with AI, replies with a personalized email, and logs everything to Google Sheets — all built from scratch and then deployed live via Netlify.

What you'll learn

- How to design an n8n workflow that combines Webhook, Respond to Webhook, AI Agent, Gmail, and Google Sheets nodes
- How to write a system prompt that makes an AI Agent classify feedback (compliment, complaint, or feature request) and generate an appropriate personalized reply
- How to use a Structured Output Parser to get separate "subject" and "body" fields from the AI Agent
- How to map AI Agent output fields into Gmail and Google Sheets nodes
- How to deploy the HTML form live with Netlify and confirm the full pipeline works end-to-end

Step-by-step notes

1. The goal / requirement

Build a "student feedback form" as a single local HTML page. When a student submits it, the back end (in n8n) should:

396. Save the feedback (name, email, mobile, feedback text) into Google Sheets.
397. Send a personalized reply email addressing the specific feedback (thanking for praise, apologizing for complaints, or acknowledging feature suggestions).
398. Show a "Thank you" confirmation message on the form itself.

The only thing that should live in the browser/local machine is the plain HTML form — all logic lives in n8n, connected via a webhook.

2. Building the workflow from scratch

399. Create a new workflow, e.g., "Web Hook Feedback Form."
400. Add a **Webhook** node:

- Method: **POST**.
- Path: something descriptive, e.g., `feedback-form` (a random default path is not recommended).
- Respond option: set to use a **Respond to Webhook** node (not immediate/default response).

401. Add a **Respond to Webhook** node and connect it right after the Webhook node.
402. Add an **AI Agent** node (with an OpenAI chat model) connected from the Webhook node, used to analyze the feedback and draft a response.
403. Add a **Gmail** node (Send Message) to email the student back.

404. Add a **Google Sheets** node (Append Row) to log the feedback.

(All nodes can branch off the Webhook node in parallel, or be chained — either design works.)

3. Designing the HTML feedback form

405. In Gemini (Canvas), request: "I want a feedback form from my students with fields name, email, mobile number, and feedback in pure HTML, CSS, and JavaScript. Connect [webhook URL]." Paste the current n8n webhook Test URL.
406. Gemini generates a single-page HTML form (169 lines in the demo).
407. Save the code locally as an HTML file (e.g., "student feedback form.html") on the desktop, saved as "All files" type with the `.html` extension typed manually.
408. Open the file directly in a browser to confirm the form displays and is running purely from the local machine.

4. Configuring the AI Agent to write personalized replies

- In the AI Agent's **User Message/Prompt**, include the submitted `name` and `feedback` fields from the webhook body data.
- In the **System Message**, instruct the agent clearly, for example:

"You are responsible for analyzing customer feedback and generating an appropriate response. First identify the type of feedback: complaint, compliment, or new feature request. If it is a compliment, thank the user and say you will continue to maintain the best services. If it is a complaint, apologize politely and say the issue will be resolved. If it is a new feature request, thank them for the suggestion and say it will be considered. Always end with 'Thanks and Regards.'"

- Test by executing just this node: for a positive feedback ("Excellent sessions, covered every concept in detail"), the agent generated a warm thank-you reply, correctly using the student's name (pulled automatically from the user message context — no need to explicitly reference "name" separately, since it's already part of the prompt).

5. Getting structured output (subject + body) from the AI Agent

- To get a clean email subject line separate from the email body (instead of one blob of text), attach a **Structured Output Parser** to the AI Agent.
- Define two output fields: `mail subject` and `mail body` (the instructor also added `appointment`/feedback-related fields as needed — in this demo just subject and body were used, described with a short example of the expected content for each).
- Re-run the node: output now cleanly separates into a subject like "Response to Ravi's Feedback" and a full body text addressed to the student.

6. Wiring Gmail and Google Sheets

- **Gmail node**:
- "Send To" = the email address from the webhook's incoming form data.

- Subject = the AI Agent's structured `mail subject` output.
- Message/body = the AI Agent's structured `mail body` output (set to plain text, "append n8n attribution" option removed).
- **Google Sheets node**:
- Operation: **Append Row**.
- Select the target spreadsheet and sheet (a "feedback" sheet with columns: name, email, mobile number, feedback).
- **Map each column manually** (recommended over "automatic" mapping) — manual mapping avoids accidentally pulling in unwanted extra fields (like source, webhook URL, execution mode) that "automatic" mode would otherwise add.

7. Testing the full flow

- Submitting a positive feedback ("Excellent sessions") produced: a "Thank you" message on the form, a properly personalized thank-you email, and a new row in Google Sheets.
- Submitting a negative feedback ("late night classes disturbing my sleep") produced an apologetic email acknowledging the specific complaint, and it was logged to Sheets too.
- Submitting a feature-request-style feedback ("please add graphs") produced a reply thanking the student for the suggestion and saying the feature would be considered — confirming the AI Agent correctly classifies and responds differently based on feedback type, driven entirely by the system prompt.
- Nodes executed one after another (not fully in parallel) even though they were laid out as parallel branches — actual execution order can vary slightly.

8. Publishing to production and deploying live

409. Click **Publish** in n8n to activate the workflow and get the **Production URL**.
410. Replace the Test URL in the HTML file with the Production URL (using find-and-replace for every occurrence).
411. No need to click "Execute Workflow" anymore once published — the production webhook listens continuously.
412. Upload the HTML file to **Netlify** (drag and drop or browse), rename it to `index.html` when prompted, and deploy — this produces a public live URL others can access from anywhere.
413. Multiple people submitted the live form during the session; entries appeared in Google Sheets and reply emails were sent (with some delay attributed to free/standard Gmail sending limits rather than the workflow itself).

9. Extra Q&A covered

- GET vs POST: by default, a webhook can be configured for GET or POST depending on how the client sends data. With GET, data travels as part of the URL (limited to a few hundred characters, less secure, cannot carry files/binary data); POST puts data in the request body (more secure, no practical size limit) — POST is the default and recommended choice for sending form data.

- Delayed email replies were traced to limitations of the free Gmail sending service, not to the n8n workflow logic; using a paid/dedicated email/SMS service would give faster, prioritized delivery.
- Workflows can be exported/imported as JSON via n8n's three-dot menu (Download / Import from file).
- n8n workflows could in principle also be self-hosted (e.g., on a service like Hostinger) at a lower cost than the n8n cloud plan — the instructor mentioned this as a topic for a future session.

Key takeaways

- A single n8n workflow can combine a webhook trigger, an AI Agent, Gmail, and Google Sheets to build a real, useful automation with zero traditional backend coding.
- Structured Output Parsers let you extract multiple clean fields (like subject and body) from one AI Agent response instead of one long blob of text.
- Manually mapping Google Sheets columns avoids unwanted extra fields being written automatically.
- System prompts are the main "programming" mechanism for AI Agents — how you phrase the instructions directly controls the tone and structure of the generated response.
- POST is the standard, secure choice for sending form data through a webhook; GET is limited by URL length and less secure, useful mainly for simple data retrieval requests.
- Publishing (making a workflow "live"/production) removes the need to manually execute the workflow before each test.

Demo 28: Building a Hospital Appointment Booking Automation with n8n Forms

This session builds a basic hospital appointment booking system using an n8n Form trigger (instead of an external HTML page), which automatically assigns an appointment date/time, sends a confirmation email, and logs the appointment to Google Sheets. It also celebrates the milestone of 100 consecutive daily sessions.

What you'll learn

- How to build a form directly inside n8n using the **n8n Form Trigger** node (rather than an external HTML page + webhook)
- How to design form fields including dropdowns, and mark fields as required
- How to use an AI Agent with a Structured Output Parser to generate a confirmation email with a computed appointment date/time
- How to use n8n's JSON/expression syntax to calculate dates dynamically (e.g., "3 days from today")
- How to save appointment records to Google Sheets and send a confirmation email
- Ideas for extending the system (SMS instead of email, doctor-specific scheduling, available time slots, mobile number validation)

Step-by-step notes

1. The requirement

Build a basic hospital appointment booking workflow: a patient fills out a form (name, email, mobile number, and which medical service/department they need — e.g., dentist, cardiology, general physician). On submission, the system should automatically:

414. Assign an appointment date 3 days after the request date, with a random time between 9 AM and 6 PM.
415. Save the appointment details to Google Sheets (for hospital staff to check).
416. Send the patient a confirmation email with the appointment details and general instructions (e.g., arrive early, bring ID).

2. Creating the form with n8n's Form Trigger

417. Create a new workflow, e.g., "Hospital Appointment Workflow."
418. Add an **n8n Form Trigger** node (not a webhook this time — the form is built directly inside n8n).
419. Set Form Title: "Appointment Request Form." Authentication and form description can be left blank/not required.
420. Add form elements one by one:

- **Full Name** — Text input, marked **Required**.
- **Email** — Email input type, marked **Required**.

- **Mobile Number** — Number input type, marked **Required**.
- **Service** — Element type **Dropdown**, with field options: Dentist, Cardiology, General Physician (more can be added). Marked **Required**.

421. Test by clicking "Execute Workflow" and filling in the generated form — confirms the form submits without errors.

3. Configuring the AI Agent to generate the confirmation content

422. Add an **AI Agent** node (with an OpenAI chat model) after the form trigger.
423. In the prompt/user message, include the submitted **name** and **service**.
424. Write a **system message** instructing the agent, roughly:

"You are an appointment confirmation email assistant for the hospital. Your job is to generate a clear, polite, professional email reply for the patient. You will receive: patient name, service name, appointment date, and hospital rules. Write an email confirming the appointment request has been received, clearly including patient name, service name, appointment date, and a section called 'Important Instructions for Appointment Day.' Keep the tone polite and use simple English. Greet the patient by name."

425. To compute the appointment date/time dynamically, two approaches were discussed:

- **Simplest (recommended in this demo):** just describe the rule directly in the system prompt, e.g., "Appointment date should be exactly 3 days from date of request, and choose a random time from 9 AM to 6 PM."
- **Alternative (expression-based):** use n8n's JSON expression syntax to compute dates, e.g. `$now.plus(3, 'days').toFormat('dd/MM/yyyy')` (the instructor demonstrated the general idea using `$now` for "today's date" plus a function to add days and a format conversion, without needing to hardcode dates).

426. Running the AI Agent node generated realistic output, e.g.: "Dear Durga, thank you for contacting the hospital... Appointment details: Patient name: Durga, Service: General Physician, Appointment date: April 24th, Appointment time: 2:30 PM. Important instructions: please arrive 15 minutes early, bring a valid photo ID..."

4. Adding a Structured Output Parser

- To split the AI Agent's output into distinct fields, attach an **Output Parser (Structured Output Parser)**.
- Define fields: `mail subject`, `mail body`, `appointment date`, `appointment time` (the last two, though also embedded inside the mail body text, are pulled out separately specifically so they can be written as their own columns in Google Sheets).
- Re-executing the node confirmed clean separate values, e.g. subject "Appointment Confirmation – General Physician," a full mail body, and separate `appointment date` (2026-04-24) and `appointment time` (2:30 PM) fields.

5. Saving to Google Sheets

427. Add a **Google Sheets** node, operation **Append Row**, targeting a "Hospital Appointments" spreadsheet/sheet.
428. Map columns manually: Name, Email, Mobile Number, Service, Appointment Date, Appointment Time — pulling Name/Email/Mobile/Service from the form trigger data, and Appointment Date/Time from the AI Agent's structured output.
429. Executing confirmed a new row was correctly appended with all fields, including the computed date and time.

6. Sending the confirmation email

430. Add a **Gmail** node (Send a Message).
431. Set recipient = email field from the form.
432. Set Subject = AI Agent's structured `mail subject` output.
433. Set Message body (plain text) = AI Agent's structured `mail body` output.
434. Remove the "append n8n attribution" option.
435. Execute and confirm: the received email showed patient name, service, computed appointment date/time, and instructions — matching what was generated.

7. Publishing and live testing

- Publish the workflow to get the Production URL/form link and share it with the class.
- Several participants submitted the live form and received confirmation emails with varying computed times (e.g., 3:00 PM, 2:30 PM, 11:30 AM), confirming the "random time between 9 AM–6 PM" logic worked per submission.

8. Discussion and suggested extensions

- Using SMS instead of email was raised as a more suitable notification channel for real hospital use — would require integrating an SMS service provider's API (not covered in this session; mentioned as a future topic, similar to earlier WhatsApp automation).
- Whether the AI Agent (LLM) is strictly necessary: for pure formatting (fixed message text), a **Set node** could replace the AI Agent, but the LLM is still useful for computing/reasoning about the appointment date and time and generating varied, natural language responses (and to keep formatting consistent for all patients, it's often better to move fixed text formatting into the Gmail node itself rather than rely on the LLM to always produce identical formatting).
- Basic validation (e.g., ensuring the mobile number has exactly 10 digits) is not built into the n8n Form node by default; would need an extra node (e.g., a "Set"/validation node or mobile number parsing node) to check and reject invalid input.
- Ideas for extending this basic version as homework/challenge:
- Store and read **available appointment slots** from Google Sheets (e.g., half-hour slots from 9 AM–6 PM) so the system picks an actually free slot instead of a random time, and marks slots as booked.

- Support **multiple doctors** via a dropdown, maintained in a Google Sheet, and send a separate confirmation email to the relevant doctor as well as the patient.

Key takeaways

- n8n's built-in **Form Trigger** node is an easy way to collect structured user input (text, email, number, dropdown fields) without needing any external HTML page.
- Marking form fields as "Required" enforces basic validation, but more complex validation (e.g., digit-count checks) requires extra nodes.
- System prompts can instruct an AI Agent to compute relative dates/times in plain English (e.g., "3 days from today, random time between 9 AM and 6 PM") without necessarily writing complex expressions — though n8n's JSON expression syntax (`$now`, `.plus()`, date formatting) is available for more precise, code-based control.
- A Structured Output Parser lets you extract multiple distinct fields (subject, body, date, time) from a single AI Agent call, useful for populating both an email and a spreadsheet row from one generation.
- For consistency across all users, it can be better to keep fixed message formatting in the destination node (e.g., Gmail) rather than leaving all formatting decisions to the LLM every time.
- This "basic version" is meant as a foundation — real-world systems would add slot availability, doctor assignment, and SMS notifications.

Demo 29: Understanding RAG and MCP (Model Context Protocol) — Concept Session

This session is a conceptual, discussion-based class (no hands-on workflow building) where the instructor and class use ChatGPT together to learn what RAG and MCP mean, working through explanations and a set of MCQs to build a working understanding of MCP in particular.

What you'll learn

- A recap of what RAG (Retrieval Augmented Generation) means and why it matters
- What MCP (Model Context Protocol) is, in plain terms
- Why MCP matters for AI agents — the difference between an AI that can only "think/answer" versus one that can "act"
- How MCP relates to other protocols (API, REST/HTTP, webhooks, gRPC, tool/function calling)
- Why MCP exists as a standardized way to connect AI models to external tools/systems

Step-by-step notes

1. Quick recap: What is RAG?

RAG = Retrieval Augmented Generation. Instead of letting an LLM answer purely from its own general/internal knowledge, you provide it with your own specific documents/data (a "library"), and instruct it: "Don't use your general knowledge — refer only to these documents and answer based on them." This matters because generic answers are often wrong or irrelevant for domain-specific or organization-specific needs — e.g., a bank's chatbot must answer according to that bank's own policies, not generic banking knowledge; a training institute's chatbot must state its own actual syllabus, not a generic one.

2. What is MCP?

The class used ChatGPT interactively to build understanding, arriving at this core definition: **MCP (Model Context Protocol) is a standardized protocol used to connect AI models with external tools, data, and systems.** Without MCP, an AI model can only think and give answers (a "brain only"). With MCP, an AI model can also take real-world actions — like sending emails, updating a spreadsheet, or calling an API (a "brain plus hands").

Example illustrations discussed:

- An AI assistant uses MCP to connect to an email system: it can read new emails and send smart replies automatically.
- An AI assistant uses MCP to connect to a CRM system: it can update customer details and send follow-up messages without manual work.

- In n8n specifically, when a chat model / AI Agent node connects to a Gmail node or Google Sheets node with zero coding, MCP-style protocol concepts are what make that connection possible under the hood (even though n8n users don't write this code themselves).

3. Key points about MCP (from the class's ChatGPT-assisted MCQ session)

- MCP helps AI connect with external tools and systems.
- It provides a **standard, consistent way** to share data between AI and applications — instead of every AI-to-tool connection needing its own custom protocol/code.
- It allows AI to perform real-world actions (sending emails, updating data, calling APIs, triggering workflows).
- It reduces the amount of manual coding needed for integrations.
- It helps build powerful AI agents and automation workflows more easily.
- MCP is specifically designed for AI systems — normal software already has APIs, REST, and webhooks and doesn't need MCP; MCP exists to help AI models understand and use those tools/APIs properly.
- A student's analogy that resonated: MCP is like a universal charger/adapter standard — instead of every device (or every AI-to-tool pairing) needing a different adapter/protocol, one standardized connector works for everything, including new tools added in the future.
- A participant explained the reasoning behind MCP's design: it functions similarly to a shared "repository" of tools/resources (comparable to a package repository like Maven) that an LLM/agent can look up and connect to directly, without needing custom code for each tool — as the number of available tools/APIs grows, this registry-style approach scales much better than one-off integrations. This also means that if a tool's underlying API changes, that complexity is handled behind the MCP layer rather than requiring the AI system's own code to be updated each time.
- MCP is compared to other existing protocols/mechanisms which serve different (or overlapping/complementary) roles:
- **API / REST / HTTP** — general request-response communication.
- **Webhook** — an event-triggered system (a URL that gets called when an event happens) — already covered in depth in Demos 26–28.
- **gRPC** — a high-speed communication protocol used in advanced systems/microservices, faster than REST.
- **Tool calling / function calling** (an OpenAI mechanism) — lets an AI decide which action/function to invoke.
- MCP is a protocol specifically designed for AI-to-tool communication, distinct from (though related in spirit to) these other mechanisms.
- A student mentioned that MCP can run either locally (using JSON-RPC over a local connection) or remotely (using an HTTP-based streaming transport) — noted as something to explore further rather than covered in full technical depth in this session.

- The class also briefly noted that Google is developing a related/complementary protocol called **A2A (Agent-to-Agent)**, still in development, for AI agents to communicate with each other (mentioned only briefly, not explained in depth).

4. "Agentic" vs "Generative" — a related distinction raised in discussion

- A tool that only replies/answers (like a plain chat model) is called **generative**.
- A tool that can actually perform actions on your behalf (send an email, update a record, trigger a task) is called **agentic** — and this action-taking capability is enabled by protocols like MCP behind the scenes. Examples discussed: GitHub Copilot taking actions (not just chatting) implies it is agentic, likely backed by an MCP-style server; similarly other AI coding/agent tools that "do" things rather than only "say" things are agentic.

5. Practical framing for the class

The instructor's main goal for this session was purely conceptual: so that if anyone asks "Do you know what MCP is?" or "Do you know what RAG is?", students should be able to confidently explain the basic idea rather than having a blank/confused reaction. Deeper hands-on MCP implementation was deferred to future sessions (mentioned as likely to come after covering LangChain/LangGraph and other groundwork).

Key takeaways

- RAG = get the AI to answer only from your specific provided documents/data, not its general knowledge — important for domain-accurate answers.
- MCP = Model Context Protocol — a standardized way for AI models to connect to and interact with external tools, data, and systems.
- Without MCP: AI can only give answers (generative). With MCP: AI can take real actions (agentic) — e.g., sending emails, updating sheets, calling APIs.
- MCP reduces the need for custom, one-off integration code between AI and each different tool/system — acting like a shared, standardized "repository/adapter" for tools that scales as new tools are added.
- MCP is AI-specific; normal software-to-software communication already has APIs, REST, and webhooks and doesn't need MCP.
- MCP sits alongside (and is complementary to, not a replacement for) other protocols like REST/HTTP, webhooks, gRPC, and tool/function calling.
- "Agentic" means an AI system can decide and take real-world actions; "generative" means it can only produce answers/content.

Demo 30: Understanding LangChain — Concept Session

This session, structured like the previous MCP session, is a conceptual walkthrough (using ChatGPT interactively plus class discussion) to build a beginner-friendly understanding of what LangChain is, why it exists, how it differs from n8n-style workflow automation, and what its core building blocks are.

What you'll learn

- What LangChain is, in plain terms, and its single-line purpose
- How LangChain differs from n8n workflow automation, and when to use each
- The core concepts inside LangChain: chains, agents, memory, tools, and retrieval (RAG)
- How ChatGPT (the product) differs from a raw LLM (like the OpenAI model) — the role of added memory/tools
- LangChain's rough syllabus/module outline for future deeper study
- That LangChain is a framework (a real code library), not just an abstract concept, and mostly used with Python

Step-by-step notes

1. What is LangChain? (Single-line purpose)

"LangChain is used to build AI applications by connecting language models with tools, data, and workflows." In other words, LangChain is a framework for building AI-powered applications where a language model (LLM) is the central component, and that LLM needs to interact with external tools, external data sources, and multi-step logic to accomplish more complex tasks than a plain chatbot could.

Example given: using LangChain, you can build an AI assistant that reads a user's question, searches relevant data (e.g., the internet or your own database), and gives a final smart answer — e.g., answering "best plots in Hyderabad" by checking both provided data and general information before responding.

2. LangChain vs n8n — when to use which

- **n8n**: mainly for **automation using visual/drag-and-drop nodes** — connecting apps (Gmail, Google Sheets, WhatsApp, forms) and automating business steps, with little/no code. Good for: "Lead comes from a website → save in a sheet → send an email" type flows.
- **LangChain**: mainly a **framework for building LLM/AI applications** — for developing agents with memory, tools, and retrieval/reasoning logic, using actual code (mostly Python). Good for: "User asks a complex question → AI searches a knowledge base → reasons → answers" type systems, or when you need deep customization of AI behavior beyond what visual automation tools allow.
- Rule of thumb discussed: use n8n when you want to automate business processes fast with minimal code; use LangChain when you want to build custom, code-based AI logic — deep agentic behavior, memory, and retrieval using LLMs directly.
- Many real projects use **both together**: LangChain to handle the "AI brain"/logic, and n8n to handle the surrounding business automation ("outer business" steps).

- A participant noted a practical distinction: n8n is more tool/node-based (with an associated license/subscription cost for the cloud service), while LangChain is open and code-based, offering more flexibility and no licensing cost — but requiring programming skill (mainly Python) to use.

3. ChatGPT vs a raw LLM (important distinction raised in Q&A)

A key clarification made during the session: **ChatGPT (the product) is not the same thing as the underlying OpenAI LLM.** The raw LLM, used directly via API calls, has no built-in memory of past messages, no UI, and no extra tool integrations by itself — it only answers based on its trained knowledge for a single request. ChatGPT-the-application wraps that raw LLM with extra engineering: conversation memory, tool access, and a user interface. This was demonstrated by recalling an earlier session's n8n test: asking "Who is the US president?" got a correct answer, but a follow-up "What is his age?" (without any memory added to the AI Agent) failed, because the raw LLM call had no memory of the earlier question. This is exactly the kind of "extra capability" (memory, tools, workflows) that frameworks like LangChain are built to add around a raw LLM.

4. Core LangChain concepts explained (via the ChatGPT-assisted MCQ walkthrough)

- **Chaining**: connecting multiple steps of AI tasks together, where the output of one step becomes the input of the next — e.g., "question → search → summarize → final answer." Conceptually similar to designing multi-step flows in n8n, but done in code.
- **Agent**: an AI system that can decide and take actions — it can choose tools, search data, and execute steps automatically, rather than just replying with static text. An LLM must be present inside an agent (there's no "agent" without an underlying LLM to reason and decide).
- **Memory**: allows the AI to remember previous conversation turns, enabling context-aware, personalized follow-up responses (e.g., remembering "my name is Durga" so a later "What is my name?" can be answered correctly). Memory is distinct from plain **storage**: storage just saves raw data (like files/databases) without understanding context or conversation flow, while memory combines data plus understanding plus usage in ongoing interactions.
- **Tools**: allow the AI to interact with external functions like APIs, calculators, or databases — giving the AI "hands" to perform real actions, not just produce answers (e.g., using a calculator tool to solve math, or an API to fetch live data).
- **Retrieval (RAG)**: helps the AI fetch real information from external sources (PDFs, databases, the web) before answering, rather than guessing from its own trained knowledge — reducing wrong answers and improving accuracy/up-to-dateness. As emphasized: "Don't guess — go check, then answer."
- Together, **memory + tools + chains + agents** form the combination that makes LangChain-built AI systems powerful: memory remembers context, tools perform actions, chains handle step-by-step flow, and agents make decisions — all working together to create real-world, useful AI systems.

5. A caveat on RAG discussed in Q&A

If the documents/data provided for retrieval (RAG) contain wrong information, the AI's answer will also be wrong — this is not a flaw in the LLM itself. The LLM's role in a RAG setup is only to retrieve from and nicely phrase/present the provided source material, not to apply its own judgment about what's "actually correct." The accuracy of a RAG-based answer is only as good as the accuracy of the underlying documents supplied.

6. Is LangChain a "concept" or a "framework"?

Explicitly clarified in Q&A: LangChain is a **framework** — a real, usable code library that developers use, not just an abstract idea or a protocol. It implements and packages together the concepts of chaining, agents, memory, and tools into ready-to-use components so developers don't have to build everything from scratch.

7. Proposed LangChain "syllabus" outline (generated via ChatGPT, for future sessions)

Roughly, in order:

436. Basics of LangChain — what it is, why it's needed, LangChain vs normal API usage, real-world use cases.
437. Core components overview — LLMs, prompts, tools, memory.
438. Prompt templates — dynamic input handling, reusable prompts.
439. Chains — workflow design.
440. Memory — context handling.
441. Tools — external integrations.
442. Agents — decision-making.
443. Retrieval — RAG.
444. Multi-step agent workflows — combining chains, tools, and memory for complex workflows.
445. Mini projects and advanced concepts — multi-agent systems and further development/integration topics.

The instructor indicated the class may spend a significant amount of time (potentially weeks) building deeper hands-on knowledge of LangChain, since getting comfortable with it will make later, more advanced concepts (like LangGraph) easier to pick up.

8. Closing notes

- LangChain is mostly used with **Python**.
- The instructor plans to briefly touch on LangGraph in an upcoming session, and to keep using this "5 minutes on one AI term per day" approach (having covered MCP the day before this session, and RAG earlier) to build vocabulary/familiarity gradually across the course.

Key takeaways

- LangChain = a framework for building AI applications by connecting a language model with external tools, data sources, and multi-step logic/workflows.

- n8n = visual, low/no-code workflow automation connecting apps; LangChain = code-based (mostly Python) framework for building custom AI application logic — many real systems use both together.
- ChatGPT (the app) is not the same as a raw LLM — the "extra" capabilities (memory, tool use, UI) come from wrapping the LLM with additional engineering, which is exactly the kind of thing LangChain helps developers build themselves.
- Four foundational LangChain concepts: **chains** (multi-step task flow), **agents** (AI that decides and acts), **memory** (remembers past conversation context, distinct from plain storage), and **tools** (let AI call external functions/APIs to perform real actions).
- **Retrieval/RAG** lets the AI check real external sources before answering instead of guessing — but the answer is only as accurate as the source documents provided.
- LangChain is a genuine, widely-used code framework/library (not just an idea), primarily used with Python, and is planned to be a major, multi-session topic later in the course.

Demo 31: LangChain vs LangGraph vs n8n — Understanding the Framework Landscape

This is a short "consistency session" where the instructor and students use ChatGPT to build their own understanding of LangGraph from scratch, then quiz themselves with 20 MCQs comparing LangChain, LangGraph, and n8n. There is no hands-on coding here — it is a concept-building and terminology session.

What you'll learn

- What LangGraph is and how it differs from LangChain
- The idea of "stateful" vs "stateless" systems
- How LangChain, LangGraph, and n8n compare and where each fits
- Why combining an AI framework (LangChain/LangGraph) with n8n is a powerful pattern
- A self-test approach: using AI to generate practice MCQs and validate your answers

Step-by-step notes

446. **Recap of LangChain.** LangChain is a framework for building complex AI-based applications. It provides ready-made building blocks — memory, tools, and chaining — so you don't have to hand-code everything yourself. Without a framework like this, building AI apps means a lot of manual coding and complexity.
447. **Introducing LangGraph (learned live via ChatGPT).** The instructor admits he doesn't know LangGraph either, and uses ChatGPT to learn it live with the class — a good example of "using AI to learn AI." The definition ChatGPT gave:

LangGraph is a framework used to build stateful, step-by-step AI agent workflows using a graph structure, with memory and control over execution flow.

448. **Key term: stateful vs stateless.**

- **Stateful** = the system can remember context from previous steps/requests.
- **Stateless** = every request is treated as brand new, with no memory of what came before.
- Memory and "stateful" are directly related — being stateful means having memory of the conversation/process so far.

449. **Simple analogy for LangGraph.** Think of it like a flowchart for AI: user asks a question → AI searches data → checks the answer → improves it → gives the final answer. Each step is a "node," and the connections between steps are "edges" (arrows). This is "step-by-step thinking plus control" — the AI can loop back, retry, and branch, not just move forward blindly.
450. **LangGraph vs n8n.**

- Both connect multiple steps ("nodes") to complete a task — that's the similarity.

- **n8n** is a visual, drag-and-drop, low-code/no-code automation tool. Good for business automation: connecting Gmail, Google Sheets, WhatsApp, forms, APIs.
- **LangGraph** is a code-based framework for AI reasoning: question → think → search → evaluate → improve → answer. You write code for each node's logic.
- A recommended pattern: "LangGraph for AI brain + n8n for automation actions" — a powerful combo where LangGraph handles the thinking and n8n handles real-world actions (sending emails, updating sheets, etc.).

451. **LangChain vs LangGraph — the core differences:**

Aspect	LangChain	LangGraph
Flow type	Linear (Step A → B → C)	Graph-based (can go forward, back, loop)
Control	Limited	Full control — retry, branching, conditions
Use case	Simple apps, chatbots, Q&A	Complex agents with reasoning/decision-making
Flow	Predefined	Dynamic (AI decides next step)
Learning curve	Easier to start	More advanced/powerful

Recommended learning path: start with LangChain to build fundamentals, then move to LangGraph for advanced agent behavior. LangGraph internally relies on graph structures (related to "Tree of Thought"/graph reasoning concepts covered earlier in the course).

452. **Practice via 20 rapid-fire MCQs (AI-generated and self-validated).** The instructor prompts ChatGPT: "Provide MCQs one at a time, wait for my answer, validate it, give a learning conclusion, then wait for confirmation before the next question." Key facts reinforced through the quiz:

- n8n = best for visual, no-code/low-code workflow automation, connecting apps like Gmail, Google Sheets, WhatsApp via drag-and-drop nodes.
- LangChain = best for simple, step-by-step AI apps using "chains" (linear sequence: question → search → summarize → answer). Great for chatbots and Q&A, and has strong native support for RAG (retrieval, embeddings, vector databases) — this is why LangChain, not n8n, is the best choice for PDF/document Q&A systems.
- LangGraph = best for advanced AI workflows needing loops, retries, conditions, and state (question → think → search → check → retry → improve → answer). Best when you want the AI to dynamically decide the next step.
- Both LangChain and LangGraph share the concept of **memory** for maintaining conversation context.

- n8n is an **automation tool** (connects apps, triggers actions on events like form submission); LangChain/LangGraph are **AI frameworks** (used to build the "thinking" part of an application).
- The best overall approach for a complete system ("AI thinks" + "action happens in real apps") is to **combine an AI framework (LangChain/LangGraph) with n8n**.
- Clarification raised by a student: n8n existed before the AI boom — it was originally just an automation platform; AI-powered nodes were added to it more recently. So n8n by itself, in its classic explanations, doesn't natively "think" like an agent — it mainly triggers and automates.

453. **Closing point.** After the quiz, everyone in class could confidently explain LangChain, LangGraph, and n8n and how they differ — going from "I don't know anything about LangGraph" to being able to discuss it fluently in under an hour, showing the value of using AI as a personalized tutor to rapidly learn new terminology.

Key takeaways

- LangGraph = framework for stateful, graph-based, step-by-step AI agent workflows with memory and execution control.
- LangChain = simpler, linear framework for chaining prompts/tools/data; great starting point and best for RAG-style document Q&A.
- n8n = visual, low-code automation tool for connecting real-world apps and triggering actions; not designed for deep AI reasoning by itself.
- Best real-world architecture: combine an AI reasoning framework (LangChain or LangGraph) with n8n for actions — "AI brain + automation engine."
- A good learning technique: ask ChatGPT to quiz you one question at a time and validate your answers to build confidence in new terminology.

Demo 32: Reviewing a Student-Built Hospital Appointment Workflow in n8n (with Code Node Logic)

A short, informal session (mostly casual conversation about tools like Codex, Cloud Code, and industry news) that transitions into a live review of a student's ("Mahesh" / "Omkar") n8n hospital appointment booking workflow, focused on how to check doctor slot availability using a JavaScript Code node.

What you'll learn

- How to read booking data from Google Sheets in n8n and split it into individual rows
- How to write custom JavaScript logic inside n8n's Code node to check slot availability
- The pattern of using AI (ChatGPT) to help write/debug that code
- Why "Get Rows" + Split Out is used instead of filters, and its limitations
- The homework/assignment given to extend the workflow to also check by date

Step-by-step notes

454. **Context/background chat.** Before the technical walkthrough, there's discussion of other tools in the industry: Codex (integrated with Visual Studio, used to pull Jira ticket patterns without logging into Jira), Cloud Code (a command-line coding assistant used even on legacy codebases from the 1970s–80s), and news about IBM's stock dropping ~23% in a day and Google/Alphabet investing $40 billion in Anthropic — illustrating how fast the AI tooling space is moving. None of this is core curriculum, just context-setting.
455. **Reviewing the hospital appointment workflow (built by a student as homework).** The student shares two versions: one using an AI agent/model, and one built purely with JavaScript code (no AI agent). The instructor asks the student to walk through his logic node by node.
456. **The form.** A simple appointment form: name, email, mobile number, service (dentist, orthopedic, cardiology, general physician), appointment date (date-type field), and appointment time (checkboxes for slots like 9:00, 9:30, 9:45, 10:00 — the student explains slots were created because he assumed each doctor is available for about 1 hour, so half-hour slots from 9 to 6 make sense).
457. **Get Rows node (Google Sheet).** After form submission, the workflow uses a Google Sheets "Get Rows" node to pull **all rows/records** from the sheet (no filter applied, since the student needs to check all bookings). The sheet stores: name, email, mobile number, service, appointment date, appointment time.
458. **Split Out node.** Because "Get Rows" returns all data as one combined object (not row-by-row), the student uses a **Split Out** node, specifying fields like appointment date, appointment time, and service. This breaks the data into individual rows/items so each booking can be checked separately — necessary input for the next JavaScript step.
459. **Custom JavaScript Code node.** After splitting, the student writes JavaScript (partly self-written, partly fixed by pasting the error into ChatGPT) to:

- Take the end user's **requested time**.
- Get all **booked times** already in the sheet (as JSON).
- Check if the requested time is already taken.
- Note: at this point the logic only checked **time**, not **date** — a gap the instructor flags as needing correction (two people could request the same time slot on different dates and incorrectly be marked as conflicting).

460. **Handling "slot not available."** If the requested slot is already booked, the workflow builds an email body (without using an AI agent — just a plain code/logic-based message) telling the patient the slot isn't available and listing which other slots are open that day, then sends this via a Gmail node.
461. **Handling "slot available."** If the slot is free, a **Structured Output** step generates the confirmation details (mail body, appointment date, appointment time), and the workflow uses **Append or Update Row** in Google Sheets to mark that slot as now booked, then emails the patient a confirmation.
462. **Design note called out by the instructor.** Throughout all the workflows built in this course so far, the pattern has consistently been: Get Rows → Split Out → custom logic → Append/Update Row. This is a reusable pattern for "check availability, then update" type workflows.
463. **Language choice: JavaScript vs Python in n8n's Code node.** Both are supported in n8n's Code node. A student asked whether Python has less capability than JavaScript in n8n — the instructor clarifies there's no real difference in code execution capability; regardless of language, data is transferred in JSON format under the hood, since JSON is a universal data format, not JavaScript-specific.
464. **Assignment given to the class.** Build/improve this workflow so that it checks availability **by both date and time** (not just time), and if a slot isn't available on the requested date, respond with the other available slots on that same day, prompting the user to resubmit. Students are asked to attempt this and share their solutions so the instructor can make a refined version available to everyone. This assignment is picked up and demoed in Demo 34.

Key takeaways

- n8n's "Get Rows" pulls the entire sheet as one object; "Split Out" is needed to turn it into individual row items you can loop/compare against.
- Custom logic (checking date + time together, not just time) is often needed on top of a basic AI agent — pure automation nodes alone may not capture full business rules.
- You don't need an AI agent for every step — plain JavaScript/Python code in a Code node can generate email bodies and handle conditional logic just as well, and is often simpler for deterministic tasks.
- n8n's Code node supports both JavaScript and Python; data always ends up as JSON regardless of language choice.

- ChatGPT can be used effectively to debug and complete code you're stuck on (paste your code + error, get a fix).

Demo 33: Understanding RAG (Retrieval-Augmented Generation) — Concepts and Self-Test

A concept-only session (no coding) that explains what RAG is, why it matters, and walks through the RAG pipeline (retriever → generator → response) using a 7-question AI-generated quiz to reinforce understanding.

What you'll learn

- What RAG (Retrieval-Augmented Generation) means and why it's needed
- How a plain LLM answers vs. how a RAG-powered system answers
- The RAG pipeline: retrieve → generate → respond
- Key components: retriever, generator, vector database, embeddings
- Real-world use cases for RAG

Step-by-step notes

465. **The problem RAG solves.** A normal LLM (like ChatGPT) answers using its own pre-trained, generic knowledge. This is a problem when you need answers specific to your organization — e.g., asking "What is the interest rate on a fixed deposit at ICICI Bank?" A generic LLM can't know a specific bank's current, internal rates.
466. **The RAG concept in plain language.** You tell the AI: "Don't use your existing knowledge — refer only to these documents I'm providing, and answer strictly from them." This is exactly what RAG (Retrieval-Augmented Generation) does. Analogy used: imagine a trainer who is told, "Teach only from these four books, not from your own general knowledge" — every answer must be traceable back to the provided source material.
467. **Key benefits of RAG:**

- Reduces hallucination (fewer wrong/made-up answers), because the AI is grounded in your provided data instead of guessing.
- Produces more specific, domain-relevant answers.
- Produces more believable/trustworthy output.
- Gives access to real-time/updated information without retraining the model.

468. **The RAG pipeline (reinforced via the quiz), in order:**
469. **Retrieve** — relevant information is pulled from external sources (PDFs, databases, documents) based on the user's query.
470. **Generate** — the LLM uses that retrieved data to produce an accurate answer.
471. **Respond** — the final answer is given to the user.

- Correct order: **Retrieve → Generate → Respond** (not train, not generate-first).

472. **Two key components:**

- **Retriever** — responsible for searching and finding the relevant documents/chunks (like a search engine). It works by converting the user's question into embeddings and comparing it (via similarity/cosine similarity) against embeddings stored in a vector database, to find the closest matching chunks.
- **Generator (the LLM)** — takes the retrieved chunks and turns them into a well-formed, final answer. The LLM does not do the searching/retrieving itself — it just prepares/generates the response text using what the retriever found. Analogy: "Retriever is like a Google search, generator is like a teacher explaining the result."

473. **Vector databases and embeddings.**

- RAG systems typically use a **vector database** to store and search embeddings (though some relational databases, e.g. PostgreSQL, now also support vector/embedding columns; plain MySQL typically does not).
- **Embeddings** = numerical representations of text that capture meaning (not images, audio, or encrypted data). Similar meanings produce similar/nearby numbers — e.g., "car" and "vehicle" have embeddings close together, while "car" and "banana" are far apart. This "semantic distance" concept connects back to earlier sessions on transformer architecture and the attention mechanism, which also relies on this kind of semantic relationship between tokens.
- Instead of keyword search, RAG systems do **semantic (meaning-based) search**.

474. **Why RAG beats relying only on a pre-trained LLM:** it allows real-time access to updated/external information without needing to retrain the model — answers stay accurate and current.
475. **Common real-world use case highlighted:** a chatbot that answers questions from a company's own documents — PDFs, internal docs, knowledge bases — for customer support, HR policies, or internal training (e.g., "What is the new policy?" — where a generic LLM would give a generic answer that may not match your specific company's actual policy, but a RAG chatbot searches your company docs and gives the exact, correct answer).
476. **Self-test method used again:** the instructor prompted ChatGPT for "7 MCQs on RAG, one at a time, wait for my answer, validate, and give a learning conclusion" — the same self-quizzing technique used in Demo 31.

Key takeaways

- RAG = "don't rely on built-in knowledge — retrieve from my provided documents, then generate the answer from that."
- The pipeline is always: Retrieve → Generate → Respond.
- Retriever finds relevant chunks (via embeddings + vector similarity search); Generator (the LLM) turns those chunks into a polished answer.
- Embeddings are numeric representations of meaning; similar meanings sit close together in vector space — this is the foundation of semantic search.

- RAG's biggest benefits: less hallucination, more domain-specific accuracy, up-to-date answers without retraining.
- Classic real-world RAG use case: a company chatbot that answers only from internal documents/policies.

Demo 34: Building a Smarter Hospital Appointment Workflow — Live Walkthrough of a Student's Solution

This session is a detailed, live code-review of a student's ("Mahesh") advanced solution to the homework from Demo 32: an n8n workflow (triggered by a custom HTML/webhook form, not an n8n-native form) that checks real doctor availability by both date and time, and dynamically offers alternative slots by generating custom JavaScript with the help of ChatGPT.

What you'll learn

- How to trigger an n8n workflow from a custom HTML form using a **Webhook** node instead of n8n's built-in form node
- How to filter a Google Sheet read so you don't pull unnecessary data (performance optimization)
- How to merge webhook input with sheet data and write JavaScript to compare them
- How to conditionally route logic with an **If** node based on a boolean flag from code
- When you actually need an AI agent node vs. when a hardcoded message is simpler and cheaper
- A live demo of the full booking flow, both for an available slot and a conflicting slot

Step-by-step notes

477. **Recap of the basic version.** In the previous session, the workflow simply saved patient info to Google Sheets and generated a **random** appointment slot 3 days later, from a 9-to-6 window, and emailed the patient. The improvement needed: let the **patient choose** their preferred date/time, check real availability, and only confirm if that slot is actually free — otherwise offer alternatives.
478. **Live demo of the improved version.**

- The student built a custom appointment form (name, phone, email, service/specialist choice — e.g., cardiologist, dentist, lab — appointment date, hour selection between 9–6, and a free-text "reason for visit").
- Submitting the form for a new date/time (e.g., 29th, 4 PM, cardiology) returned "Success — your appointment has been booked," and the patient received a confirmation email generated by the AI agent, including instructions like arriving 15 minutes early and bringing ID.
- Submitting the **same date, time, and service (cardiologist) again** correctly triggered a rejection: "We regret to inform you that the requested appointment slot is not available," followed by a list of four other available time slots on the same day.

479. **How the form was built.** Unlike previous workflows that used n8n's native Form node, this form was a custom **HTML page** (built with the help of "Gemini Canvas," referred to as an AI coding tool), and it submits to the n8n workflow via a **Webhook** node/URL rather than an n8n Form trigger.
480. **Workflow structure (node by node):**
481. **Webhook** — receives the form submission (name, phone, email, service, appointment date, appointment time, reason).

482. **Respond to Webhook** — sends a default acknowledgment message back to the browser/caller immediately after submission (this is separate from the final email).
483. **Google Sheets — Get Rows (filtered)** — reads existing confirmed appointments, but **filtered by the requested appointment date** (using a filter condition on the "appointment date" column matching the value from the webhook). This avoids retrieving the entire sheet (e.g., 1,000 rows) when only one day's data is needed — a deliberate performance optimization. The instructor also suggested adding a **second filter on service** (not just date), since the same time slot could be free for one specialist but booked for another — the student agreed this was a gap to fix.
484. **Merge node** — combines two inputs: (a) what the user just submitted via the webhook, and (b) the existing matching rows from the Google Sheet, so both are available to the next step. Configured with default settings (Append mode, 2 inputs).
485. **Code node (JavaScript)** — compares the user's requested date/time against the existing booked slots (from the sheet) for that day. Outputs a boolean flag `available` (true/false), and if not available, calculates the other free slots between 9 AM and 6 PM (parameters used: start 9:00 AM, end 6:00 PM, 60-minute slot duration) by excluding times already taken from the sheet.
486. **If node** — branches based on the `available` flag: true → confirmation path; false → rejection/alternative-slots path.
487. **AI Agent (only used on the success/confirmation path)** — generates a polished confirmation message. On the "not available" path, the student decided **not** to use an AI agent at all — instead just hardcoded the apology message directly in the Gmail node, to save on LLM token costs, since a fixed message format is cheap and doesn't need generative flexibility.
488. **Code node (JavaScript, again)** — formats the list of available alternative slots into JSON and builds the email body used in the Gmail node.
489. **Gmail node** — sends the final confirmation or apology-with-alternatives email.
490. **How the JavaScript code was generated.** The student (self-described as a "data guy," a solution architect who knows Python but not JavaScript) used ChatGPT to generate the comparison code. His prompt approach: describe the n8n workflow goal in plain English ("In n8n I'm creating a workflow for hospital appointment requests... the workflow should review date/time based on availability and respond accordingly — how can I check appointment availability?"). ChatGPT initially suggested using a Google Calendar node; the student then said "I want to check Google Sheet instead," and ChatGPT adjusted its suggested solution and code accordingly. He initially tried Python but hit issues (e.g., variable naming with underscores not working as expected in n8n's Python support), so switched to the JavaScript version ChatGPT provided. He also had to specifically ask ChatGPT for a merge step, since he initially didn't realize the webhook data and sheet data needed to be combined before comparison.
491. **Debugging note.** The student's code initially only checked whether a **time** was available, not whether it was on the correct **date** — a real bug the instructor caught and asked him to fix by adding date-matching logic.
492. **Discussion: do you always need an AI agent node?** A student (Ashish) asked whether the AI agent node (with its chat model) is mandatory whenever you use n8n's AI Agent feature. The

answer: **no** — an AI agent needs a connected chat model only if you're actually using it to generate content. In this workflow, the AI agent was used only to draft the success message; for the "not available" branch, the team removed the AI agent entirely and hardcoded the response format directly into the Gmail node, since there's no real "thinking" needed for a fixed apology message.

493. **Broader discussion: should you use an LLM/AI agent for the comparison logic instead of raw code?** Another student asked why not do the date/time comparison using an LLM/chat model instead of JavaScript. The instructor's answer, grounded in real-world practice: yes, it's technically possible, but in production you should **reserve LLM calls for tasks that truly need reasoning/generation**, because LLM usage is tied to token cost. Using an LLM for a simple deterministic comparison (is this slot booked or not) wastes money — plain code is faster, free, and equally accurate for that kind of task. Use LLMs where they add real value (natural language generation, judgment calls), not for basic logic.

494. **n8n Code node — reminder.** The Code node supports only two languages: **JavaScript** or **Python**. n8n is described as "zero-coding or low-coding automation" — you *can* still add custom code when the built-in nodes aren't flexible enough, but most of the workflow remains visual/no-code.

Key takeaways

- You can trigger n8n workflows from **any external source** via a Webhook node — not just n8n's built-in Form trigger — enabling custom-built HTML/JS forms (e.g., built with an AI coding tool like Gemini Canvas).
- Filter your Google Sheets reads (by date, and ideally also by service/category) to avoid pulling unnecessary rows — this is a real performance optimization technique.
- Use a **Merge** node whenever you need to combine data from two different sources (e.g., incoming webhook data + existing sheet data) before running comparison logic.
- Use an **If** node with a boolean flag (computed in a Code node) to branch between "available" and "not available" paths.
- Don't over-use AI agents: reserve them for tasks that genuinely need natural-language generation or reasoning; use hardcoded messages/plain code for deterministic, fixed-format outputs to save token cost.
- ChatGPT is effective for generating workflow logic code, but you must clearly describe your exact data source and constraints (e.g., "use Google Sheets, not Google Calendar") and iterate/debug with it.
- n8n's Code node supports JavaScript and Python only (not Java).

Demo 35: Telegram Bot Automation in n8n — Building a Course Info & Interview-Question Bot

After a short detour on AI image-generation prompts (ChatGPT/OpenAI's image tool for photo-realistic image generation, not core curriculum), this session is a live, hands-on build of a Telegram chatbot in n8n: creating a bot via Telegram's BotFather, wiring it into an n8n workflow with an AI agent, adding memory for conversation context, and troubleshooting several real credential/configuration errors along the way.

What you'll learn

- How to create a Telegram bot using **BotFather**
- How to connect that bot to n8n using a Telegram API access token
- How to build a simple Telegram automation: Telegram Trigger → AI Agent → Telegram (send message)
- How to add a system prompt to give the bot a specific persona/purpose
- How to add session-based memory so the bot remembers a conversation
- Common real-world errors when setting up Telegram credentials in n8n, and how to fix them

Step-by-step notes

495. **Brief aside on image generation (not part of the main topic).** The instructor noted that OpenAI's newer image generation tool (referred to as "Image 2.0") is currently better than Google's "Nano Banana" for maintaining facial consistency in generated images, and demoed a few prompt styles for turning an uploaded photo into stylized images (e.g., a "bride" themed image, "photo album" effects). This was framed as a fun, optional aside, not a technical building block for agent development.
496. **Why Telegram automation, and how it compares to WhatsApp automation.** The instructor notes that WhatsApp automation took the class three days of struggle to get working previously, but Telegram automation is much simpler by comparison. The basic pattern is:

Telegram Trigger → AI model (AI Agent) → Telegram node (send message)

- Telegram Trigger fires whenever the bot receives a message.
- The message is sent to the AI agent, which prepares a response.
- The response is sent back to the user via the Telegram node.

497. **Step 1 — Create a new n8n workflow and add a Telegram Trigger node.** Select the **"On message"** trigger type (other options include "On callback query," "On channel post," "On edited channel post," "On edited message" — but "On message" is what's needed here).
498. **Step 2 — Create a Telegram bot using BotFather.**
499. Open Telegram (desktop or app) and search for **"BotFather"** (an official bot used to create and manage other bots — described as "father of all bots").

500. Click **Start** (or type `/start`) to begin.
501. Send `/newbot` (via the "New Bot" option) to create a new bot.
502. Choose a **display name** for the bot (e.g., "Durga AI agent bot").
503. Choose a **username** for the bot — this **must end in "bot"** (e.g., "DurgaAIagentBot"). The display name does not strictly have to end in "bot," but it's recommended to keep both consistent and ending in "bot."
504. BotFather then generates an **API access token** (a long alphanumeric string) — this is the credential you'll paste into n8n. Keep this token private.
505. **Step 3 — Configure the Telegram credential in n8n.** In the Telegram Trigger node, click "Set up credential," paste the API access token from BotFather, and save. n8n will show "Connection tested successfully" if the token is valid.
506. **Step 4 — Troubleshooting a real credential error (a large chunk of this session).** The class hit a recurring error: *"Credential with ID does not exist for the Telegram API"* / *"Telegram trigger node does not have access to the credential."* Things attempted while debugging:

- Deleting and recreating the Telegram credential.
- Creating multiple new bots via BotFather (hitting a note that Telegram limits a user to **20 bots**).
- Checking whether deleting a chat / blocking a bot too quickly after creation caused an issue.
- Eventually the actual root cause was found: the credential's **sharing settings** in n8n. Under the credential's "Sharing" tab, the correct **project** ("my project two" in this case) was not selected/shared — once the instructor added that project under "Share with users/projects," the credential started working immediately. This is an n8n workspace/project permissions issue, not a Telegram-side problem.
- Lesson emphasized: n8n credentials can be scoped to specific projects, and if a workflow lives in a project that doesn't have access to a given credential, you'll see exactly this kind of "does not exist"/"does not have access" error — check the credential's sharing settings first.

507. **Step 5 — Test the trigger.** Open the bot's Telegram link (e.g., `t.me/YourBotName`), click Start, send a message like "hello," and execute the node in n8n to confirm the trigger receives it.
508. **Step 6 — Add the AI Agent node.**

- Set the **source for the prompt** to "Define below" (since there's no separate chat trigger node connected).
- For **user message**, drag in the field that holds the incoming text: `{{ $json.message.text }}` (the Telegram message text).
- Add a **System Message / system prompt** describing the bot's persona and behavior. Example used: *"You are DurgaSoft, an official assistant of DurgaSoft, responsible for providing course information and new batch information in a precise and polite way."* ChatGPT was used to draft a fuller system prompt (including contact numbers, email, and address for DurgaSoft) which was pasted into the system message field.

509. **Common execution errors encountered and their fixes (real troubleshooting shown live):**

- *"The workflow has issues and cannot be executed"* — often resolved by stopping the current execution, letting the previous node's data load, and re-executing.
- *"No input data — json.message.text only. Did you add JSON in model?"* — fixed by referencing the object property correctly instead of a malformed expression.
- *"Parameter prompt / user message required"* — needed the user-message field properly mapped from the trigger's JSON output.
- General workaround tip that worked repeatedly: execute the workflow in n8n (so it's "listening"), and *while it's running*, send a message from the actual Telegram app — this triggers a live, real execution that populates all the node data correctly.

510. **Step 7 — Add Simple Memory for conversation continuity.** n8n's Simple Memory node requires a **Session ID** so that the same chat/conversation is remembered across messages. The session ID used here is the **chat ID** from the Telegram Trigger node's output (`$json.message.chat.id`) — this uniquely identifies each Telegram conversation, so mapping it into the memory node's Session ID field (set to "Define below") lets the bot remember prior messages within that same chat.
511. **Step 8 — Send the response back via Telegram.** Add a Telegram "Send a text message" node:

- **Chat ID** — mapped from the Telegram Trigger's chat ID (so the reply goes to the right person/chat).
- **Text** — mapped from the AI Agent node's **output** field (the generated reply).

512. **Step 9 — Publish the workflow.** Clicking "Publish" makes the workflow live, so it responds automatically without needing to manually click "Execute workflow" each time in the editor.
513. **Live test of the finished bot.** Messaging the published bot ("Hello," "Which courses you are offering?") returned an AI-generated reply listing DurgaSoft's courses (Java Full Stack, Python Full Stack, Data Science, AI, AWS, Selenium). When asked about faculty names (not included in the system prompt), the bot correctly said it didn't have that information and suggested contacting DurgaSoft directly — showing the system prompt's information boundaries in action.
514. **Note on the "sent automatically" watermark.** Every bot message included the line "This message was sent automatically with n8n" — this appears because the class is using n8n's **trial/free version**; a licensed n8n plan removes this watermark.
515. **Second worked example — "Interview Questions Bot."** To reinforce the pattern, the instructor built a second bot from scratch (new BotFather bot, e.g., "Just Learn 7 Bot") with a simpler system prompt: *"Generate three important interview questions on the provided topic."* This time, **no memory node was used** at all (a student asked why — the instructor and class noted memory isn't strictly required if you don't need multi-turn context; the earlier bot's session ID was taken directly from the Telegram Trigger node's chat ID and passed straight through to the Telegram send node without truly needing the memory node either). Testing it with topics like "Python," "Java," "AI," and "n8n" correctly returned three relevant interview questions each time (e.g., for Python: "difference between list and tuple," "explain the GIL"; for Java: "== vs .equals()," "ArrayList vs LinkedList").
516. **Small UI tip.** The "sent automatically with n8n" watermark can also be toggled off via an "Additional Fields" option inside the "Send a text message" node (similar to how additional fields work in the Gmail node).

Key takeaways

- The core Telegram bot pattern in n8n is: **Telegram Trigger (On message) → AI Agent (with system prompt, optional memory) → Telegram (Send a text message)**.
- Create bots via Telegram's **BotFather**: `/newbot`, choose a display name, choose a username ending in "bot," and copy the generated API token into n8n's Telegram credential.
- A Telegram account can have at most **20 bots**.
- If you see "credential does not exist" or "does not have access to the credential" errors in n8n, check the credential's **Sharing settings** first — the workflow's project may not have been granted access to that credential.
- Map the **Session ID** in the Memory node to the Telegram chat ID (`$json.message.chat.id`) so the bot remembers context per conversation.
- Memory isn't mandatory for every bot — simple, stateless bots (like a quick interview-question generator) can skip it if there's no need to remember prior turns.
- The n8n trial version stamps outgoing messages with "sent automatically with n8n"; this can be removed via a licensed plan or an additional-fields setting.
- A reliable debugging trick when a node "has no input data": execute the workflow so it's actively listening, then trigger a real message from the live app (e.g., Telegram) rather than relying only on manual test data.

Demo 36: Building a Telegram Bot in n8n with an AI Agent (Course Recommender) + Two Student Project Reviews

This session shows how to build a Telegram bot from scratch in n8n that uses an AI agent to recommend life-skills courses based on a student's problem, then adds clickable buttons that link to course pages. The second half is a review of two student-built workflows: an AI-based customer support/FAQ bot and a hospital appointment booking system.

What you'll learn

- How to create a brand-new Telegram bot using Telegram's "BotFather"
- How to wire a Telegram trigger to an AI agent (AI model) in n8n and send the AI's reply back to the user
- How to write a system prompt that constrains an AI agent to a specific task (recommend a course from a fixed list)
- Why AI can "hallucinate" when a prompt has no guardrails, and how a system prompt acts as a guardrail
- How to add inline buttons with a URL to a Telegram message so users can click through to a course page
- The difference between an "AI agent" node and an "Information Extractor" node (thinking power vs. token efficiency)
- How a student built a hospital appointment system using Google Sheets as a database, JavaScript code nodes for business logic, Docker + ngrok for free local hosting, and Gmail for confirmation emails

Step-by-step notes

Part 1: Building the Telegram course-recommendation bot

517. Open n8n, create a new workflow, and add a **Telegram Trigger** node set to "trigger on message."
518. To create a new bot, open Telegram and message **BotFather** (the official Telegram bot for creating bots). Use `/newbot`, give it a display name (e.g., "Life Skills with Durga"), and a username that must end in "bot" (e.g., `life_durga_bot`). BotFather returns an API token.
519. In n8n, create a new Telegram credential using this API access token.
520. Test the trigger: send "Hello" to the bot in Telegram, then execute the workflow in n8n — the message should appear in the trigger node's output.
521. Add an **AI Agent** node (with OpenAI, Gemini, or any LLM of your choice as the model). Since there's no chat trigger node, you set the "Text" input manually (choose "Define below") and select the incoming Telegram message field.
522. Write a **system prompt** describing the agent's job. Example structure used: "You are an AI designed to guide students, job seekers and professionals. Understand the user's problem clearly,

suggest the most suitable course from the available list, explain how that course solves their problem." The prompt included a list of the instructor's actual courses (e.g., Atomic Habits, Stop Worrying and Start Living, Psychology of Money, Extreme Ownership) and required a fixed reply structure: problem understanding → recommended course → why this course.

523. Add a **Telegram node → Send a text message** action after the AI agent. Set:

- Chat ID (needed to reply to the correct conversation/session)
- Text = the AI agent's output

524. Publish the workflow to production and test live in Telegram by describing a problem (e.g., "I overthink everything and always worry") — the bot replies with a matching course recommendation.
525. **Guardrails matter:** without instructions to ask for more detail, the AI will guess and can hallucinate a random course from any vague input (e.g., recommending "Ikigai" from meaningless text). Adding a rule to the system prompt (e.g., "if the problem is unclear, ask the user to clarify") prevents this — this is called adding a **guardrail**.
526. The AI model works in any language automatically (tested with Hindi) because the underlying LLM handles multilingual input/output — no extra configuration needed.
527. To add clickable buttons: in the Telegram "Send a message" node, set **Reply Markup → Inline Keyboard**, add a keyboard row, add a button with display text (e.g., "Click here to access the course") and set an additional field **URL** to the course link. Clicking the button in Telegram opens that URL directly.
528. This can be made dynamic (instead of hardcoding URLs) by having the AI model output the URL itself using a structured/output parser, rather than typing it manually.
529. You can extend this concept further: send images (AI-generated via Gemini/ChatGPT), audio, or video messages; build multi-level button menus (e.g., "Choose a course category" → "Choose specific course"); or capture user name/email/mobile before responding and store it in Google Sheets.
530. To track bot usage: n8n's **Executions** log shows how many times the workflow ran; you can also explicitly log each conversation into Google Sheets for analytics.
531. n8n has thousands of pre-built workflow **templates** (search "Telegram" to find ~1,200 relevant templates) — useful for ideas before building from scratch.

Part 2: Student project — Customer support/FAQ Telegram bot (by Omkar)

532. Uses a Telegram trigger → **AI agent using OpenRouter's free LLM** (specifically a free "GPT-OSS 120B" style model) to classify the type of customer question, since OpenRouter offers many free models and helps avoid token/rate limits.
533. An **Information Extractor** node (not a full AI agent) determines whether the question is a "billing" or "order status" question — more deterministic and uses fewer tokens than a full AI agent, since it just needs to classify, not "think."

534. A **Switch** node routes based on the extracted category to different hardcoded response templates (e.g., billing questions ask for invoice number/date).
535. Each interaction is also logged into a Google Sheet so the support team can analyze what customers are asking about.

Part 3: Student project — Hospital appointment booking workflow (by Omkar)

536. Patient fills out a form (name, service/department needed, date, and time slot).
537. A workflow reads **all existing bookings from a Google Sheet** (acting as the database).
538. A **JavaScript Code node** compares the requested date/time/service against existing bookings to determine if the slot is already taken, returning `true`/`false` for availability plus a list of that day's free time slots (with already-booked slots removed).
539. An **If node** branches: if the slot is available, another JavaScript node builds an email confirmation message, updates/appends the Google Sheet row, and sends a confirmation email via Gmail. If not available, the system emails back the list of remaining open slots for that day/department instead.
540. The whole system runs on the student's **local machine using Docker**, exposed to the internet via **ngrok** (which maps a local host to a public URL) — so it works with zero paid hosting, though the free ngrok URL isn't a custom domain.
541. Known limitations discussed: the "update row" logic only matches on the patient's name column, so the same patient can't separately book two different departments/times without conflicts; the instructor suggested using two or three separate Google Sheets (e.g., one for doctor slot availability) for more flexibility. Reading "all records" every time is also flagged as a future performance bottleneck at scale.
542. Field validation (e.g., checking mobile numbers have the correct number of digits) requires either n8n's built-in validation-style nodes/parsers, or additional coding — plain no-code forms alone don't give full control.

Key takeaways

- An AI agent in n8n needs a chat/text input source (a message field) and a system prompt to define its exact job — without a tightly scoped system prompt, the AI will guess (hallucinate).
- Guardrails = explicit rules in the system prompt that stop the AI from answering when information is missing or out of scope.
- Telegram bots are created via BotFather; a new bot per project needs a new access token/credential.
- Chat ID is what lets a bot maintain a "session" and reply to the correct person.
- Inline keyboard buttons (with URLs) let a Telegram bot deliver clickable links, not just plain text.
- n8n is a general no-code/low-code automation platform that predates the AI wave — many workflows (like the hospital scheduler) need no AI agent at all, just triggers, logic (If/Switch), and code nodes.

- Coding (e.g., JavaScript Code nodes) gives full flexibility that pure no-code nodes can't always match — combining code + no-code + AI is the "beautiful hybrid" approach the instructor recommends.
- Free/local tools like Docker + ngrok let you host and demo n8n workflows without paying for hosting.
- Reading/writing directly to a spreadsheet (like Google Sheets) as a "database" can cause write conflicts under concurrent use — a real database is recommended for production-scale needs.

Demo 37: Telegram Callback Queries — Building Button-Driven Menus in n8n

This session builds on Demo 36 by teaching how a Telegram bot can respond differently depending on whether the trigger came from a typed message or from a button click (a "callback query"), and how to build a multi-level button menu using If and Switch nodes.

What you'll learn

- The difference between a Telegram trigger firing because of a typed **message** vs. a button click, called a **callback query**
- How to use an **If node** to separate "message" flows from "callback query" flows
- How to use a **Switch node** to route multiple possible button clicks to different responses
- How to arrange inline keyboard buttons into multiple rows/columns for better layout
- How n8n's massive template library (9,500+ workflows, 1,200+ Telegram-specific) can be used for ideas

Step-by-step notes

543. Create a new workflow and a new Telegram bot via BotFather (same process as Demo 36), and connect its API token as a new n8n credential.
544. Add a **Telegram Trigger** node and test it by sending "hello" — confirm the trigger fires on a plain message.
545. Add an **If node** right after the trigger. Condition: check whether the incoming **message** field "is not empty."

- If **true** → the trigger fired because of a typed **message**.
- If **false** → the trigger fired because of a **callback query** (i.e., someone clicked an inline button), since callback queries don't populate the message field the same way.

546. On the **true (message)** branch, add a **Telegram → Send a text message** node with a welcome text (e.g., "Welcome to Life Skills with DurgaSoft. Choose your required category.") and attach an **inline keyboard** with buttons for each course category (e.g., "Mindset and Life Control," "Habits, Growth and Success," "Money, Influence and Business Thinking," "Leadership, Execution and Leverage"). Each button has:

- Button text (what the user sees)
- **Callback data** (an internal identifier string, e.g., `mindset_life_control`) attached via "Add Field."

547. Realize that **the Telegram trigger node must be told to also listen for callback queries**, not just messages — this is done by adding the "On new incoming callback query" event to the Telegram Trigger node. Without this, clicking a button does nothing because the flow was only listening for messages.

548. When a user clicks a button, the trigger now fires with an empty "message" field but a populated "callback query data" field, so the If node correctly routes into the **false** branch.
549. Layout tip: buttons added to the "same row" all appear side-by-side, which can truncate long text. Splitting buttons into separate rows (using "Add keyboard row" for each) or arranging 2 buttons per row gives better readability.
550. Since there are multiple possible button clicks (4 categories), a simple If node isn't enough — add a **Switch node** after the If node's false branch to route based on which callback data value was received:

- Each Switch output checks `callback_query.data` **is equal to** a specific value (e.g., `mindset_life_control`, `habits_growth_and_success`).
- Each Switch branch leads to its own **Telegram → Send a text message** node, which replies with the category's specific course list, again using inline keyboard buttons — this time each button's "URL" field points directly to that specific course's webpage (not just callback data), so clicking it opens the course page immediately.

551. This can be extended to any number of categories/courses; nodes can be copy-pasted and their text/callback values updated to avoid rebuilding from scratch.
552. Bugs encountered live: a typo in the bot's username link (missing an underscore) caused the bot link to not resolve — always double-check the exact bot username.
553. Beyond text: Telegram nodes also support sending video, image, and audio messages — not covered in depth here, but noted as future topics.
554. n8n's **template library** was shown (search "Telegram" — roughly 1,223 Telegram-related templates exist among 9,500+ total) as a resource for pre-built automation ideas (e.g., AI video generation, content agents, personal assistants).
555. A student asked whether it's possible to auto-reply to any personal message sent to your own Telegram account (not just a bot) — the instructor confirmed this is possible but wasn't demonstrated in this session (it isn't the "bot" concept, it targets personal account messages).

Key takeaways

- A Telegram trigger can fire for two different reasons: a typed **message**, or a **callback query** (button click) — your workflow must explicitly enable listening for both, and branch logic (If node) to tell them apart.
- **Callback data** is an internal string tag attached to a button, used purely for routing logic — it is not shown to the user.
- Use an **If node** for a simple two-way split (message vs. callback), and a **Switch node** when there are more than two possible routes (e.g., multiple button options).
- Inline keyboard rows control button layout — one row = buttons side by side; multiple rows = vertically stacked groups.
- Buttons can either carry callback data (to trigger more n8n logic) or a direct URL (to open a link immediately).

- This "menu-driven" bot pattern (welcome message → category buttons → sub-menu buttons → course links) is a realistic, reusable pattern for building interactive Telegram bots without needing users to type free-text every time.

Demo 38: Prompt Chaining in n8n — Multi-Agent Workflows for Better Quality Output

This session revisits the "prompt chaining" concept from earlier prompt-engineering lessons and shows, step by step, how to implement it as a real n8n workflow using multiple chained AI agents to generate a full personalized study package (explanation, notes, quiz, and study plan) from a single topic request, then email it to the student.

What you'll learn

- Why asking one single AI agent to do many different things at once produces lower-quality, less detailed, more hallucination-prone output
- What **prompt chaining** means: breaking a big task into smaller subtasks, each handled by its own AI agent/prompt, where one agent's output becomes the next agent's input
- How to build a 4-agent chained workflow in n8n: Topic Explainer → Notes Generator → Quiz Creator → Feedback/Study Plan Generator
- How to pass the output of earlier nodes as input to later nodes via expressions
- The six main advantages of prompt chaining (quality, debuggability, reusability, control, scalability, real-world workflow matching, reduced hallucination)
- How multiple AI Agent nodes can share a single Chat Model node instead of needing a separate model connection for each agent
- Practical concerns: token cost, Google Sheets write conflicts under concurrent requests, and why "separate workflow instances per request" removes the need for a request queue

Step-by-step notes

556. **The problem with a single big prompt:** If you ask one AI agent for an explanation, notes, a quiz, a learning plan, a PPT, and an Instagram reel script all in one go, the AI can't give full-fledged quality on all of them — it tends to hallucinate/guess and give only surface-level answers.

557. **The fix — prompt chaining:** Instead of one AI agent doing everything, create one AI agent per subtask. The output of AI Agent 1 becomes the input of AI Agent 2, whose output becomes the input of AI Agent 3, and so on.

558. Build a **Form Trigger** ("AI Material Request Form") collecting:

- Student name (required text field)
- Email (required email field)
- Topic (required text field)
- Level: a dropdown with options Beginner, Intermediate, Advanced

559. **AI Agent 1 — "Topic Simplifier / Explanation node":** Connect an OpenAI (or OpenRouter/other) Chat Model. Set the agent's input ("Define below") to use the form's **topic** and **level** fields. System prompt (paraphrased from the transcript): "You are a friendly Indian teacher. Explain the given topic

in very simple English, like teaching a beginner. Use daily life examples, avoid technical jargon. Output should have: simple meaning, daily life example, why it is useful." Execute to confirm it outputs a plain-English explanation.

560. **AI Agent 2 — "Notes Generator":** Takes the **output of Agent 1** as its input message. System prompt: "You are a student study-notes generator. Convert the explanation into clean student notes — short points, simple English, suitable for revision, no unnecessary theory. Output should have: key points, important terms, one simple example, quick revision summary."
561. **AI Agent 3 — "Quiz Creator":** Takes the **notes output** as input. System prompt: "You are a quiz creator for students. Create 5 MCQs from the given notes. Each MCQ should have 4 options (A–D), mention the correct answer, and give a small explanation for the correct answer. Questions should test understanding, not memory."
562. **AI Agent 4 — "Feedback / Study Plan Generator":** Needs the student's name, topic, the explanation, the notes, AND the quiz output all combined into its user prompt (all passed in via expressions referencing each prior node's output). System prompt: "You are a learning mentor. Prepare a final learning conclusion for the student: summarize what they learned, give 3 practice tasks, suggest the next topic to learn. Use simple, motivating Indian English."
563. **Send the results by email:** Add a **Gmail → Send a message** node. Combine the student's name, and the outputs of the Explanation, Notes, Quiz, and Feedback/Study Plan nodes into the email body; set subject like "AI Generated Study Material on <topic>."
564. **Log to Google Sheets:** Add a **Google Sheets → Append Row** node in parallel to record student name, email, and topic requested into a "Students Data" sheet — useful to see what topics people are asking about over time.
565. Live test: submitting "Java 8 new features" at intermediate level produced a full emailed package — plain-English explanation covering Lambda expressions, functional interfaces, Stream API, default methods, method references, Optional class, new Date/Time API, etc.; clean bullet-point notes; 5 MCQs with answers/explanations; and a personalized study plan with practice tasks and a suggested next topic.
566. Published the workflow live and had a few students test it with their own topics (e.g., Python, RAG concept) — all received correctly structured, topic-specific emails.
567. **Six advantages of prompt chaining discussed:**

- Better output quality ("one big prompt = confusion; multiple small prompts = clarity")
- Easier to debug (if the notes are wrong, you know exactly which agent/prompt to fix)
- Reusability (the same "quiz generator" agent could be reused in a different workflow)
- More control over output (want more detailed notes? Only edit the Notes agent's prompt)
- Scalability (need a new capability, e.g. PPT or video script generation? Just add one more agent)
- Matches real-world, step-by-step processes (like a hospital visit: reception → doctor → lab → billing)
- Reduces AI hallucination (a smaller, focused task gives the AI less room to guess)

568. **Chat model reuse:** All system/user prompt configuration lives inside the AI Agent node, not the Chat Model node (which typically only holds the API key/model selection) — so the same Chat

Model node connection can technically be reused by multiple AI Agent nodes rather than creating a new model connection each time.

569. **Operational notes/cautions raised by students and instructor:**

- Every incoming form submission runs its own separate instance of the workflow — there is no need for a manual queue; n8n handles concurrent executions natively.
- The only real risk of conflict is **simultaneous writes to the same Google Sheet row** — the instructor recommends a proper database over a spreadsheet for anything beyond casual/demo use, and one student suggested using a "Loop" node to force sequential (one-by-one) sheet writes to avoid write collisions, at the cost of some speed.
- Since prompt chaining in n8n is implemented using multiple **AI Agent nodes** chained together (there's no separate "prompt" node), the instructor called this "prompt chaining" implemented as "chaining of AI agents" — same underlying concept, different terminology in n8n.

Key takeaways

- Prompt chaining = break a large task into smaller subtasks, each with its own AI agent and focused system prompt, where each agent's output feeds the next agent's input.
- In n8n, prompt chaining is implemented as a chain of **AI Agent nodes**, not literal "prompts" — each node represents one link in the chain.
- Chaining gives higher-quality, more specific, less hallucinated output at the cost of more tokens/LLM calls.
- All chat model configuration for a chain can point to the same underlying Chat Model node/credentials — you don't need a new model connection per agent.
- A well-designed chained workflow mirrors a real-world, multi-step process (explanation → notes → quiz → feedback), making it easier to reason about, debug, and extend.
- For production use, avoid depending on spreadsheets as a write-heavy "database" — use one for logging/reference but expect conflicts under concurrent load; a real database is safer.

Demo 39: Routing in n8n — Auto-Labeling Gmail with a Text Classifier

This session introduces the **routing** pattern in n8n: automatically classifying incoming Gmail messages (e.g., as Java, Python, or AI inquiries) using a specialized **Text Classifier** node, then automatically applying Gmail labels and forwarding important messages (e.g., AI inquiries) to Telegram — replacing what would otherwise be manual email triage.

What you'll learn

- What "routing" means in a workflow context, and how If/Switch nodes are themselves simple routing mechanisms
- What a **Text Classifier** node is, how it differs from a full AI Agent, and why it's more efficient for pure classification tasks
- How to create and color-code Gmail labels, and auto-apply them to incoming mail based on AI classification
- How to forward a summarized version of important emails to a Telegram bot
- Why reusing the same Chat Model connection across multiple nodes doesn't reduce token usage (each request is still a separate call with its own context)
- Why "simple memory" is needed for an AI agent to remember earlier turns in a conversation, and why, without it, every request is treated independently
- A student's proposed improvement: storing category/label metadata in a Google Sheet instead of hardcoding categories directly into the classifier, for easier scaling to many categories

Step-by-step notes

570. **The concept of routing, explained with an analogy:** A call center routes a call to the right department based on what the caller needs (press 1 for Java, 2 for Python, 3 for AI). In email, the same idea applies: incoming mail can be automatically labeled/routed based on its content — e.g., colored Gmail labels for Java (blue), Python (green), AI (red) — instead of manually reading and labeling every email.

571. **Building the "Routing Flow" workflow:**

- Add a **Gmail Trigger** node ("on message received"), polling every minute.
- Manually create three Gmail labels ahead of time (this step is **not automatable** — labels must be created in Gmail directly): "Java" (blue), "Python" (green), "AI" (red).

572. Add a **Text Classifier** node right after the Gmail trigger. This is described as "a specialized, more limited version of an AI agent" — a normal AI agent can take many kinds of actions (send messages, make decisions, generate content), but the Text Classifier is purpose-built only for classification, which makes it simpler and more efficient for this task.

- Requires a Chat Model connection (e.g., OpenAI Chat Model) just like a normal AI agent.
- **Text to classify:** set to the email's **snippet** field (Gmail's short body preview) from the trigger node.

- **Categories:** add one category per label (e.g., "Java," "Python," "AI"). For each category, write a **category description** (functioning like a mini system prompt) explaining exactly what should/shouldn't match it — e.g., for Java: "Classify as Java if the email is about Java programming, Java training, or Java development — includes mentions of Java, Core Java, Advanced Java, Spring, Spring Boot, Hibernate, Java syllabus/fees questions. Do NOT classify as Java if the email is mainly about other technologies like Python or AI."
- Enable the **"Enable auto-fixing"** option under Add Option — this fixed a "failed to parse" JSON error encountered during testing (the classifier internally uses LangChain output parsing, which can occasionally produce malformed output without auto-fix).

573. After the classifier, each category branch connects to its own **Gmail → Add label to message** node:

- Requires the email's **message ID** (from the Gmail trigger) and the specific label to apply (e.g., "Java").
- Nodes can be duplicated/copy-pasted and just have their label field changed for Python and AI branches.

574. Test: sending real emails with "Java," "Python," or "agentic AI" in the body correctly triggers the matching label to be applied automatically in Gmail (confirmed live with multiple students sending test emails).

575. For the **AI category specifically**, an additional **AI Agent** node is attached after the "AI" label node — this agent's job is to summarize the email, and its summary is then sent to a **Telegram node**, forwarding a notification about important AI-related inquiries to a Telegram bot/chat, so someone monitoring Telegram gets notified without checking email.

576. Java and Python branches in this initial version only added labels — no further action (summary/forward) was implemented for them, though the instructor noted this could easily be extended similarly.

577. **Handling emails that mention multiple topics (e.g., both Java and AI):** By default, the classifier is forced to pick a single best-matching category, controlled entirely by the category descriptions (system-prompt-like text). To properly support a mail matching two topics at once, the instructor added a **new combined category**, "Java and AI," with its own description ("classify as Java plus AI if the email contains both Java-related and AI-related content"), and a corresponding combined label. This is more reliable than trying to auto-apply multiple separate labels, since the classifier's description explicitly needs to know about the combined case.

578. **Q&A / clarifications covered:**

- It IS technically possible for a classifier to output multiple matching labels for one email, but the instructor recommends being explicit/deliberate about which combined categories you actually want, rather than relying on it happening implicitly.
- Reusing the same Chat Model connection across multiple AI nodes (Text Classifier + AI Agent) does **not** reduce token usage — each node execution is still an independent LLM request; it just saves you

from re-entering the same API key/model configuration repeatedly. It also means shared "context" only applies if you explicitly add a memory component.

- Without a "Simple Memory" node, every workflow execution/request is treated as a completely separate, stateless call — the AI has no memory of a previous message unless memory is explicitly configured (referenced back to an earlier lesson where asking "who is the US president?" then "what is his age?" failed without memory, because "his" had no context).

579. **Student proposal (Salim) for a more scalable design:** Rather than hardcoding N separate Gmail "Add Label" nodes for N categories, store all category metadata (course/category name, description, and the corresponding Gmail label ID) in a **Google Sheet**. The classifier or a filter/lookup step could then read this sheet at runtime to decide which label to apply, making it easy to add new categories (e.g., scaling to 50 subjects) by just editing the spreadsheet instead of rebuilding the workflow. The instructor found this "wonderful" as an idea but asked the student to prototype and demo it in a following session.

Key takeaways

- **Routing** = classifying an input and directing the workflow down a different path based on that classification; If/Switch nodes are basic routing tools, and the **Text Classifier** node is a specialized routing tool designed specifically for content classification.
- Text Classifier nodes work like a lightweight AI agent — same need for a Chat Model connection, but scoped only to output a category, guided by per-category descriptions that function like mini system prompts.
- Gmail labels and their colors must be created manually in Gmail first; automation can only apply pre-existing labels, not invent new label names/colors on the fly.
- To handle emails matching more than one category well, define explicit combined categories rather than expecting automatic multi-labeling to "just work."
- Reusing one Chat Model connection across nodes saves configuration effort, not tokens — every node execution is still a separate LLM call.
- "Memory" (e.g., Simple Memory) is required for an AI agent to remember prior turns in a conversation — without it, each execution is stateless.
- Storing classification metadata (categories, descriptions, label IDs) in a Google Sheet instead of hardcoding it directly in nodes is a promising pattern for scaling to many categories without constantly rebuilding the workflow.

Demo 40: Extending Routing — Auto-Reply and Telegram Forwarding by Category, Plus a Student's Scalable Metadata-Driven Design

This session continues the Gmail routing workflow from Demo 39, adding automatic email replies for AI-related inquiries and automatic Telegram notifications for Java-related inquiries — both generated by dedicated AI agents chained after the routing/classification step. A student (Salim) then demos an alternative, more scalable design that stores category metadata in Google Sheets instead of hardcoding it in nodes.

What you'll learn

- How to add an auto-reply email step after a routing/classification node, using an AI agent to draft the reply
- How to reference an AI agent's structured output (`$json.output`) in a later node
- How to get a Telegram **chat ID** to send messages to a bot when there is no Telegram Trigger node in the workflow (by messaging a helper "user info" bot)
- How to forward a category-specific email summary to Telegram, similar to Demo 39 but shown end-to-end with troubleshooting
- A student's alternative design: storing category name, description, and Gmail label ID as rows in a Google Sheet, read at runtime, to avoid hardcoding one branch per category
- The idea of using **sticky notes** in n8n to document what each node does, and disabling unused nodes rather than deleting them

Step-by-step notes

Recap: Prompt chaining (output of one AI agent feeds into the next) and routing (Text Classifier + Gmail labeling from Demo 39) were briefly recapped as two distinct, reusable workflow "styles."

580. **Adding an auto-reply for AI-category emails:**

- After the AI-labeled branch, add a new **AI Agent** node (reusing the same Chat Model connection used elsewhere in the workflow — confirmed this reuse is fine since all prompt config lives in the AI Agent node, not the Chat Model node).
- Set the agent's input to the Gmail **snippet** (the email body preview).
- System prompt (paraphrased): "You are a friendly and professional support member from DurgaSoft. Reply to the student's inquiry about AI in very simple English, with motivation about AI's importance, and mention our support team will contact them soon."
- Add a **Gmail → Send a message** node after this agent: To = the sender's email (from the trigger), Subject = "Reply," Body/Text = the AI agent's output — referenced using the expression **`{{$json.output}}`** (this is the field name where AI Agent nodes store their generated text).

- Troubleshooting encountered live: a "cannot publish — invalid required parameters" error occurred until a stray placeholder ("Hello") was replaced with the proper `$json.output` expression, and until the field type was correctly set to "Expression" rather than plain text.
- Once fixed, sending a test email containing "AI inquiry" correctly triggered a full auto-reply email back to the sender.

581. **Adding a Telegram forward for Java-category emails:**

- After the Java label branch, add another **AI Agent** node using the Gmail snippet as input.
- System prompt: summarize the incoming Java-related email and prepare a message for Telegram that includes who it's from (sender email) and a summary of what they're asking, in the format "A new mail came from [sender] about Java. Summary: [details]."
- Add a **Telegram → Send a text message** node after this agent.
- **Getting a Telegram chat ID without a Telegram Trigger node:** since this workflow has no Telegram Trigger (it starts from Gmail), the chat ID can't be pulled from a trigger automatically. Instead, the instructor messaged a pre-existing helper bot (referred to as a "user info" / "get user info" bot) inside Telegram — sending it `/start` reveals your numeric chat/user ID directly in the chat, which can then be hardcoded into the "Chat ID" field of the Send Message node.
- Set the message Text field to the AI agent's output field (`$json.output`), same pattern as the email reply.
- Published and tested live: sending an email with "Java" in the subject and body correctly triggered the label to be applied AND a summarized Telegram notification to be sent (e.g., "A new mail came from DurgaSoft about Java. Summary: student request for full-stack Java course details... Action: please contact the student soon and share the Java course details"). Some test requests failed intermittently during the live demo, which the instructor attributed to minor formatting/parsing issues rather than a fundamental design flaw.

582. **Student demo — Salim's alternative, metadata-driven routing design:**

- Instead of one Gmail "Add Label" node per category (which doesn't scale well to, say, 50 course categories), Salim built a Google Sheet holding: **category name, category description (the same text that would otherwise go in a Text Classifier's category description), and the corresponding Gmail label ID**.
- His workflow uses an AI agent with a **structured output parser** to first standardize/clean up inconsistent category names in incoming requests (e.g., correcting typos or short forms like "DS" for "Data Science," or "Java," "Pyhton" style misspellings) before matching against the sheet.
- A **JavaScript/Code node acts as a filter**, reading the Google Sheet's metadata and matching the (cleaned) requested category against it, replacing the need for a dedicated Text Classifier node per category.
- Cited advantages of this approach, agreed on by the group:
- **Scalability** — adding a new course/category is a one-time edit to the spreadsheet, no workflow/code changes needed.

- **Maintainability/readability** — non-technical staff can update categories without touching the workflow.
- Fewer duplicate nodes to maintain if there are many categories (avoids "one classifier per category" bloat).
- Trade-offs discussed: this approach still needs an LLM/AI step specifically to normalize spelling/naming inconsistencies (a JavaScript-only filter can't reliably catch typos or abbreviation variants); reading the Google Sheet on every single request adds some overhead compared to fully hardcoded logic; and Google Sheets' filter tool doesn't natively support multi-value/list-based filtering well, which required extra handling in the code node.
- Also demonstrated: n8n's **Sticky Note** feature — right-click on the canvas (or use the "+" toolbar) to add a sticky note, which can be placed behind/near a node to document what that node does. The instructor emphasized this is a **real-world standard practice**: every workflow delivered to a client should have this kind of documentation so others can understand its purpose.
- Also demonstrated: **disabling** a node (instead of deleting it) when it's no longer needed in the current design — keeps it available for future re-enabling without losing the configuration.

Key takeaways

- `$json.output` (or similar) is the standard way to reference an AI Agent node's generated text output in a later node's expression — make sure to set the field to "Expression" mode, not plain text.
- When a workflow has no dedicated trigger node for a channel (e.g., sending to Telegram from a Gmail-triggered workflow), you can still obtain required IDs (like a Telegram chat ID) manually via a helper bot, then hardcode them.
- Routing workflows can be extended per category with their own downstream actions (auto-reply email for one category, Telegram alert for another) — each branch is independent and can do something completely different.
- For workflows with many categories, storing category metadata (name, description, label ID) in an external sheet and reading it at runtime is a more scalable alternative to hardcoding one node-branch per category — at the cost of needing an AI/LLM step to normalize messy input text, and a small runtime lookup overhead.
- Documentation matters in real deliverables: use **sticky notes** to explain what each node/section of a workflow does, especially before handing a workflow off to a client or teammate.
- Disable (rather than delete) nodes you're not currently using, so configurations aren't lost if you need them again later.
- The instructor closed by previewing the next topic: **parallelism** in n8n workflows (running steps simultaneously rather than one after another), to be covered in a following session.

Demo 41: Parallelism in n8n — Running Multiple AI Agents at the Same Time

This session introduces the **parallelism** design pattern for n8n workflows and builds a live "career guidance" workflow where three AI agents (roadmap, skills, interview) run at once and their answers are combined and emailed to the user.

What you'll learn

- The difference between prompt chaining (dependent, sequential tasks) and parallelism (independent, simultaneous tasks)
- How n8n visually shows nodes executing "one after another" even though the underlying API calls happen in parallel
- The difference between the **Merge** node and the **Aggregate** node
- How to build a multi-agent workflow: a form trigger feeding three specialised AI agents, then merge → aggregate → a final summarising AI agent → Gmail
- Why using the same chat model/API key for multiple agents does not prevent parallel execution

Step-by-step notes

583. **Recap of earlier patterns.** The instructor reminds students that they already covered *prompt chaining* (output of one AI agent becomes the input of the next, used when tasks are dependent) and *routing* (using a Text Classifier node to send requests down different branches).

584. **What is parallelism?** If tasks have no dependency on each other, they don't need to run one after another — they can run simultaneously. Example: a student asks "How do I get a job in Java?" This really has three independent sub-questions: (a) what's the roadmap, (b) what skills are needed, (c) how to prepare for interviews. Instead of one "teacher" answering all three in sequence, you can have three "teachers" (agents) answer at the same time, giving a faster overall response.

585. **Building the workflow (career guidance workflow):**

- Add a **Form Trigger** node named "Student request form" with fields: Name (text, required), Email (email, required), Question (text, required).
- Add three **AI Agent** nodes side by side, all connected to the same Form Trigger output, each with a different name and system prompt:
- **Road map agent** — "You are a DurgaSoft road map advisor. Based on the student question, give a simple learning road map. Use simple English. Keep it practical."
- **Skills agent** — "You are a DurgaSoft skills advisor. Based on the student question, list important skills required. Use simple bullet points."
- **Interview agent** — "You are a DurgaSoft interview preparation advisor. Based on the student question, suggest interview preparation topics. Keep it short and beginner friendly."
- Each agent needs its own **OpenAI Chat Model** node attached (the same credentials/model can be reused for all three — this does NOT prevent parallel execution).

- Connect all three agent outputs into a **Merge** node configured for 3 input streams.
- Connect the Merge output into an **Aggregate** node.
- Connect the Aggregate output into a final **AI agent** whose job is to combine everything: "You are a DurgaSoft student mentor. Combine the road map, skills, and interview preparation outputs into one clear final answer. Use simple English. End with a motivation line."
- Connect that to a **Gmail (Send a message)** node that emails the final combined answer to the student.

586. **Understanding Merge vs Aggregate (important distinction):**

- **Merge node** (mode = *Append*) just collects the outputs of multiple agents and places them side by side at one place — after merge you still have **3 separate JSON items** (a "messy list").
- **Aggregate node** takes those 3 separate items and converts them into **1 single structured JSON object** — this is what makes it easy for the final AI agent to read a clean, combined input.
- Without the Aggregate node, the final agent (and everything downstream) would receive 3 items, and downstream nodes would execute once per item — resulting in 3 separate emails instead of 1.

587. **Why the workflow still looks like it runs "node by node".** A student asked whether using the same LLM/chat model for all three agents defeats parallelism. The instructor clarifies: n8n's canvas UI always visualises execution node-by-node for display purposes, but internally each AI agent makes its own **separate API call**, and those calls happen simultaneously. The visualisation is not a true picture of the internal execution order — it's "just a visualization feeling."

588. **Live demo.** The instructor submits the form asking "I want to get a job on N8N automation. What is the roadmap? What is the skill agent? And what is the interview agent?" The workflow returns a full roadmap (8-week plan, core skills, APIs/webhooks, JavaScript basics, common integrations like Google Sheets/Slack/Gmail/Airtable/Stripe/Shopify, debugging, deployment, portfolio building) combined into a single well-organized email.

589. **Workflow published** as a live production form so students could test it themselves by submitting different career questions.

Key takeaways

- Use **prompt chaining** when tasks are dependent (output of one feeds the next).
- Use **parallelism** when tasks are independent — this reduces response time.
- n8n's visual canvas always shows sequential node execution, but the underlying API calls for independent branches run in parallel.
- **Merge** = collect multiple outputs into one place (still separate items). **Aggregate** = combine those items into one clean, single object.
- Advantages of parallelism: faster response, "multi-expert thinking," independent processing, easy to scale (just add another independent agent), better final output quality.
- The only requirement for using parallelism is that the tasks must be genuinely independent of each other.

Demo 42: Merge vs Aggregate Deep Dive, Plus Two Real Student Projects

This session continues the parallelism discussion with a hands-on comparison of Merge's "Append" mode vs "Combine" mode against using a separate Aggregate node, then two students demo their own n8n projects: a Google-Maps-based local business finder and a dynamic doctor-appointment booking system.

What you'll learn

- The practical difference between Merge (Append mode) and Merge (Combine mode) vs a dedicated Aggregate node
- Why skipping the Aggregate node causes duplicate emails/executions
- How to force AI agents to output in a strict, fixed format so Merge (Combine) can be used safely instead of Aggregate
- A real example of using an HTTP Request node with a third-party search API (Serper) to fetch live Google Maps data
- A real example of building a dynamic, symptom-based doctor appointment booking app using n8n custom HTML forms and Google Sheets

Step-by-step notes

590. **Recap of yesterday's career guidance workflow** (roadmap/skills/interview agents → merge → aggregate → final agent → Gmail).

591. **Experiment 1: Removing the Aggregate node while keeping Merge in "Append" mode.**

- The instructor deactivates the Aggregate node so the Merge node's output (still 3 separate items) goes straight into the final AI agent.
- Result: because there are 3 items, every downstream node (AI agent, Gmail) executes **once per item** — the student receives **3 separate emails** instead of one combined email. This demonstrates that Merge (Append) alone does not consolidate data into a single object.

592. **Experiment 2: Changing Merge mode from "Append" to "Combine."**

- Under Merge node configuration, change **Combine by** to "Combine items based on their order" (position-based), set number of inputs to 3.
- This time the Merge node itself produces **1 single item**, so only 1 email is sent.
- However, the output is a bit "messy" — fields/duplicates aren't as cleanly labeled as when using a dedicated Aggregate node.

593. **Fixing the messiness with formatted AI outputs.** To make Merge (Combine) produce clean results, the instructor updates each of the three agents' system prompts to require an **exact output format** (e.g., "Return output in this exact format: Skills required: - first - second - third"), and updates the final agent's system prompt to expect that exact structure ("Use this format exactly: Hi

[student name], thank you for your question... Learning road map: ... Important skills: ... Interview preparation: ... Final suggestion/motivation ... Regards, DurgaSoft"). With this discipline, Merge (Combine) alone can replace the Aggregate node.

594. **Trade-off discussion / final recommendation:**

- **With Aggregator:** clean structured output, final agent gets one clear input, easy to label outputs (roadmap/skills/interview), best for real projects, easy to debug and scale. Downsides: one extra node, slightly more setup, slightly more execution time.
- **Without Aggregator (Merge Combine only):** faster, fewer nodes, simple for demos, easy to build. Downsides: less control, output can get confusing if field names aren't clear, harder to scale as more agents are added, depends heavily on every agent formatting its output correctly.
- **Recommendation:** for real production automations, use Merge (Append) + Aggregator (the "industry standard" approach). For quick classroom/demo purposes, formatted AI outputs + Merge (Combine) is an acceptable shortcut.
- The instructor notes this same merge/aggregate concept applies later when working with **LangChain / LangGraph** in code — n8n is really just visualizing what those frameworks do internally.

595. **How the final agent receives combined data:** A student asks how the system prompt "knows" what job to do vs where the data comes from. Answer: the **system message** tells the agent its role/job ("you are a mentor, combine these three things..."), while the **user message** is where you actually paste in the placeholder data (name, question, roadmap output, skills output, interview output) — the system prompt references that this information will be supplied via the user message.

596. **Student demo 1 — Local service finder using Google Maps data (built by a student, "Salim/Shashikumar" style project):**

- A simple **Webhook** node receives a POST request from a custom HTML form (hosted separately, not built with n8n's form node) where the user enters a service type (e.g., "school," "dentist," "cinema hall") and a location (e.g., "Hyderabad") plus how many results to fetch.
- An **HTTP Request** node calls the **Serper API** (a third-party API for getting real-time Google Maps search results) with the location and service query.
- A **Code node** (JavaScript, written with help of ChatGPT rather than hand-typed) formats/cleans the raw results (address, phone number, open/closed status, website, rating) and sorts them by rating (highest first).
- The final node responds to the Webhook, returning the formatted list directly to the web page (not via email) — the student explained his wife suggested showing results directly on the page instead of checking email.
- Instructor explains **Serper API**: it brings real-time/live web data (AI agents alone only know their trained data, not current information). Serper gives 1,000 free API calls/month, then charges. A similar tool mentioned for research work is **Tavily**, which also has a free tier (~500–1,000 free hits) before charging.

597. **Student demo 2 — Doctor appointment booking app (built by "Murali"):**

- Built using n8n with a **custom HTML form** (not the built-in n8n form node) for a more flexible, dynamic UI.
- Patient enters symptoms/problem, name, email, mobile number — and the **AI automatically identifies which doctor category/specialization** is relevant based on the symptoms (no manual category selection).
- The appointment slots shown are **dynamic** — pulled live from a Google Sheet tracking each doctor's available slots per day, and slots update in real time as bookings are made (e.g., if a doctor's slots fill up, they no longer show as available).
- The system prevents a user from double-booking the same doctor at the same time, but does allow booking a different doctor or a different time.
- On successful booking, it shows a confirmation message on-screen and also sends a confirmation email in parallel.
- Rescheduling isn't supported yet (first version only), but the student noted it could be extended later.
- The instructor praised the project for demonstrating that a real production-quality app can be built with n8n plus a custom HTML front end, multiple Google Sheets, and AI-driven logic.

Key takeaways

- Merge (Append) alone produces multiple items → downstream nodes re-execute per item → multiple/duplicate outputs.
- Merge (Combine) can act like a lightweight aggregator (produces 1 item) but requires very disciplined, fixed output formatting from every upstream agent to avoid messy results.
- A dedicated **Aggregate** node is the more reliable, industry-recommended way to consolidate multiple AI agent outputs into one clean object, at the cost of one extra node and a little more time/setup.
- Third-party APIs like **Serper** or **Tavily** let AI agents/workflows fetch live, real-time web data since LLMs alone only know their training data.
- n8n workflows can use custom HTML forms (hosted outside of n8n's built-in form node) connected via Webhook for more flexible, professional-looking front ends.

Demo 43: Introducing the Evaluate–Optimize (Evolve-Optimizer) Framework and Looping in n8n

After a long personal/motivational segment, the instructor introduces a fourth n8n design pattern: the **evaluate-and-optimize** (called "evolve optimizer" in the session) framework, which adds a self-checking loop around an AI agent's answer before it's delivered to the user. This is the first time looping is used in the course's n8n workflows.

What you'll learn

- The evaluate-optimize (self-reflection) design pattern and why it improves output quality
- How to build a loop in n8n using an IF node that branches back to an earlier node
- The three roles/agents needed: an **answer generator**, an **evaluator**, and an **optimizer**
- How to use a **Structured Output Parser** to get consistent JSON (score, status, feedback) from the evaluator agent
- The danger of infinite loops and why a retry counter is necessary (introduced conceptually here, implemented in later sessions)

Step-by-step notes

598. **Recap of the three patterns covered so far:** prompt chaining (dependent tasks, sequential), routing (Text Classifier node for branching), and parallelism (independent tasks executed simultaneously).

599. **Introducing the evaluate-and-optimize pattern.** Normally, an AI agent generates an answer and that's used directly (sent by email, shown on screen, etc.) without checking whether it's actually good. The new idea: after the AI agent generates a response, add a step that **evaluates** whether the answer meets requirements. If yes, deliver it. If no, send it to an **optimizer** agent that improves the answer based on feedback, then **loop back** to evaluate again. This repeats until the answer is good enough (or a limit is reached).

- A student compares this to the "self-reflection prompt" technique covered earlier in prompting lessons — the instructor confirms it's the same idea, now implemented as a multi-node workflow with an actual loop.

600. **Why three separate AI agent nodes are needed (not one).** A student asks why not use a single AI agent node with multiple system messages (one for generating, one for evaluating, one for optimizing). The instructor explains each n8n AI Agent node only accepts **one system message**, so each distinct job (generate / evaluate / optimize) needs its own agent node. Reusing the same node repeatedly with different system messages each time would make the workflow logic much harder to manage — separate nodes keep the workflow clear.

601. **Advantages of the evaluate-optimize pattern:**

- Better quality output (answers are rechecked before delivery)

- Fewer errors/hallucinations (rechecking catches mistakes)
- Less manual work (the checking is now automated, not done by a human)
- Continuous improvement (answer can be refined across multiple iterations)

602. **Building the workflow (first walkthrough, high-level):**

- Reuse the "student request form" trigger from earlier sessions.
- **Original AI agent**: generates the first-level answer to the student's question.
- An **Edit Fields** (Set) node stores this output into a variable called `current answer` — this makes it easier to reference the "latest" answer regardless of which node produced it (original agent or optimizer).
- **Evolve/Evaluate agent**: checks the current answer against a checklist and produces a **structured JSON output** (score, status, feedback) using a **Structured Output Parser**.
- An **IF node** checks whether the evaluation is good ("approved")
- If **true**: send the response to the end user (via Gmail, Google Docs, Google Sheets, etc.)
- If **false**: pass the original answer + feedback to an **Optimizer agent**, which regenerates an improved response. That improved response is written back into the `current answer` variable via another Edit Fields node, and the flow loops back to the Evaluate agent again.

603. **Danger of infinite loops.** The instructor points out that if this loop runs unchecked (e.g., 1,000+ times), token costs would spiral — "we have to sell our company to pay the tokens cost." The fix is to introduce a **counter/retry-count variable** with a maximum limit (e.g., stop after 3 optimization attempts and just deliver whatever answer exists at that point), but implementation of the counter is deferred to the next session.

604. **Detailed second build (step-by-step, this time actually executed):**

- New workflow: Form trigger ("student request form": name, email, question) → **AI agent** to generate the first answer, with a system prompt like: "You are a DurgaSoft faculty assistant. Answer the student question in simple Indian English... Give one real-time example. If it's technical, give simple code... Output format: title, simple explanation, real-time example, code, final summary."
- **Edit Fields (Set) node**: creates a variable `current answer` = the AI agent's output.
- **Evolve/Evaluate agent** (renamed from generic "AI agent"), with inputs: the original `question` and the `current answer`, and a system prompt like: "You are a strict DurgaSoft answer evaluator. Check the given answer... Give a score out of 100. Return only valid JSON: {score, status, feedback}. If score ≥ 80, status = approved; if below 80, status = needs improvement. Do not add extra text outside JSON."
- A **Structured Output Parser** node is attached to this agent so `score`, `status`, and `feedback` become clean, directly usable variables (instead of a raw JSON string).
- An **IF node** checks `score >= 85` (the threshold used in this demo).
- **True branch** → Gmail node sends the answer to the student (subject: "Answer for your question").
- **False branch** → **Optimizer agent**, fed with the original question, current answer, and evaluator feedback, with a system prompt like: "You are a senior DurgaSoft faculty optimizer. Improve the

previous answer based on the evaluator feedback. Make explanation simpler, add missing points and a real-time DurgaSoft-style example, simple code, improve structure... Output: title, improved explanation, real-time example, code example, student takeaway." Its output is written back into `current answer` and the flow loops back to the Evaluate agent.

605. **Live test.** Testing with the question "Explain N8N": the first-level answer is generated, evaluated as not good enough, optimized, re-evaluated, optimized again, etc. — visibly looping several times in front of the class, confirming the loop mechanism works, but also confirming that **without a retry/counter limit this can run indefinitely** (the instructor manually stops it after several iterations as a safety demonstration).

Key takeaways

- The **evaluate-and-optimize** (self-reflection style) pattern adds a quality-control loop: generate → evaluate → (if not good) optimize → evaluate again → ... → deliver.
- Each distinct role (generator, evaluator, optimizer) needs its own AI Agent node because each n8n Agent node only supports one system message.
- A **Structured Output Parser** is essential for getting consistent, machine-readable fields (score/status/feedback) out of the evaluator agent instead of free-form text.
- Looping without a stopping condition risks an **infinite loop** and runaway token costs — a retry counter with a maximum limit is required in production (to be implemented in the next sessions).
- This pattern is one of four core design patterns covered in the course so far: prompt chaining, routing, parallelism, and evaluate-optimize.

Demo 44: Live Debugging Session — Fixing the Retry-Counter Bug in the Evaluate-Optimize Loop

This entire session is a live, unresolved debugging exercise. The instructor tries to add a retry-counter (to stop the evaluate-optimize loop after 3 attempts) but the counter variable keeps resetting to zero instead of incrementing, causing an infinite loop. The class analyzes the bug together but does not fully solve it in this session — it's left as homework, with the fix delivered in the next session.

What you'll learn

- How to attempt adding a retry-limit mechanism to a looping n8n workflow using Edit Fields (Set) nodes and IF nodes
- A common n8n data-flow bug: referencing a "stale" node's output instead of the most recently updated value
- How to simplify two chained IF conditions into a single IF node using AND/OR logic
- Why understanding **which node's output an expression is reading from** matters enormously in loop-based n8n flows
- General debugging practice: isolate/execute nodes one at a time, use `console.log`-style reasoning to trace variable values step by step

Step-by-step notes

606. **Context.** Continuing from the previous session's evaluate-optimize loop, the instructor wants to add a safeguard: only allow the optimizer to retry up to 3 times; after that, send whatever answer exists even if the score is still low. He spent "an hour and a half" before class trying to implement this and is still stuck, so he walks the problem through live with the class.

607. **Rebuilding step-by-step (new workflow "optimize flow two"):**

- Form trigger ("student request form": name, email, question).
- An **Edit Fields** node initializes two variables: `retry count` = 0 (number) and `current answer` = "" (empty string).
- **Original AI agent** generates the first answer (system prompt reused from the previous session: DurgaSoft faculty assistant, beginner-friendly explanation with real example/code).
- **Evolve/Evaluate agent**, using a **Structured Output Parser**, outputs `score`, `status`, `feedback` in strict JSON (rule: score ≥ 90 → approved, else needs improvement).
- **IF node**: `score >= 93` (threshold changed a few times during the demo, e.g., 93, then 95, then 97) → true branch sends the Gmail message; false branch should check the retry count and continue to the optimizer.

608. **The core bug.** The instructor placed a second **Edit Fields node** right after the initial "retry count = 0" node, intending to reassign `retry count = retry count + 1` on each optimizer pass. But when he referenced "retry count" downstream, the expression was always pulling from the **original**

Edit Fields node (which is hardcoded to 0), not from the updated Edit Fields node further down the loop — so every "increment" attempt started again from 0 instead of building on the previous value. He demonstrates this live: the retry count variable stays at 0 no matter how many loop iterations run, and the workflow enters what looks like it could become an infinite loop.

609. **A related smaller bug identified by a student.** When first attempting `retry count + 1`, the "+1" was mistakenly placed **outside** the field-value expression, causing string concatenation ("01") instead of numeric addition — turning `0 + 1` into the text "01" rather than the number 1. The fix discussed live: keep the +1 **inside** the same field value expression so it's treated as arithmetic, not string concatenation.

610. **Simplifying two IF nodes into one.** A student points out that the workflow currently uses two separate IF nodes (one to check the score, another to check whether the retry count has hit the limit) — these can be combined into a **single IF node** using an OR condition: `score >= threshold` OR `retry count >= 3` → if either is true, send the email; otherwise increment the retry count and go to the optimizer. The instructor and class implement this simplification.

611. **Optimizer agent inputs.** The optimizer agent needs: the original `question`, the `current answer`, the evaluator's `score`, and the evaluator's `feedback` — all passed in via the user message, with a system prompt like: "You are a senior DurgaSoft faculty optimizer. Improve the previous answer based on the evaluation framework/feedback..."

612. **Repeated live test failures.** Multiple test runs show the retry count staying at 0 or 1 no matter how many loop iterations occur, causing the workflow to keep looping (observed running through 3rd, 4th+ iterations without stopping as intended). The instructor and several students (using terms like "scope issue," "which edit field are we pulling from," "it's like variable declaration vs. updated value") discuss possible causes but do not land on a definitive fix during the session.

613. **Outcome.** The instructor acknowledges the bug is a small configuration/reference issue rather than a logic flaw, assigns it as a challenge/homework for students to try fixing on their own, and promises to reveal (or confirm) the fix in the next session. He also announces that after this loop-counter issue is resolved, the course will move on to either **RAG** or **MCP** next.

Key takeaways

- In n8n loop-based workflows, expressions must reference the correct/most recent node's output — pointing at an old Edit Fields node instead of the latest one will silently keep resetting a "counter" back to its initial value.
- When incrementing a numeric field in an expression, keep the +1 inside the field's value expression to avoid accidental string concatenation.
- Two IF nodes that check related conditions can usually be merged into one IF node using AND/OR logic, simplifying the workflow.
- Always test/execute nodes one at a time when debugging a loop, and check the actual value of each variable at each step rather than assuming it updated correctly.
- This bug (a "stale reference" to an old Set/Edit Fields node) is a common trap when building loops in low-code tools like n8n.

Demo 45: Fixing the Retry-Counter Bug, Recapping the 4 n8n Patterns, and Introducing MCP

This session opens by fixing yesterday's retry-counter bug using a conditional (ternary-style) expression, recaps all four n8n design patterns covered in the course, and then introduces a brand-new topic: **MCP (Model Context Protocol)** — what it is, why it exists, and how to enable it so ChatGPT can directly control an n8n workflow.

What you'll learn

- The actual fix for the retry-counter bug from Demo 44 (a conditional/ternary expression)
- A recap of all four n8n workflow design patterns: prompt chaining, routing, parallelism, evaluate-optimize
- What MCP (Model Context Protocol) is and why it was created
- How MCP relates to earlier concepts like REST APIs, HTTP, JSON/XML
- How to enable MCP access in n8n (platform level and workflow level)
- How to connect ChatGPT (via Developer Mode / Create App / Custom Connector) to an n8n workflow using MCP, and trigger/run that workflow directly from ChatGPT

Step-by-step notes

Part 1: Fixing the retry-counter bug

614. **Diagnosing the real cause.** The instructor explains the mistake clearly this time: the workflow always pulled the `retry count` value from the **first** Edit Fields node (which is hardcoded to 0 as the initial value), even after a later Edit Fields node had updated it. So every time the loop went back around, the value was being **reassigned to 0** instead of continuing from the previously incremented value.
615. **The fix — a conditional (ternary) expression.** Instead of hardcoding the reference to the first Edit Fields node, the field should check: **"Has the second (update) Edit Fields node executed yet?"**

- If yes → take the retry count value **from that updated node**.
- If no (i.e., this is the very first pass through the loop) → take the value from the **first/initial** Edit Fields node (which is 0).
- The instructor demonstrates this using a simple ternary operator refresher first: `C = (A < B) ? 30 : 40` — if the condition is true, use the first value; if false, use the second value. Same idea applied here: `condition ? value-from-updated-node : value-from-initial-node`.
- After making this change (referencing the correct node's output in the retry count field), the instructor reruns the workflow and confirms: retry count correctly becomes 1, then 2, then 3, and the workflow correctly stops looping once the retry limit is hit (in the demo, the retry count reaching 3 stopped further optimization even though the score requirement, e.g. ≥97, was never actually met).

616. **Simplified single IF node.** As started in the previous session, the two IF checks are combined into **one IF node**: `score >= threshold` **OR** `retry count >= 3` → true branch sends the Gmail message; false branch increments retry count and sends to the optimizer agent. Careful attention is paid to the boundary values (>= vs >) to avoid off-by-one errors.
617. **Final working test.** With the fix in place, the workflow is tested end-to-end: it loops through the optimizer up to 3 times, then delivers the best available answer to the student via Gmail, confirming the evaluate-optimize pattern with a working retry limit.

Part 2: Recap of the four n8n design patterns

618. The instructor summarizes all patterns covered in the course so far:

- **Prompt chaining** — dependent, sequential tasks; output of one step feeds the next.
- **Routing** — using a Text Classifier node to send a request down different branches based on its content.
- **Parallelism** — independent tasks executed simultaneously to improve response time (with the caveat that n8n's visual canvas always displays node-by-node execution, even though it's parallel internally).
- **Evaluate-Optimize (loop)** — generate an answer, evaluate it, and optimize/retry if it doesn't meet requirements, with a retry limit to avoid infinite loops.
- These are described as being like **design patterns**: you choose the right one based on your specific requirement, and the same concepts apply later when working directly in code frameworks like LangChain/LangGraph.

Part 3: Introducing MCP (Model Context Protocol)

619. **Why MCP is needed — analogy from web services.** The instructor builds up the concept from familiar ground: two normal applications (e.g., a Java app and a .NET app) that need to talk to each other require a common "language" — this is what **HTTP** (as a protocol) and **JSON** (as a lightweight message format, replacing the older, heavier XML) provide. This is the basis of **REST APIs / web services** — a concept students should already be familiar with.
620. **The AI-specific problem.** Now that AI apps (ChatGPT, Lovable, etc.) need to communicate with *other* applications (databases, n8n workflows, Gmail, Google Sheets, etc.), there needed to be a **specially designed protocol for AI-based applications** to securely connect and communicate with external tools/data sources in a standardized way. That protocol is **MCP — Model Context Protocol**.

- Example given: if ChatGPT needs to check how many nodes exist in your n8n workflow, it has no way to know unless your n8n instance and ChatGPT are connected via MCP.

621. **Enabling MCP in n8n (step-by-step):**
622. Go to **n8n → Settings → Instance level MCP** (bottom-left of the n8n interface).
623. By default, **MCP access is disabled** — toggle "Enable MCP access" to turn it on.

624. Once enabled, n8n shows **Connection details**: a **Server URL**, an **Access Token**, and a **Configuration JSON** — any of these can be used by an external client (like ChatGPT) to connect to your n8n instance.
625. For a *specific workflow* to be reachable via MCP, that workflow must be:

- **Published / made live** (in production).
- Explicitly marked **"Available in MCP"** via the workflow's own settings (three-dot menu → Settings → enable "Available in MCP" → Save).

626. Confirm by returning to Settings → Instance level MCP → "Enabled workflows" list — the workflow should now appear there.
627. **Enabling MCP in ChatGPT (step-by-step):**
628. Go to your ChatGPT **profile → Settings → Apps → "Explore apps" / "Manage apps."** ChatGPT supports connecting to many built-in apps (lifestyle apps, productivity apps like GitHub, Google Calendar, Google Drive, Gmail, SharePoint, Outlook, Canva, etc.) — most users never explore this.
629. Since n8n isn't in the predefined app list, go to **Settings → Apps → Advanced settings → enable "Developer mode."**
630. With Developer mode on, a **"Create app"** option appears at the top.
631. Click **Create app** → choose to configure a **custom tool**, give it a name (e.g., "n8n"), an optional description, and paste in the **MCP Server URL** copied from n8n's connection details.
632. ChatGPT shows a security warning ("OpenAI has not reviewed this MCP... attackers may attempt to steal your data...") — acknowledge and click **Create**.
633. On the authentication/permissions screen, ChatGPT lists what it will be allowed to do: **get a list of your workflows, get details for a specific workflow, and execute workflows on your behalf.**
634. Once created, the custom "n8n" tool appears under your enabled apps.
635. **Using the connection — live demo.** In a ChatGPT chat (with Developer mode enabled, indicated by a red-highlighted input box), the instructor clicks **+ → More → n8n** to attach the custom tool, then asks ChatGPT to "provide list of n8n workflows." ChatGPT successfully lists all of the instructor's n8n workflows and confirms which ones have MCP access enabled.
636. **Executing the workflow from ChatGPT.** The instructor asks ChatGPT to execute the specific evaluate-optimize workflow, supplying name, email, and a question ("N8N jobs"), directly through the chat. ChatGPT calls the workflow via MCP:

- First attempt fails because the `question` field wasn't passed correctly to the AI agent ("No prompt specified" error) — illustrating that data must be mapped correctly even when triggering remotely via MCP.
- After correcting the input and lowering the evaluator's score threshold (since it had been set unrealistically high, e.g. 97, for demo purposes), a second execution succeeds: the AI agent generates the answer, the evaluator scores it, and (with the retry-count fix from earlier in the session) the workflow completes and sends the email correctly — with execution now triggered entirely from ChatGPT instead of from n8n directly or a web form.

- A side note: running the OpenAI-backed nodes purely through the free/non-paid ChatGPT connector may hit limitations ("not able to fetch the data") unless a real OpenAI API key/credits are used in the underlying n8n workflow.

637. **Wrap-up.** The instructor confirms this was just a basic introduction to MCP and that more depth/examples will follow in the next session.

Key takeaways

- The retry-counter bug from Demo 44 was caused by referencing the wrong node — always pull the "current" value from the most recently updated node, using a conditional/ternary check ("has the update node run yet?") rather than blindly referencing the initial node.
- The four n8n design patterns covered so far: **prompt chaining** (sequential/dependent), **routing** (branch by classification), **parallelism** (independent tasks run simultaneously), and **evaluate-optimize** (generate → check → improve loop, with a retry limit).
- **MCP (Model Context Protocol)** is a standardized, secure protocol that lets AI applications (like ChatGPT) connect to and communicate with external tools and data sources (like an n8n workflow) — conceptually similar to how HTTP/REST/JSON let two regular web applications talk to each other, but purpose-built for AI systems.
- Enabling MCP in n8n requires two steps: enabling **instance-level MCP access** in Settings, and separately marking each individual workflow as **"Available in MCP"** after publishing it.
- Connecting ChatGPT to an external MCP server (like n8n) requires enabling **Developer Mode** and creating a **custom app/tool** with the server's MCP URL.
- Once connected, ChatGPT can list, inspect, and directly **execute** n8n workflows — effectively letting an AI chat interface trigger real automations without needing a separate web form or webhook.

Demo 46: Understanding MCP (Model Context Protocol) and Connecting ChatGPT to n8n

This video introduces the Model Context Protocol (MCP) concept in simple terms and then walks through a live demo of building an n8n workflow with a database table, publishing it, and connecting ChatGPT to n8n through MCP so that ChatGPT can list, inspect, and execute n8n workflows.

What you'll learn

- What MCP stands for and why AI models need it
- The difference between a "generalized" AI model and one connected to external data via MCP
- How MCP compares to the older way of connecting apps (APIs, webhooks, HTTP/JSON/XML)
- How to build a simple n8n form that stores data into an n8n Data Table
- How to enable MCP access at both the n8n platform level and the workflow level
- How to connect ChatGPT to n8n as an MCP client and use it to list and execute workflows
- How to add a second "get data" workflow so ChatGPT can read back stored data and analyze it

Step-by-step notes

1. What is MCP, conceptually

- MCP = Model Context Protocol. "Model" refers to the LLM (e.g., ChatGPT), "Context" refers to the extra information the model needs, and "Protocol" means a standard set of rules for communication.
- An LLM on its own only answers using its trained data, which can be outdated or missing company/client-specific information (e.g., data in your Google Sheets or database).
- To make the model more powerful ("100 elephants power" becomes "1000 elephants power," as the instructor puts it), you connect it to external apps, tools, databases, and workflows. Whatever data comes from those external sources is called "context," and whoever supplies that data is the "context provider" (e.g., n8n, a MySQL database, etc.).
- Before MCP, applications talked to each other using APIs, HTTP endpoints, JSON/XML, or webhooks. MCP is a newer, simpler, standardized way specifically designed for AI models to talk to external tools/data sources, replacing a lot of one-off custom integration work.
- If a third-party tool/database is not "MCP enabled," the AI model cannot connect to it directly via MCP. In that case you can go through an intermediate app that does support MCP (e.g., n8n) which then talks to the database on the model's behalf.
- Important clarification: connecting more tools via MCP does not make the LLM itself "smarter" or increase its token capacity — the LLM is just a processor. What changes is that it now receives better, real/fresh raw data as input, so its output is more useful and accurate.

2. Building the n8n side of the demo

638. Create a new n8n workflow with an **n8n Form** trigger (e.g., "Student Feedback Request Form / Feedback Submit Form").
639. Add form fields: Name (required), Email (required, email type), Mobile Number (required, number type), Feedback (required, text/text-area).
640. Add an n8n **Data Table** node (n8n's built-in database, shared across all workflows in the project).
641. Create a new table (e.g., "student feedback 2") and manually add columns: name, mail id, mobile number, feedback (avoid spaces in column names). Two default columns — Created, Updated — plus an ID column are added automatically.
642. Add an **Insert Row** operation on the Data Table node, mapping the form fields to the table columns.
643. Execute/test the workflow — submitted form data appears both in the workflow execution and in the Data Table (visible under the n8n **Overview/Home page → Data Tables**).
644. Publish the workflow and share its production URL so multiple students can submit feedback; each submission inserts a new row.

3. Enabling MCP access in n8n

- MCP must be enabled in **two places**:

645. **Platform level**: Go to n8n Settings (bottom-left profile menu) → enable "Instance level MCP" (disabled by default).
646. **Workflow level**: Open the specific workflow → the "…" menu → Settings → enable MCP access for that workflow (also toggled automatically when you check "Enable MCP access" while publishing).

- Once both are enabled, n8n exposes an **MCP server URL** (like a client-server setup: ChatGPT is the MCP client, n8n is the MCP server).

4. Connecting ChatGPT to n8n via MCP

647. In ChatGPT: Settings → Apps → Advanced Settings → enable **Developer Mode** (disabled by default; needed because n8n isn't a pre-built/official ChatGPT app).
648. Click **Create App**: give it a name (e.g., "n8n"), paste the MCP server URL you copied from n8n, and create.
649. Approve the connection when ChatGPT asks to confirm ("ChatGPT wants to access your n8n instance... get list of workflows, execute workflows on your behalf...").
650. The new tool appears (shown in red because developer mode is on, indicating higher risk — anyone with that URL could potentially connect, so keep it private).
651. In a ChatGPT chat, enable the n8n tool (via the "+"/tools option) and ask things like "list out available workflows" — ChatGPT calls the MCP tool and returns the list of n8n workflows (name, ID, active status, MCP availability).

652. Ask ChatGPT to fetch details of a specific workflow by ID, and even to **execute** that workflow directly from the chat, supplying field values (name, email, mobile, feedback) — this actually runs the n8n workflow and inserts a new row, provable by checking the Data Table.

5. Limitation discovered and fixed: reading data back

- Giving ChatGPT MCP access to a workflow only allows what that workflow does (here, only *insert*). Asking ChatGPT to directly query the database table fails — there's no workflow exposed for that.
- Fix: Create a **second workflow** ("get data workflow") with a dummy form trigger (just to have a trigger) plus a Data Table node set to **Get Rows** operation on the same table. Publish it and enable MCP for it too.
- Now ChatGPT can call this second workflow through MCP to retrieve data (in this test, only 10 rows came back at once — the instructor suspected a default limit to be checked later), export it to Excel, and even analyze it (e.g., "which students gave negative feedback," "give me 3 action items to improve").
- Alternatively, both operations (insert row and get rows) can be added as two separate nodes inside the **same** workflow, avoiding the need for two workflows.

6. Security note raised in class

- Enabling MCP is like "opening a door" to your data — only give the URL to people/systems you trust; developer mode explicitly warns about this risk.
- A benefit of using a real database (via n8n Data Tables) instead of Google Sheets is that concurrent submissions from many users don't get lost/overwritten, unlike shared spreadsheet editing.

Key takeaways

- MCP = Model Context Protocol — a standard way for AI models to talk to external tools, data sources, and workflows to get "context," instead of relying only on their frozen training data.
- MCP does not make the LLM's brain smarter or cheaper to run — it just gives the LLM better/fresher input.
- You must enable MCP on both sides of a connection (e.g., both in n8n and in ChatGPT/the client) and, for n8n specifically, at both the platform and the workflow level.
- ChatGPT (via Developer Mode) can be turned into an MCP client that lists, inspects, and executes n8n workflows.
- Any capability you want ChatGPT to have (insert, get, update, etc.) must exist as an actual workflow — MCP only exposes what your workflows can do, not raw direct database access.
- Enabling MCP is a security-sensitive action; treat the MCP server URL like a password.

Demo 47: Connecting Lovable (No-Code App Builder) to n8n — Via MCP and Via Webhook

This video shows how to connect a Lovable-built front-end app to an n8n backend workflow. The instructor first attempts to use MCP (like Demo 46) but hits a paywall (Lovable requires a Pro plan for MCP), so he falls back to a webhook-based integration, builds a working student inquiry form end-to-end, and revisits how MCP-based querying works with ChatGPT (including a discussion on conversation context avoiding repeated tool calls).

What you'll learn

- How Lovable (a no-code/AI app builder) can act as a "front end" while n8n acts as the "back end + database"
- Why MCP in Lovable requires a paid (Pro) plan, and the webhook-based alternative
- How to build an n8n webhook workflow that receives POST data and inserts it into a Data Table
- How to prompt Lovable to build a form that posts to a specific webhook URL
- How to troubleshoot a "map automatically" vs "map each column manually" issue when extra fields break an insert
- How to connect ChatGPT to n8n via MCP again, ask multi-step business questions, and even have the AI add new columns to a data table based on analysis
- How conversation context in ChatGPT avoids the need to re-trigger a workflow for follow-up questions on the same data

Step-by-step notes

1. Recap and setup

- MCP connects an AI/app "front end" model to an n8n "back end" — the "bridge" between them is MCP.
- The instructor built a Lovable app (front end) meant to connect to an existing n8n workflow (back end, holding the database).

2. First attempt: MCP in Lovable fails due to a paywall

- In Lovable: **+ (plus) → Connectors → Manage Connectors** shows available app connectors, including an n8n MCP connector.
- Clicking "Connect" on the n8n MCP connector asks for the n8n **MCP server URL** — but Lovable then shows: "Custom MCP servers require a Pro plan or higher." MCP integration in Lovable is a paid feature.
- Decision: since Lovable is only used occasionally, paying for Pro isn't worth it — switch to the webhook approach instead (webhooks have no such restriction).

3. Building the webhook-based n8n workflow

653. Create a new n8n workflow with a **Webhook** node: HTTP Method = POST, Authentication = None, Respond = "Using Respond to Webhook node."
654. Add a **Data Table → Insert Row** node targeting the existing table (e.g., "student inquiries"), first trying "map automatically."
655. Add a **Respond to Webhook** node so the webhook returns a response after insert.
656. Give the webhook a meaningful path (e.g., "inquiry") so the auto-generated webhook URL is readable.

4. Prompting Lovable to build the front-end form

657. In Lovable, prompt: "Build an app to submit student inquiry data" with fields (name, mobile number, email, course dropdown, message).
658. Revise the prompt to explicitly say "create webhook integration, do not use MCP," and paste in the webhook URL, instructing it to POST the exact field values, show a success message on submit, and show an error message on failure. Keep it "simple, clean, mobile-friendly, beginner-friendly."
659. Lovable builds the page in under a minute (a full clean form with dropdown for course and message field).

5. Debugging the first submission failure

- First test submission returned "something went wrong." Cause: the workflow hadn't been **published** yet (only draft/test mode was active) — publishing was required for real execution.
- After publishing, the workflow executed, but the Data Table row insert had extra/unexpected fields, so **"map automatically"** picked up the wrong columns. Fix: switch to **"map each column manually"** and explicitly map name→name, mobile→mobile number, course→course, message→message.
- After the fix, submissions correctly landed in the Data Table.

6. Making the Lovable app live

660. Publish the n8n workflow → get the **production URL** (not just the test URL).
661. In Lovable, update the code/prompt to use the production webhook URL (the instructor asked Lovable to update this directly since the code is in files like index.tsx that are hard to hand-edit without knowing the framework used — Lovable generates code in a React/Angular-like structure with HTML/CSS/component files).
662. Publish the Lovable project itself, then test the live form — submissions now flow through the production webhook into the n8n database reliably.
663. The instructor notes Lovable can build much bigger, multi-page, styled websites too (not just single forms) — you just keep prompting for more pages/pages of functionality (even login/session features), and it can take as little as 10–15 minutes to get a decent website.

7. Connecting ChatGPT to n8n via MCP to read the collected data

664. Add a second "get rows" workflow (similar to Demo 46), publish it, and enable MCP for it (platform level + workflow level).

665. In ChatGPT (Developer Mode still enabled from before), create/re-use an MCP tool connection to n8n's server URL.
666. Ask ChatGPT: "List MCP-enabled workflows" → it correctly identifies the get-data workflow.
667. Ask business questions like "how many Java inquiries are there?" and "how many AI inquiries are there?" — ChatGPT triggers the workflow, retrieves data, and answers accurately (e.g., "9 AI inquiries").
668. Impressively, when asked to analyze inquiry timing patterns and "add a best batch timings" recommendation, ChatGPT was able to **add a new column** to the Data Table and populate/report new information via MCP, demonstrating that MCP-connected tools can also perform write/update actions, not just reads.

8. Context behavior discussion

- A student asked whether every follow-up question re-triggers the workflow. Answer: no — once ChatGPT has already retrieved data into the conversation, follow-up questions can be answered directly "from the above information" without re-calling the tool, if you phrase your prompt that way (e.g., "don't connect again, just use the data already retrieved").

9. Wrap-up: MCP video generated with Gamma + Google Vids

- The instructor showed a short auto-generated explainer video about MCP (created by giving one prompt to Gamma to build a presentation, then converting that presentation into a video using Google Vids). Key points from that video:
- AI is "smart but stuck" — it knows a lot from training but can't see live/current data (calendar, emails, news) without help.
- MCP acts like a "phone" that lets the AI call out for live information or to perform an action.
- Before MCP, connecting AI to tools required custom-built integrations for every tool ("a unique door for every room") — slow and expensive. MCP is like a "universal door key" (compared to how USB standardized device connections).
- MCP architecture: **Host** (the AI's brain receiving the user request), **Client** (coordinator that finds the right tool/server), **Server** (the worker that executes the task and returns the answer).
- Example: "Book a meeting with John for 10am tomorrow" → MCP host recognizes it needs help → client finds the calendar MCP server → server books the meeting and confirms.
- MCP is a key reason AI is evolving from simple chatbots (which just answer) into AI agents (which understand a task, pick the right tools, and complete it on their own).

Key takeaways

- Any time a model needs to connect to an external app/database/workflow, that connection standard is MCP — but not every platform offers MCP for free (Lovable requires a Pro plan; webhooks remain a free, simpler fallback).
- Webhook-based integration (POST + Respond to Webhook) is a solid, no-cost alternative to MCP when the paid tool restricts it.

- Always double check "map automatically" vs "map manually" when inserting webhook data into a table — extra/unexpected fields can silently break automatic mapping.
- You must publish an n8n workflow (and use its production URL) before external apps can reliably call it.
- MCP-connected tools aren't limited to reading data — an LLM with the right access can also modify/add data (e.g., add a new column) as part of "agentic" behavior.
- Conversation context lets you avoid unnecessary repeated tool calls — explicitly tell the AI to reuse already-retrieved data for follow-up questions.

Demo 48: MCP with Xfield (AI Video Generation) + Introduction to RAG (Retrieval Augmented Generation)

The first half of this video demonstrates connecting ChatGPT to Xfield.ai (a hybrid multi-model AI video/image/audio generation platform) via MCP so ChatGPT can trigger AI video generation using various underlying video models. The second half introduces the RAG (Retrieval Augmented Generation) concept in plain language, using a "refer only to this book" analogy, and explains why RAG reduces hallucination and avoids costly model retraining.

What you'll learn

- How to connect ChatGPT to a third-party AI platform (Xfield.ai) via MCP to generate videos on demand
- That ChatGPT's own reasoning ("brain") is used to interpret/enhance prompts even when the actual generation happens on another platform
- How credits/tokens work across a connected MCP tool (Xfield credits vs OpenAI tokens)
- The basic concept of RAG: retrieval, augmentation, generation
- Why RAG reduces hallucination, keeps answers up to date, and is more cost-effective than retraining an LLM
- The distinction between a pure LLM's knowledge (frozen at training cutoff) and a UI application's extra live-search tools

Step-by-step notes

1. Recap of MCP from previous days

- Lovable (front end) posts data via webhook to n8n (back end/database); ChatGPT (via MCP) can read/display that data. The bridge enabling all these app-to-app / model-to-app communications is MCP.

2. Introducing Xfield.ai

- Xfield.ai is a paid hybrid platform offering access to many different LLMs and generation models (video, image, audio) in one place — e.g., video models like Sora, Kling, Google VO 3.1, Cling, Hailuo, Wan, Marketing Studio, Cinematic Studio, etc.
- Because ChatGPT's own native video tool (Sora) wasn't up to the instructor's needs, the goal was: prompt ChatGPT, have it call Xfield via MCP, and have Xfield actually generate and return the video.

3. Enabling MCP between ChatGPT and Xfield

669. In Xfield.ai: there's an "MCP and CLI" section (a new feature) — enable MCP there (by default it may install/enable "CLI" first).
670. Xfield exposes a **custom connector URL** ("Turn ChatGPT/Claude/OpenCLI into a creative engine") — copy that URL.

671. In ChatGPT: Settings → Apps → Advanced Settings → enable Developer Mode → Create a new app (no predefined "Xfield" app exists, so a custom one is created) → paste in the Xfield MCP server URL → authorize/login when prompted.
672. Once connected, both ChatGPT and Xfield can communicate via MCP.

4. Using MCP to generate videos from ChatGPT

673. Ask ChatGPT: "What are the various available video generation models?" — via MCP it fetches Xfield's live model list (V3.1, V3, Kling, Cinematic Studio, Marketing Studio, etc.), and ChatGPT itself (using its own reasoning) recommends which model suits which use case (e.g., "Marketing Studio is better for ads," "Cinematic Studio v3 for cinema-grade video") — this analysis/recommendation comes from ChatGPT's own "brain," not from Xfield.
674. Give a video generation prompt specifying: content description, model name (must match Xfield's exact model name), resolution (e.g., 720p), aspect ratio (e.g., 16:9), and duration if relevant (e.g., V3.1 Light supports up to 8 seconds).
675. ChatGPT enhances/expands the prompt automatically before sending it to Xfield (e.g., turning "monkey jumping from branch to branch" into a fuller cinematic description).
676. The video generation happens on Xfield's side and is credited against Xfield's monthly credit allowance (e.g., ~6000 credits/month), with different models consuming different numbers of credits (e.g., Google VO ≈ 10 credits per generation) — separate from ChatGPT's own token usage.
677. The generated video can be verified as genuinely created in Xfield by checking the Xfield platform's own video library — the same video appears there.
678. Some generation attempts failed or timed out (e.g., "cinematic studio does not support 720p, setting omitted," or upstream errors) — this is normal with cross-tool integrations; retrying or switching models (e.g., to Google V3.1) usually resolves it.

5. Cost/token discussion

- Both ChatGPT (OpenAI API/credits) and Xfield (its own credit system) charge separately; incoming and outgoing "calls" both cost, similar to how old mobile phone plans charged for both incoming and outgoing calls. Input tokens and output tokens are both billed on OpenAI's developer platform.
- Whenever evaluating a new AI tool subscription, first check: "Is MCP enabled/supported?" — that determines if it can be wired into an AI-driven "factory" style workflow.

6. Transition to RAG (Retrieval Augmented Generation) — first pass

- Analogy used: imagine the instructor is the LLM. Normally he answers from what he already knows (pre-trained data). But if told "answer only from this specific book, not your own knowledge," he must look up the answer inside that book before responding, and if the answer isn't in the book, he should say so (or optionally fall back to general knowledge, depending on configuration) — this book-referencing behavior models RAG.
- RAG = **R**etrieval + **A**ugmented + **Generation**:

679. **Retrieval**: when a query comes in, relevant matched "chunks" (pieces of data) are retrieved from a vector database (also called a RAG database).
680. **Augmented**: the user's query and the retrieved matched chunks are combined ("augmented" = combined).
681. **Generation**: the combined input is handed to the LLM, which generates the final response using both the question and the retrieved context.

- Benefits of RAG:
- **Reduces hallucination** — answers are grounded in real, retrieved data rather than guesswork.
- **Company/organization-specific answers** — instead of generic answers (e.g., generic "minimum balance" info), the model can answer using your specific documents (e.g., your bank's actual policy).
- **Cost-effective and easy to update** — if company data changes (e.g., a leave policy), you simply replace the old PDF with a new one in the knowledge source; you do NOT need to retrain the LLM, which is expensive.
- A short Gamma-generated presentation (again converted into a narrated video via Google Vids, matching the pattern from Demo 47) reinforced these points:
- The problem: AI models are trained on massive datasets but have a "knowledge cutoff date" — anything after that is unknown to them, and they can "hallucinate" confidently.
- RAG works like a "research assistant": before answering, it searches an external knowledge base for relevant facts (retrieval), adds those facts to the AI's context (augmentation), then the AI combines its general knowledge with the retrieved information to produce a grounded, accurate answer (generation).
- Four-step loop: **You ask → Assistant searches → Info retrieved → AI answers.**
- Real-world RAG examples: customer support bots that search a company help doc for accurate answers; internal knowledge bases that help employees find information buried in company docs/wikis; research assistants delivering source-based answers beyond a model's training cutoff.
- Key benefits recap: accuracy (grounded in real external data), always up to date, trustworthy responses, and cost-effective (cheaper than retraining a model every time data changes).
- Clarification during Q&A: whether ChatGPT can search the live web depends on extra tools built into the ChatGPT product (not the underlying LLM itself) — the pure LLM only knows its trained data and cannot look up "today's news" unless an external live-search capability is explicitly wired in (this foreshadows the deeper explanation given in Demo 49/50).

Key takeaways

- MCP isn't limited to workflow platforms like n8n — it can connect ChatGPT to specialized AI creative tools like Xfield.ai for on-demand generation (video, image, audio).
- When using MCP-connected tools, the "reasoning/analysis" (e.g., recommending which model to use) is done by the calling LLM (ChatGPT), while the actual task execution (e.g., video rendering) happens on the connected platform, consuming that platform's own credits.

- MCP tool calls can be flaky (timeouts, unsupported parameters) — this is a normal characteristic of cross-platform integrations, not a fundamental flaw.
- RAG = Retrieval + Augmented + Generation: retrieve matching chunks from a vector/RAG database, combine them with the user's query, then let the LLM generate a grounded answer.
- RAG's main benefits: less hallucination, more organization-specific answers, always up-to-date knowledge, and much cheaper than retraining a model whenever data changes.
- A pure LLM only knows its training data; live web search or "fresh" answers require separate tools layered on top of the base model.

Demo 49: RAG Deep Dive — Embeddings, Vector Databases, Semantic Search, and Building a RAG Pipeline in n8n

This video goes deeper into RAG mechanics: what embeddings and vector databases are, why semantic search (not keyword search) is used, what "chunking" means, and then walks through building a real two-workflow RAG pipeline in n8n (one workflow to store a PDF's content as vector embeddings, another to answer user questions using that vector store).

What you'll learn

- What embeddings are and why AI models can only "understand" embeddings, not raw text
- The difference between keyword search and semantic search, and why semantic search matters for RAG
- What "chunking" means and why large documents are split into chunks before embedding
- How to build an n8n "vector pipeline" workflow (Trigger → Google Drive download → Simple Vector Store → Default Data Loader → Embeddings)
- How to build a second n8n workflow (Chat Trigger → AI Agent → Chat Model + Vector Store tool + Embeddings) that answers questions using the stored vectors
- Why the system prompt matters for controlling whether the AI sticks strictly to retrieved data or falls back to general knowledge
- That images are not stored directly as text embeddings; they typically go through cloud storage and a URL, or descriptive conversion

Step-by-step notes

1. Quick recap of RAG basics

- Without RAG, an LLM only answers from its pre-trained data, which becomes stale and is too generic for organization-specific needs (e.g., DurgaSoft-specific course info).
- With RAG, you provide your own documents (PDFs, Excel, text) as a reference: "Answer only from this book/knowledge base; if the answer isn't there, say so (or optionally give a generic answer, per the system prompt's instruction)."

2. Why raw documents must become vector embeddings

- AI models don't understand plain English or normal numbers directly — internally, tokens are converted into **vector embeddings** (a "vector" = a group of numeric values, not a single number).
- Search inside a normal SQL/relational database (MySQL, Oracle, etc.) is **keyword-based** (exact match), whereas AI-driven search is **semantic** — it searches by **meaning**, not exact keywords. Example: a document says "duration 45 days"; if you ask "how long is the Python course," a keyword search would fail (no "how long" text in the doc), but semantic search understands the question means the same thing as "duration" and returns "45 days."
- This "search by meaning" is called **semantic search**.

- Illustrative toy example of embeddings: imagine words like "apple" and "orange" each represented by ~100 yes/no or numeric parameters ("Is it a fruit?" "Is it edible?" "Is it drivable?" "Is it round?"). Words with similar answers across these parameters have embeddings that are numerically close together — this closeness is how the model judges words/concepts as "related." (This is a simplified teaching analogy, not the literal mechanism.)
- Because relational databases only support keyword search, you need a special **vector database** to store embeddings and support semantic search. Examples of vector databases mentioned: the instructor's demo uses n8n's **built-in "Simple Vector Store"**; third-party vector databases include Supabase, Pinecone, Chroma, and others (covered more in Demo 50).

3. Chunking

- You cannot practically convert an entire large document into one giant embedding at once — instead, the document is split into smaller pieces called **chunks**, and each chunk is converted into its own vector embedding, then stored in the vector database.
- The chunking strategy (how much text per chunk) is decided by the **embedding/data-loading component** you use (e.g., n8n's "Default Data Loader"), not something you manually calculate word-by-word.
- Two conceptual steps in a RAG setup:

682. **One-time step**: Convert your source documents into chunks → embeddings → store in vector database ("vector pipelining"). This only needs to be redone when the source document changes (e.g., updating a PDF with new info).
683. **Repeated step**: Answer user queries by retrieving matching chunks from the vector database, combining with the query, and generating a response — this happens every time a user asks a question.

4. Building Workflow 1 — the Vector Pipeline (store PDF as embeddings)

684. Add a trigger node (any trigger just to let the workflow run once, e.g., a manual/dummy trigger).
685. Add a **Google Drive** node to download the source PDF file (e.g., "DurgaSoft Python Course PDF") stored in Google Drive — output is binary data.
686. Add an n8n **Simple Vector Store** node, operation = "Add documents to vector store" (a.k.a. "Insert documents"):

- Give the vector store a name/key (e.g., "DASoft course PDF" or "Python information document") — this name is how other workflows will reference the same database later.
- Set **embedding batch size** (default 200) — this is how many chunks are processed at a time, not the chunk size itself.
- Connect two required sub-inputs: (a) **Document** input — via a **Default Data Loader** node (loads the binary PDF data and splits it into chunks), and (b) **Embeddings** input — via an **Embeddings OpenAI** node (converts chunks into vector embeddings; other providers like Gemini can be used instead).

687. Execute the workflow once — the PDF's content is now stored as chunks + embeddings inside the named vector store.

5. Building Workflow 2 — Answering Queries Using the Vector Store

688. Add a **Chat Trigger** (or form trigger) so a user can type a question (e.g., "What is the duration of the Python course?").

689. Add an **AI Agent** node with:

- A **system message** instructing the agent to answer only from the provided documents/knowledge base, and to say "sorry, this information is not available in the course document" if it can't find an answer (rather than guessing) — or, alternatively, to fall back to a generic best-guess answer if configured that way (the instructor explicitly notes this "fallback" instruction should generally NOT be used in real RAG systems — it was shown only for testing/demonstration).
- An **OpenAI Chat Model** (the "brain"/LLM that ultimately generates the response).
- The **Simple Vector Store** added as a **tool** for the agent (operation: "Retrieve Documents"), pointing at the *same* vector store name created in Workflow 1 (e.g., "Python information document") — plus its own **Embeddings OpenAI** node, because the user's incoming question must ALSO be converted into an embedding before it can be matched against the stored document embeddings.
- A required **tool description** field on the vector store tool (e.g., "Use this tool to answer questions from DASoft course PDF knowledge base including course duration, syllabus, trainer, projects, timings, fees...") — this description is what the AI agent uses internally to decide when to call this tool.

690. Test queries:

- "What is the duration of the Python course?" → correctly answers "45 days" from the PDF.
- "How long [is the] Python course?" (different phrasing, no exact keyword match) → still answers correctly, demonstrating semantic search.
- "Who is the trainer?" vs. "Who is the instructor?" (synonym, not in the document verbatim) → still correctly matches "Dasar" as the trainer, because the AI understands "instructor" and "trainer" mean the same thing.
- "What is the syllabus?" → correctly lists topics from the PDF.
- "Who is the trainer for the Java course?" (not in the document, since only Python info was provided) → correctly responds "sorry, this information is not available in the document" — confirming answers are grounded, not hallucinated, when the system prompt enforces "no guessing."
- If the system prompt is changed to allow a "best possible answer based on your knowledge" as a fallback, then asking about a topic not in the document (e.g., Java course duration) causes the LLM to generate a plausible but non-grounded generic answer — the instructor stresses this fallback should not normally be used in production RAG systems; it was shown only to illustrate the contrast.

6. Clarifying confusion points raised by students

- The two OpenAI-embeddings components (one in Workflow 1 for the documents, one in Workflow 2 for the incoming question) are conceptually the same kind of tool but used at different points: one embeds the stored knowledge once, the other embeds each new user question at query time so it can be matched.
- The vector store's embeddings model doesn't have to match the LLM's own internal (pretraining) embeddings — those are two different, unrelated systems. The vector store's embeddings exist purely to support similarity search over your documents; the LLM's own internal token embeddings exist to help it predict the next token during generation.
- Images are **not** stored directly in the vector database as-is; typically the image itself is stored in cloud storage (e.g., S3, Google Cloud), and a URL/reference (plus possibly text description) is what actually gets embedded or referenced.
- You do not need to keep both workflows inside the same n8n project — as long as the vector store name/key matches, any workflow can reference the same stored vector data.
- This RAG behavior is fundamentally different from **fine-tuning** an LLM: fine-tuning changes the model's own internal token-level embeddings/weights (a costly, complex process), whereas RAG just retrieves external chunk data at answer-time without touching the model itself.

Key takeaways

- AI models understand embeddings (numeric vector representations), not raw text — this is why documents must be converted before an LLM can meaningfully search them.
- Semantic search (search by meaning) is what makes RAG powerful over plain keyword search — it can answer paraphrased or differently-worded questions correctly.
- Large documents are split into "chunks" before embedding; chunking strategy is handled by the data loader component, not manually.
- A working n8n RAG pipeline needs two workflows: one to build/update the vector store (one-time/occasional), and one to answer user queries using that vector store (runs every query).
- The vector store used for storing and the vector store used for retrieving must reference the exact same name/key to work together.
- A well-designed RAG system prompt should instruct the model to answer strictly from retrieved documents and admit when information isn't available, rather than silently guessing — avoid "fallback to general knowledge" phrasing in real RAG deployments.
- RAG and fine-tuning are different concepts: RAG retrieves external data at query time (cheap, easy to update); fine-tuning retrains the model itself (expensive, complex).

Demo 50: RAG in Practice — Chunk Size, Overlap, Embedding Batch Size Explained with a Cricket Rules Example

Building on Demo 49, this session repeats the RAG pipeline build (using a new, richer "50 Cricket Rules" PDF instead of the earlier course PDF) to reinforce the concepts, and goes deeper into exactly how chunk size, chunk overlap, and the "top-K matches" retrieval limit work — including a live look at how chunks and their vector embeddings actually appear inside Supabase (previewed ahead of a fuller Supabase walkthrough planned for the next session).

What you'll learn

- Why n8n's RAG concepts require two workflows (vector pipelining + query answering), reinforced with a second example
- How default chunk size (1000 characters) and chunk overlap (200 characters) work together to preserve context between chunks
- What "embedding batch size" (default 200) actually controls
- How the "top-K" (e.g., top 4, or top 3) matched-chunks setting affects what gets sent to the LLM
- How to peek at real chunk text and vector embeddings using Supabase
- The distinction between "vectorless RAG" (mentioned as a newer, emerging alternative for small documents) and standard vector-based RAG
- That RAG output formatting can vary between runs even though the underlying retrieved data/content stays the same

Step-by-step notes

1. Setting the scene: introducing Supabase as a third-party vector database

- The instructor noticed the word "MCP" mentioned on the Supabase website and used that as a segue to reintroduce the concept that Supabase supports MCP.
- Supabase is one of several popular third-party **vector databases** (others mentioned: Pinecone, Chroma/ChromaDB, Neo4j was also discussed by a student as a graph-based alternative). These exist parallel to standard relational databases (Postgres, MySQL, Oracle) but are purpose-built to store chunks and their embeddings for semantic search.
- Unlike n8n's built-in Simple Vector Store (where you can't directly view stored chunks/embeddings), Supabase lets you literally see the chunk text and its corresponding embedding values in a table.
- Supabase setup is a bit more involved (manual database setup, credentials, similar to setting up a Telegram/WhatsApp bot integration) — deferred to the next session.

2. Rebuilding the RAG pipeline with a new example: 50 Cricket Rules

- The instructor created a new PDF ("50 Important Rules of Cricket") using ChatGPT (asked it to generate cricket rules 10 at a time), covering things like: 11 players per team, toss decides batting/bowling, 1 over = 6 balls, 4 runs (boundary after bounce) vs 6 runs (boundary without

bounce), types of dismissal (bowled, caught, run out, LBW, hit wicket, etc.), no-ball, free hit, powerplay, duck, golden duck, DLS method, super over, follow-on, declaration, hat-trick, and more. (Mentions Kaggle.com as a general source for datasets when needed for other topics like data science.)

- Uploaded the PDF to Google Drive, then rebuilt the same two-workflow structure as Demo 49:

691. **Workflow 1 (Vector Pipelining)**: Manual trigger → Google Drive (download the cricket rules PDF, binary output) → Simple Vector Store (operation: Add documents; created a new database named "cricket rules database") → Default Data Loader (loads binary, splits into chunks) → Embeddings OpenAI.
692. **Workflow 2 (Answering)**: Chat Trigger → AI Agent (system message: "get the required data from the vector database and provide response; if matched content is not available, say 'sorry, no information available'") → OpenAI Chat Model + Simple Vector Store as a tool (pointing to "cricket rules database," with a required description "use this tool to get information about cricket") + Embeddings OpenAI (to embed the incoming question).

- Note: multiple separate PDFs/document sets CAN exist in the same n8n project; you aren't limited to one vector database per project.

3. Deep dive: chunk size and overlap

- In the Simple Vector Store's connected **Default Data Loader**, the **Text Splitter** setting controls chunking. Default = "Simple," which splits text into chunks of **1000 characters each, with a 200-character overlap** between consecutive chunks.
- Worked example: for 10,000 characters of text, walked through step by step why the overlap prevents context loss between chunks:
- Chunk 1: characters 1–1000
- Chunk 2: characters 801–1800 (starts 200 characters *before* chunk 1 ended, hence "overlap")
- Chunk 3: characters 1601–2600, and so on.
- The overlap exists so that information spanning a boundary between two chunks (e.g., a sentence cut in half) isn't lost — similar to needing surrounding context to correctly interpret a quote taken out of context (the instructor used an analogy about a media clip being taken out of context, and a movie scene where a character acted on incomplete information).
- A "Custom" text-splitting option also exists if you want to override the simple 1000/200 default, though the default is usually sufficient.
- For the 50-rule cricket PDF (more than 10,000 characters), the actual result was **9 chunks and 9 embeddings** (one embedding per chunk) — confirmed by checking the node's output item count after execution.
- The **Default Data Loader** node is responsible for both loading the raw binary data AND splitting it into chunks; the **Embeddings** node is responsible only for converting each chunk into its vector embedding.

4. Deep dive: embedding batch size

- "Embedding batch size" (default 200) = how many chunks are processed/sent to the embedding model at one time — a performance/throughput setting, not the chunk size itself. It's kept at its default in these demos.

5. Deep dive: are generation-time (pretraining) embeddings the same as RAG chunk embeddings?

- A student asked whether token embeddings used internally during model pretraining (to predict the next token) are the same numbers as the chunk embeddings used in RAG retrieval. Answer: **No — conceptually similar (both are "numeric representation of something"), but functionally different and model/purpose-specific.**
- Pretraining-time embeddings: numeric representation of a *token* (word/sub-word/character), used to help the model predict the next token.
- RAG chunk embeddings: numeric representation of a *chunk* (a large block of ~1000 characters), used purely to perform semantic search/matching against a user's question.

6. Live look at chunks and embeddings inside Supabase

- Using Supabase's table view (from an earlier example built with the smaller, one-page Python course PDF), the instructor showed:
- Only **1 chunk** existed for that short PDF (since it had far fewer than 1000 characters).
- The `content` column shows the actual chunk text; a `metadata` column holds extra info; an `embedding` column holds the corresponding vector — pasted out, it's a long list of decimal numbers representing a **multi-dimensional vector** (not binary 0s/1s), where each number represents one "dimension"/point of the vector.
- This is something n8n's built-in Simple Vector Store does NOT let you inspect directly — one practical reason to consider a third-party vector database like Supabase.

7. Deep dive: the "top-K matched chunks" (limit) setting

- In the Simple Vector Store tool config, a "Number of results" (limit) setting (e.g., 4, later changed to 3) controls how many of the closest-matching chunks are retrieved and sent to the chat model per query.
- Walked through the flow live: a user's question (e.g., "how many balls in an over?") is converted to an embedding → compared against all stored chunk embeddings (internally using something like cosine similarity / semantic distance — the closer the distance, the better the match) → the **top-N most similar chunks** (not random, not just the single best match) are retrieved → these matched chunks + the original question are combined ("augmented") → handed to the chat model, which generates the final natural-language answer.
- Demonstrated changing the limit from 4 to 3 and re-running a query — could visually confirm in the execution log that exactly 3 chunks were retrieved and fed to the chat model as input, and the chat model correctly answered using only those chunks (e.g., ODI = 50 overs).
- Because the chat model reformats/styles the final answer each time, **the exact wording of the response can vary between runs even when the same underlying chunk data is used** — the factual

content stays grounded in the retrieved chunks, but presentation (style/verbosity) is generated fresh by the LLM each time.

- If a question's answer isn't found in the vector database at all (e.g., asking about "how many players in football" when the PDF only covers cricket), the agent correctly responds that no information is available in the vector database, rather than guessing — confirming answers are genuinely coming from the retrieved documents, not the LLM's general knowledge.

8. Brief mention: comparison to uploading a PDF directly into ChatGPT

- A student asked what's the difference between this RAG pipeline and simply uploading a PDF into ChatGPT and asking questions about it. Answer: there's no guarantee ChatGPT will strictly stick to only the PDF's content unless you explicitly instruct it to every time; a dedicated RAG pipeline with its own vector database gives more reliable, consistent, strictly-grounded control over the answer source.

9. Brief mentions of related/advanced concepts (not covered in depth)

- **Neo4j**: a graph-style database (nodes and relationships/branches) that some organizations use instead of flat vector storage for RAG-like retrieval — mentioned as something to potentially compare later, not demonstrated.
- **"Vectorless RAG"**: described by a student as a newer, emerging concept where no vector store/embeddings step is needed at all — noted as potentially suitable for smaller document sets; not demonstrated in this session.
- **Databricks' "AI Parser" function**: mentioned by a student as an example of a platform that bundles the entire chunking/embedding/vector-store pipeline into a single callable function/agent, so you don't have to manually build each pipeline step yourself (contrasted with n8n's more manual, node-by-node approach). Not demonstrated.
- These were flagged as topics the instructor said he would look into and possibly cover later, rather than fully explained in this session.

Key takeaways

- Default n8n chunking behavior: 1000-character chunks with a 200-character overlap between consecutive chunks, to preserve context across chunk boundaries.
- "Embedding batch size" controls how many chunks are processed together at once (a performance setting), separate from chunk size itself.
- The number of matched chunks returned per query (the "limit"/top-K setting) directly controls how much context the chat model receives — changing it changes what's included but not necessarily the correctness, as long as the true answer is among the retrieved chunks.
- Chunk-matching for retrieval is based on semantic similarity (closest vector distance), not random selection or a single best match.
- The same underlying retrieved data can produce differently worded (but factually consistent) answers across runs, because the LLM re-generates the phrasing/style each time.

- Third-party vector databases like Supabase let you directly inspect stored chunks and their embeddings, which n8n's built-in vector store does not expose.
- RAG chunk embeddings and an LLM's internal pretraining token embeddings are conceptually similar (both are numeric representations) but serve different purposes and are not interchangeable.
- A dedicated RAG pipeline gives more reliable, strictly source-grounded answers than simply uploading a document into a general chat interface and hoping it stays on-topic.

Demo 51: Life Lessons on Systems, Consistency, and Delegation (Mindset Session)

This session is a motivational/mindset talk rather than a technical demo. The instructor uses a nature-themed image (spider, rat, elephant, bamboo tree) to teach life lessons about building systems, consistency, ignoring criticism, patience, and delegation — all framed as skills needed to succeed with AI and in life generally. Near the very end, he briefly returns to a technical recap of vector embeddings.

What you'll learn

- Why building repeatable "systems" beats manual daily effort (the spider-web lesson)
- Why staying in a "comfort zone" limits growth (the rat-race lesson)
- Why small, consistent daily effort beats big, random bursts of effort (the rabbit-and-tortoise lesson)
- Why you should ignore criticism and stay focused on your goal (the elephant lesson)
- Why results take time and patience (the bamboo tree lesson)
- Why delegation is essential for scaling yourself and your success (the painter/artist story)
- A recap of how vector embeddings store semantically similar words close together in multi-dimensional space

Step-by-step notes

1. The spider web lesson — build systems, not manual effort

A spider used to manually search for food. One day it built a web instead, and now food (small insects) comes to it automatically. The lesson: instead of "manually" working every day like labor, build a system or pipeline that keeps generating results (money, opportunities) for you even when you're not actively working.

2. The rat race lesson — get out of your comfort zone

A rat running on a wheel (or a person on a treadmill) exerts a lot of effort but never moves forward — it stays in the same "comfort zone" forever. Many people do this with a steady job: same routine every day for decades with no real growth. The lesson: take calculated risks and step outside your comfort zone to access new opportunities.

3. The rabbit and tortoise lesson — consistency beats random big efforts

The rabbit sprints, gets overconfident, and rests; the tortoise moves slowly but never stops — and wins. The lesson: studying/working in big random bursts (e.g., "I'll study 4 hours today, then skip 4 days") doesn't work. A small, consistent daily action (even just 5 minutes) beats occasional huge effort over the long run. The instructor cites his own example: this "5 minutes a day" AI course has been running consistently for over a year without a missed session, and he separately kept a daily habit for 300+ consecutive days and a daily 100-rupee savings habit.

4. The elephant lesson — don't react to criticism

An elephant walking down the road ignores barking dogs and keeps moving toward its destination; if it stopped to respond to every dog, it would waste time and energy and never reach its goal. The lesson: people will always criticize or comment on you, especially as you make progress or become visible. Don't waste time responding — stay focused on your goal. Your value comes from your results, human values, and helpfulness — not from how strongly you argue back.

5. The bamboo tree lesson — patience and long-term thinking

A bamboo seed shows no visible growth above ground for up to 5 years, but during that time it is building extremely strong roots underground. Once ready, it can shoot up 80–100 feet in about a month. The lesson: don't expect instant ("microwave" or "Maggi noodle") results. Big achievements need patience and long-term thinking (planning in terms of years, not days/weeks). Giving up early (like someone quitting the gym after 1 day with no visible results) is a common mistake.

6. Delegation — the mantra for success (the painter/artist story)

An artist named Andy initially did everything himself: arranging canvas, brushes, paints, drawing outlines, filling colors, and cleanup. Step by step, he delegated each low-value task (arranging materials, filling in colors, later even outlining and generating ideas) to trained assistants, keeping only the highest-value work for himself. Over time this scaled into a company with 85,000+ employees. Key points:

- Don't do everything yourself — identify what can be delegated to others.
- Focus only on high-level, high-value activities; delegate low-level ones.
- Calculate your "buyback rate" (your hourly value = annual salary ÷ ~2000 working hours). If a low-value task (e.g., housework, errands) can be done by someone else for far less than your buyback rate, delegating it and using that time for higher-value work is smarter — doing "10-rupee value work" will only ever get you "10-rupee value results."
- Ultimate delegation goal: train people to think, work, and behave like you — effectively creating "clones" of yourself so growth multiplies.

7. Quick technical recap: vector embeddings (brief, near end of session)

- Vector embeddings are stored in multi-dimensional vector space.
- Similar/related words (e.g., apple and banana, both fruits) are stored close together (small "semantic distance"); unrelated words (e.g., car) are stored far apart.
- This semantic distance is what allows AI systems to perform similarity/semantic search — similar to how a supermarket groups similar items (all rice together, all vegetables together) to make searching easy.
- This concept sets up the next day's topic: exploring the Supabase vector database.

Key takeaways

- Build systems/pipelines instead of relying on constant manual daily effort.
- Don't stay stuck in a comfortable routine — small calculated risks lead to bigger opportunities.
- Consistency in small daily actions beats occasional large bursts of effort.

- Ignore criticism; keep your focus on your goal, like an elephant.
- Big results take patience — don't expect instant results; think in terms of years.
- Delegate low-value/low-level tasks so you can focus on high-value activities; this is the key to scaling yourself.
- Vector embeddings store semantically related concepts close together in multi-dimensional space — this is the foundation of semantic/similarity search used in RAG systems.

Demo 52: Building a RAG Workflow in n8n Using Supabase as the Vector Database

This is a hands-on demo showing how to build a full Retrieval-Augmented Generation (RAG) pipeline in n8n using Supabase (instead of n8n's built-in vector store) to store and search document embeddings, then query them through a chat-based AI agent.

What you'll learn

- How to prepare a sample knowledge-base document (using ChatGPT to generate content) for a RAG system
- How to sign up for Supabase and create a project (free tier)
- How to configure the Supabase Vector Store node in n8n (host + service role secret)
- How to create the required database table and functions in Supabase using its provided SQL Quick Start script
- How to build the "ingestion" workflow: download PDF → split into chunks → generate embeddings → store in Supabase
- How to build the "retrieval" workflow: chat trigger → AI agent → Supabase vector store as a tool → embeddings → generate answer
- How system messages control whether the AI agent should only use the RAG database or also fall back to its own pre-trained knowledge
- How tokens are counted and billed for both input and output

Step-by-step notes

1. Prepare the knowledge base document

- The instructor asked ChatGPT to generate "100 interview questions on n8n automation," 10 at a time with answers and learning conclusions, to build a realistic sample knowledge base.
- After collecting all ~100 Q&As (with some copy-paste hiccups), he saved the content as a PDF file and uploaded it to Google Drive.

2. Set up Supabase

693. Go to supabase.com and sign up (free plan is enough for learning).
694. Create a new organization if prompted (free plan allows a limited number of projects).
695. Click "Create new project," set a database password (remember it), and optionally enable automatic RLS (Row Level Security).
696. Wait for the project status to turn "healthy" (green) — this can take a few minutes; red status at first is normal.

3. Build the ingestion workflow in n8n (Project workflow #1)

697. Add a **Manual Trigger** node just to start the workflow.

698. Add a **Google Drive** node → operation "Download file" → select the uploaded PDF. This converts it to binary data.
699. Add a **Supabase Vector Store** node (not the plain "Supabase" relational node — make sure to pick the vector store option). Set the operation to "Add documents to vector store."
700. Create Supabase credentials in n8n using:

- **Host** = the Project URL, found on the Supabase project's main dashboard page.
- **Service role secret** = found under Project Settings → API keys → "Legacy anon/service role API keys" → click "Reveal" then copy.

701. Since no database table exists yet, go to Supabase's own documentation for the Vector Store node ("Quick start for setting up your vector store"), which provides ready-made SQL. Copy the **entire** SQL script (not just the table creation — it also creates the search function needed later).
702. In Supabase, open the **SQL Editor**, paste the SQL, and run it. This creates a `documents` table (and its supporting search function).
703. Back in n8n, the Supabase Vector Store node's "Table Name" field should now show `documents` — select it.
704. Add a **Default Data Loader** node connected to the vector store node:

- Data type: binary data (auto-detected via the PDF's MIME type).
- Text splitting: "Simple" — splits every 1,000 characters with 200 characters of overlap.

705. Add an **Embeddings** node (OpenAI Embeddings) connected to the vector store node.
706. Execute the workflow. Result: 84 chunks/embeddings were generated and automatically inserted into the Supabase `documents` table (confirmed by refreshing the Table Editor in Supabase).

4. Build the retrieval (chat) workflow (Project workflow #2)

707. Add a **Chat Trigger** ("On chat message") node.
708. Add an **AI Agent** node. Configure a system message such as: "Find the required answers from the RAG database. If the answer is not available, respond: 'Sorry, currently we are not in a position to answer your query.'"
709. Connect an **OpenAI Chat Model** to the AI agent (no extra configuration needed beyond the existing credential).
710. Connect a **Supabase Vector Store** node as a **tool** to the AI agent:

- Tool description: e.g., "Use this tool to search for matched chunks and provide answers for the user query" (this description tells the AI agent when to use the tool).
- Table name: `documents` (selected from list — it's mandatory).
- Limit: 4 (the number of top matching chunks it retrieves per query).

711. Connect an **Embeddings** node (OpenAI) to the vector store tool — this is required because user questions must be converted into embeddings before Supabase can compare them against stored chunks (Supabase can't understand raw text directly).

712. Test by asking questions related to the PDF content (e.g., "What is a trigger node?", "What is RAG?") — correct answers came back from the matched chunks. Asking something unrelated (e.g., a cricket question, when the PDF was about n8n interview Q&A) correctly triggered the "not in a position to answer" fallback message, because no matching chunks existed.

5. How the AI agent uses retrieved data

- The agent does not simply copy-paste the retrieved chunk text; it "beautifies"/rephrases the answer for the end user while still basing the substance strictly on retrieved data.
- If the source PDF contains a question but a wrong/incorrect answer, the agent will faithfully return that wrong answer — it only reflects what's in the database, not real-world truth.
- If you want the agent to fall back to its own pre-trained knowledge when no matching chunk is found, you must explicitly say so in the system message (e.g., "If the answer is not available, provide the best matched answer from your pre-trained knowledge"). By default, without that instruction, it will only use the RAG data and say it cannot answer otherwise.

6. Tokens and cost

- Every chat interaction consumes tokens (parts of words), and both input tokens and output tokens are billed separately by the AI provider (similar to how old mobile plans charged for both incoming and outgoing calls).
- Example shown: a single query consumed 2,271 tokens.
- This is why "prompt engineering" (writing efficient prompts) matters for cost optimization.

7. Q&A notes from the session

- n8n supports many databases beyond Supabase for vector storage; Supabase was chosen here as a popular third-party option (open source, built on PostgreSQL + pgvector).
- To avoid duplicate/growing data when re-running ingestion on an updated PDF, either use an option to clear existing embeddings before re-inserting, or create a separate table (e.g., `documents2`) for a different PDF.
- MCP tools exist for many third-party apps (e.g., Jira) — most modern software tools/apps are expected to support MCP integration, and n8n also has many prebuilt nodes for common apps.

Key takeaways

- A RAG (Retrieval-Augmented Generation) pipeline has two parts: an ingestion pipeline (PDF → chunks → embeddings → vector database) and a retrieval pipeline (user question → embedding → similarity search → AI-generated answer).
- Supabase is a popular third-party vector database, built on PostgreSQL with the pgvector extension, that can store both regular relational data and vector embeddings.
- Vector Store nodes need a Document Loader (to chunk source data) and an Embeddings node (to convert both source chunks and user queries into vectors) to function.

- The AI agent's tool description and system message control its behavior — including whether it strictly uses only the RAG database or falls back to general knowledge.
- The RAG agent's answers are only as accurate as the source data — it will faithfully return whatever is in the database, right or wrong.
- API usage (embeddings, chat completions) is billed per token, for both input and output.

Demo 53: Building a Voice-to-Voice AI Agent in n8n Using Telegram and ElevenLabs

This session extends the previous chatbot work into a voice-based agent: users send a voice message on Telegram, the bot transcribes it, generates an AI response, converts that response back into speech, and sends the audio reply back on Telegram — using the ElevenLabs API for speech-to-text and text-to-speech conversion.

What you'll learn

- How to build a full voice-in / voice-out chatbot workflow in n8n
- How to use the Telegram Trigger and Telegram "Get file" nodes to receive and download voice messages
- How to sign up for ElevenLabs and get an API key
- How to use the ElevenLabs node for "Transcribe audio" (speech-to-text) and "Convert text to speech"
- How to select/customize a voice (by ID) for the spoken reply
- How to send the generated audio back to the user via the Telegram "Send audio file" action
- Real-world troubleshooting of a live multi-user workflow (chat ID issues, intermittent errors)

Step-by-step notes

1. Why build this (business context)

The instructor explained that this kind of workflow supports business automation use cases — e.g., a hospital or business wanting a voice-enabled chatbot, or automatic email reply/summary systems — replacing manual work with automated pipelines.

2. Set up the trigger

713. Add a **Telegram Trigger** node, event "On message received."
714. Configure Telegram credentials using the access token obtained from Telegram's **BotFather** (start a chat with BotFather, use "I want to create a new bot," and it returns an access token and bot name) — this was covered in an earlier session.
715. Send a voice message to the bot on Telegram to test — the trigger fires and captures the message data (as binary/audio, since it's a voice note).

3. Download the voice file

716. Add a second **Telegram** node, resource "File," operation "Get file."
717. Set the File ID field using the file ID available from the trigger's output data.
718. Executing this step downloads the actual binary audio file into the workflow.

4. Convert speech to text (ElevenLabs)

719. Add an **ElevenLabs** node (first use requires installing/enabling this community node in n8n — click "install" when prompted).
720. Set resource "Speech," operation **"Transcribe audio or video."**
721. Set up ElevenLabs credentials:

- Go to elevenlabs.io (11labs.io), sign up (free tier includes 10,000 free credits).
- Go to the bottom-left "Developers" section → "API keys" → "Create key."
- When restricting the key's permissions, either select "Speech to text" and "Text to speech," or simplify by choosing "Unrestricted" (less secure, but simpler for testing).
- Copy the key and paste it into the n8n credential (same pattern as any other API key setup, e.g., OpenAI).

722. Set the binary property to `data` (the default name for binary input).
723. Execute — the audio is converted into text output (visible in the node's output panel).

5. Send the transcribed text to an AI agent

724. Add an **AI Agent** node, feeding it the transcribed text as the user message.
725. Add a system message (e.g., "You are a helpful assistant. Provide response in a concise way.") — concise responses matter here because longer replies mean longer/costlier generated audio.
726. Connect an **OpenAI Chat Model** node (no extra config needed if credentials already exist).
727. Execute — the agent returns a text response.

6. Convert the AI's text response back to speech (ElevenLabs)

728. Add another **ElevenLabs** node, this time with operation **"Convert text to speech."**
729. Set the text field to the AI agent's output text.
730. Choose a voice:

- Either select from the built-in "voice from list" dropdown (many named voices are available, e.g., celebrity-style or generic voices).
- Or browse ElevenLabs' "Voices" section (left sidebar) → "Trending voices" → filter by language/accent/use case, then copy the desired voice's ID (via the "..." menu → "Copy voice ID") and paste it into the "Voice" field using the "By ID" option instead of "From list."

731. Execute — this generates an MP3 binary audio file as output.

7. Send the audio back to Telegram

732. Add a final **Telegram** node, action **"Send an audio file."**
733. Enable "binary file" and confirm it picks up the generated audio.
734. Set the **Chat ID** — copy it from the Telegram Trigger node's schema/output (this ensures the reply goes to the same chat the message came from).
735. Execute — the bot sends the audio reply back to the original Telegram chat.

8. Making the bot live

- Instead of manually running "Execute Workflow" every time, the workflow can be **activated/published** so it runs automatically whenever a new voice message arrives (tested live by having multiple students message the bot).
- Live testing surfaced some real-world issues: occasional "bad request" / file-not-found errors on the "Get file" step when multiple people used the bot around the same time (possibly due to concurrency or the file no longer being available); a text-only message (not a voice note) did not get a response because the workflow was built specifically to expect an audio file.
- The instructor noted that text-to-speech and speech-to-text calls are **chargeable** via ElevenLabs, so testing should be done by only one or two people at a time, not everyone simultaneously.

Key takeaways

- A voice-to-voice AI agent chains together: receive voice → download binary audio → speech-to-text → AI agent (text in/text out) → text-to-speech → send audio back.
- ElevenLabs is the third-party service used here for both speech-to-text ("Transcribe audio or video") and text-to-speech ("Convert text to speech") conversion, via its own API key.
- Telegram's Chat ID (from the trigger node) is essential to route replies back to the correct chat.
- Keep the AI agent's system message instructing "concise" responses when doing voice output, since longer text means longer, more expensive generated audio.
- Live/production workflows can behave differently from single-user testing — plan for concurrency issues and test carefully with a small number of users first.

Demo 54: Auto-Generating n8n Workflows with n8n's Built-In AI Assistant

This session shows how to use n8n's own built-in AI feature to auto-generate workflows just by describing your requirement in plain English — from a simple student registration form workflow up to an ambitious (and only partially working) automated video-creation pipeline.

What you'll learn

- Why consistency (even doing something small every single day) produces much better long-term results than sporadic, high-effort bursts — reinforced with the instructor's personal example
- How to access and use n8n's built-in "AI" assistant/chat panel inside the workflow editor
- How to describe a requirement in natural language and let n8n generate a starter workflow (nodes + rough configuration)
- How to review, customize, and fix the AI-generated workflow (it typically gets you ~90% there)
- How to use the AI panel itself to make further modifications to the workflow ("vibe coding" style)
- The limits of this approach for very ambitious workflows, and how to keep improving them using ChatGPT for node-by-node explanations

Step-by-step notes

1. Consistency mindset recap

The instructor described a personal experiment: instead of cancelling a class when his preparation wasn't "up to the mark," he decided to deliver whatever partial content he had prepared and continue the next day. This maintained an unbroken daily streak (120+ days at the time of recording, other habits sustained 300+ days), and he found that consistency mattered far more than how much was covered in any single session. The takeaway: commit to small daily actions (even just 2 minutes) rather than skipping days — stopping is "the starting point of failure."

2. Accessing n8n's AI assistant

736. In the n8n workflow editor toolbar, locate the icons: open notes, command bar, sticky note, focus panel, and the **n8n AI** option (the assistant/chat icon).
737. Click it to open a chat-style panel where you can type your workflow requirement in plain English.

3. Example 1 — Simple requirement: student registration form

738. Typed a requirement describing: a form with fields (name, email, mobile number, course), which on submission should (a) store the data in a Google Sheet, and (b) send a confirmation email to the student.
739. n8n's AI thought through the requirement ("getting suggested nodes, searching nodes, getting node definitions") and generated a complete workflow automatically, including a sensibly named

workflow ("Student registration...") and appropriate nodes (form trigger, Google Sheets, Gmail/send message).

740. The instructor emphasized: the generated workflow is a **template**, roughly 90% complete — you're expected to review, customize field names/options, connect your own credentials, and fix small issues (similar to "vibe coding").

741. Customizations demonstrated: editing form field options (e.g., changing course dropdown options to Generative AI, Data Science, Agentic AI, Prompt Engineering); connecting Google Sheets credentials; populating column mappings (name, email, mobile number, course) since they weren't auto-populated; fixing a Gmail node error about missing credential access by re-sharing/reconnecting credentials; fixing a "cannot read properties of undefined" error by correcting how the email body referenced field values (e.g., using the right JSON path from the Google Sheet or form data).

742. Result: successfully tested end-to-end — form submission → row appended to Google Sheet → confirmation email sent.

4. Example 2 — Ambitious requirement: AI-generated poster + social media posting

- Typed a bigger requirement: pick a random topic in "Agentic AI," create a poster using Google's "Nano Banana" (Gemini image generation) tool summarizing the topic, then post it to YouTube, Instagram, and Facebook.
- The AI assistant again generated a starter workflow with suggested nodes and a checklist of steps needed to finish it (e.g., "add missing credentials," "update API endpoint" for Instagram/Facebook posting).
- Key point: nodes generated this way may be **renamed** to reflect their actual purpose (e.g., a Gemini/Google node relabeled as a "Nano Banana" node) — you can ask the AI assistant "What are the original names of each node used in this workflow?" to clarify.

5. Example 3 — Very ambitious requirement: auto-generate a 150-minute video and upload to YouTube

- Typed an intentionally ambitious requirement: given a topic, generate a full video script (broken into scenes), generate images and videos per scene, combine them into a single 150-minute video, and publish it to YouTube — without worrying about whether it's actually feasible.
- The AI assistant produced a workflow skeleton with nodes such as: Start → Generate Video Script (via an LLM, e.g., ChatGPT/Gemini, with a system prompt instructing it to break the script into exact-duration scenes as a JSON array with scene number/duration/narration/visual description) → Parse Scenes (structured parser) → Split Scenes → Generate Scene Image → Generate Scene Video → Download Scene Video → Aggregate Scenes → Combine Videos → Prepare YouTube Upload → Upload to YouTube.
- To understand an unfamiliar generated workflow, the instructor demonstrated taking a screenshot of the whole workflow and pasting it into ChatGPT, asking it to "explain each node in detail with its functionality" — ChatGPT was able to describe each node's purpose from the screenshot alone.

- When testing, several nodes failed (e.g., the "Start" node needed to be replaced with a Chat Trigger to actually accept a topic input; the "Generate scene image" step failed with a "bad request" because the visual description field mapping wasn't correctly wired) — illustrating that AI-generated workflows for complex, novel requirements need real debugging and node-by-node investigation, not just one-click execution.
- Practical limitation raised by a student: models like Google Gemini can only generate short video clips (e.g., ~8 seconds) at a time, so a 150-minute video would still require significant additional engineering, not just this template.

6. Key working principle taught

- Whether the requirement is small or large, the process is the same: describe it in plain English to the AI assistant, get a starter workflow, then dig in and refine node-by-node.
- This is especially useful when you don't know which nodes to use for an unfamiliar client requirement — the AI-generated skeleton gives you a starting point and confidence to begin, even if you eventually rebuild parts manually.
- Other workflow automation tools mentioned as alternatives to n8n: **Zapier** and **Make**. The instructor noted that skills transfer well between such tools once you know one well.
- Google Sheets was reused here as a lightweight "database" for storing captured form data (an alternative to a real database).

Key takeaways

- Consistency (small daily effort without skipping) compounds into much bigger results than occasional large bursts of effort.
- n8n has a built-in AI assistant that can auto-generate a working workflow skeleton from a plain-English description — useful for both simple and highly ambitious requirements.
- Generated workflows are typically ~90% complete "templates" — expect to review, customize field options, wire up credentials, and fix mapping/logic errors yourself.
- For very complex/novel requirements, the generated skeleton is best used as a learning aid and starting point, not a guaranteed working solution — screenshotting the workflow and asking ChatGPT to explain each node is a practical way to understand and debug it.
- Both n8n's own automation platform subscription AND separate paid subscriptions for any third-party AI tools/APIs used inside the workflow (e.g., OpenAI, Gemini) are typically required for production use.

Demo 55: Self-Hosting n8n on Hostinger, Plus Community Forum Resources

This session addresses a practical production concern: n8n's free cloud account only lasts 14 days, and the official paid n8n cloud plan is relatively expensive. The instructor demonstrates self-hosting n8n cheaply on Hostinger as an alternative, and also introduces the official n8n community forum as a support resource. It ends with live troubleshooting of a Google Sheets "create sheet with columns" question from a student.

What you'll learn

- Why free n8n accounts (14-day limit) and official paid n8n cloud plans (relatively costly) aren't ideal for real production use
- How to self-host n8n cheaply using Hostinger's VPS hosting
- That n8n can also be self-hosted on AWS, Azure, GCP, or other cloud/VPS providers using Docker
- How the self-hosted (Hostinger) n8n instance works identically to the standard n8n.cloud dashboard, just at a different URL
- How to find help via the official n8n community forum (announcements, questions, tutorials sections)
- A live troubleshooting example around creating a Google Sheet with named columns from within a workflow

Step-by-step notes

1. The problem with free and paid n8n cloud accounts

- A free n8n.io account only stays active for **14 days**; after that, a new account/email would be needed to continue, which is impractical for a workflow meant to run continuously (365 days, 24/7) for a real business (e.g., auto-processing inquiry emails and sending welcome replies).
- The official n8n paid cloud plans start around **20 euros/month** (≈ ₹2,200/month) up to higher tiers (₹50/₹667+ equivalents for business plans), which can add up to roughly ₹24,000–₹30,000/year — considered too costly for many learners/small projects.

2. Self-hosting alternative: Hostinger

- For lower-cost production use, n8n workflows can be **self-hosted** on cloud/VPS platforms — AWS, Azure, and GCP are all possible, but the instructor specifically demonstrated **Hostinger**, described as the most commonly used self-hosting platform for n8n.
- Steps to set up:

743. Search "Hostinger n8n" and go to Hostinger's self-hosted n8n offering page (described as "secure and scalable automation," a free-to-use no-code workflow tool you connect your apps to).
744. Choose a VPS plan (e.g., **KVM 1**: 1 CPU core, 4GB RAM, 50GB disk space, 4TB bandwidth). Pricing shown: roughly ₹599–₹799/month depending on plan, but the price drops significantly with a longer

commitment — e.g., ~₹599/month when billed for a **24-month** period (versus ~₹1,178 all-in for just 1 month).

745. Sign in with Gmail, fill in billing details (name, phone number, address), and pay via UPI, card, PayPal, or EMI.
746. After payment, Hostinger provisions a VPS running **Ubuntu** (a Linux flavor) and automatically installs the **n8n** app you selected during setup.
747. You'll receive a confirmation email with your VPS hostname, IP address, and panel access URL. Click the panel access URL to reach your n8n dashboard — it looks and works exactly like the standard n8n.cloud interface.

- Key distinguishing detail: comparing URLs shows the difference — a normal n8n.cloud account has a URL like `yourname.app.n8n.cloud`, while the Hostinger-hosted instance has a URL like `n8n.srv####.hstgr.cloud` (or similar) — confirming it's running on Hostinger's infrastructure instead of n8n's own cloud.
- Once hosted, workflow-building is identical to the standard n8n interface — the instructor rebuilt a simple example (student inquiry form → AI agent drafts a reply email based on the selected course → send via Gmail) to demonstrate this. One difference noted: the Hostinger-hosted instance did **not** have the built-in n8n AI assistant feature available (unlike the n8n.cloud dashboard shown in Demo 54).
- Setting up Gmail credentials in a self-hosted instance can require additional OAuth details (callback/redirect URL, client ID, client secret from Google Cloud Console) rather than just a simple "Sign in with Google" — the instructor suggested asking ChatGPT for step-by-step help getting these values if needed, since it's only a one-time setup.
- A student pointed out you could similarly self-host n8n on your own local machine to run 24/7 and avoid hosting costs, though production live URLs might need extra configuration for that approach.

3. n8n's official community forum

- Accessible via n8n.io → "Community" → "Community Forum."
- Sections include: **Announcements** (platform updates), **Questions** (post problems and get help — e.g., a thread titled "Help me build my workflow" had 59 replies and 10.5k views), **Tutorials** (structured beginner/advanced video courses covering nodes, triggers, webhooks, APIs, debugging, error handling, etc.), and category-specific discussions (e.g., self-hosting issues, billing issues).
- This is described as **platform-level support** directly from the n8n team/community (as opposed to third-party sites like Stack Overflow), making it a good first place to search or post when stuck on a workflow problem.

4. Live troubleshooting: adding named columns to a newly created Google Sheet

A student had a workflow where a "Create Google Sheet" node created a new spreadsheet each time the flow ran, but didn't know how to specify column headers (name, email, course, etc.) at creation time — the create node alone doesn't expose a straightforward "add columns" option, and using "Append Row" without predefined columns leads to unmapped/missing headers.

- The instructor's guidance during the live debugging: if you only need one persistent sheet (not a new sheet every run), remove the "Create Google Sheet" node from the automated flow, manually create the Google Sheet once (with the column headers you want), and then just use "Append Row" (or "Update Row") pointing at that existing, manually-created sheet/document ID for every subsequent run.
- If you do need to programmatically create sheets with specific columns, it requires either an additional node/step or manual pre-setup — this was left as something to investigate further ("we will check, don't worry") rather than fully resolved live in this session.

Key takeaways

- Free n8n accounts expire after 14 days and official paid n8n cloud plans can be expensive for production use — self-hosting is a practical, much cheaper alternative.
- Hostinger is a commonly used, low-cost VPS provider for self-hosting n8n (Ubuntu + n8n pre-installed via their control panel); other options include AWS, Azure, GCP, or your own local machine.
- A self-hosted n8n instance behaves identically to n8n.cloud for building workflows, though some features (like the built-in AI assistant) may not be available, and some credential setups (like Gmail OAuth) may need extra manual configuration.
- The official n8n community forum is a strong first resource for troubleshooting — it has dedicated sections for announcements, Q&A, and structured tutorials.
- For a Google Sheet that needs specific named columns, it's often simplest to create and label the sheet manually once, then have your workflow only append/update rows to that existing sheet rather than re-creating it on every run.